

EXPEDIENTE

DIREÇÃO UNICESUMAR

Reitor Wilson de Matos Silva **Vice-Reitor** Wilson de Matos Silva Filho **Pró-Reitor de Administração** Wilson de Matos Silva Filho **Pró-Reitor Executivo de EAD** William Victor Kendrick de Matos Silva **Pró-Reitor de Ensino de EAD** Janes Fidélis Tomelin **Presidente da Mantenedora** Cláudio Ferdinandi

NEAD - NÚCLEO DE EDUCAÇÃO A DISTÂNCIA

Diretoria Executiva Chrystiano Mincoff, James Prestes, Tiago Stachon **Diretoria de Design Educacional** Débora Leite **Diretoria de Graduação e Pós-graduação** Kátia Coelho **Diretoria de Cursos Híbridos** Fabricio Ricardo Lazilha **Diretoria de Permanência** Leonardo Spaine **Head de Curadoria e Inovação** Tania Cristiane Yoshie Fukushima **Head de Produção de Conteúdo** Franklin Portela Correia **Gerência de Contratos e Operações** Jislaine Cristina da Silva **Gerência de Produção de Conteúdo** Diogo Ribeiro Garcia **Gerência de Projetos Especiais** Daniel Fuverki Hey **Supervisora de Projetos Especiais** Yasminn Talyta Tavares Zagonel **Supervisora de Produção de Conteúdo** Daniele C. Correia

Coordenador(a) de Conteúdo

Flávia Lumi Matuzawa

Projeto Gráfico e Capa

Arthur Cantareli, Jhonny Coelho e Thayla Guimarães

Editoração

Matheus Silva de Souza

Design Educacional

Amanda Peçanha

Revisão Textual

Sarah Mariana Longo Carrenho Cocato

Ilustração

Wellington Oliveira
Natalia de Souza Scalassara

Fotos

Shutterstock

FICHA CATALOGRÁFICA

C397 **CENTRO UNIVERSITÁRIO DE MARINGÁ.**

Núcleo de Educação a Distância. **LUZ**, Carlos Danilo, **FLORINDO**, Rafael Alves.

Programação Back End II.

Carlos Danilo Luz, Rafael Alves Florindo.

Maringá - PR: Unicesumar, 2021. Reimpresso em 2022.

240 p.

"Graduação - EaD".

1. Programação 2. Back End 3. EaD. I. Título.

Impresso por:

CDD - 22 ed. 005.1

CIP - NBR 12899 - AACR/2

ISBN 978-65-5615-372-8

Bibliotecário: João Vivaldo de Souza CRB- 9-1679



NEAD - Núcleo de Educação a Distância

Av. Guedner, 1610, Bloco 4 - Jd. Aclimação - Cep 87050-900 | Maringá - Paraná

www.unicesumar.edu.br | 0800 600 6360

Neste mundo globalizado e dinâmico, nós trabalhamos com princípios éticos e profissionalismo, não somente para oferecer educação de qualidade, como, acima de tudo, gerar a conversão integral das pessoas ao conhecimento. Baseamo-nos em 4 pilares: intelectual, profissional, emocional e espiritual.

Assim, iniciamos a Unicesumar em 1990, com dois cursos de graduação e 180 alunos. Hoje, temos mais de 100 mil estudantes espalhados em todo o Brasil, nos quatro campi presenciais (Maringá, Londrina, Curitiba e Ponta Grossa) e em mais de 500 polos de educação a distância espalhados por todos os estados do Brasil e, também, no exterior, com dezenas de cursos de graduação e pós-graduação. Por ano, produzimos e revisamos 500 livros e distribuimos mais de 500 mil exemplares. Somos reconhecidos pelo MEC como uma instituição de excelência, com IGC 4 por sete anos consecutivos e estamos entre os 10 maiores grupos educacionais do Brasil.

A rapidez do mundo moderno exige dos educadores soluções inteligentes para as necessidades de todos. Para continuar relevante, a instituição de educação precisa ter, pelo menos, três virtudes: inovação, coragem e compromisso com a qualidade. Por isso, desenvolvemos, para os cursos de Engenharia, metodologias ativas, as quais visam reunir o melhor do ensino presencial e a distância.

Reitor

Wilson de Matos Silva

— “ —

Tudo isso para honrarmos a nossa missão, que é promover a educação de qualidade nas diferentes áreas do conhecimento, formando profissionais cidadãos que contribuam para o desenvolvimento de uma sociedade justa e solidária.

— ” —



Esp. Carlos Danilo Luz

Mestrado em Ciência da Computação pela Universidade Estadual de Maringá – UEM (2020). Especialista em Educação à Distância e Tecnologias Educacionais (2016), em Gestão Estratégica de Pessoas (2017) e em Docência no Ensino Superior (2017). Graduado em Redes de Computadores pela Unicesumar (2009) e em Análise e Desenvolvimento de Sistemas (2018). Professor universitário atuante em outras instituições de Ensino Superior. Professor autor, formador e mediador do Ensino a Distância – EAD da Unicesumar, desde 2015, nos Cursos de TI: Sistemas para Internet, Análise e Desenvolvimento de Sistemas, Gestão da Tecnologia da Informação e Engenharia de Software. Conta com experiência de nove anos em Desenvolvimento de Sistemas Web e Projetos.

<http://lattes.cnpq.br/7063667454769099>

Me. Rafael Alves Florindo

Mestrado em Gestão do Conhecimento nas Organizações, na linha de pesquisa Educação e Conhecimento, pela PPGGCO do Centro Universitário de Maringá – Unicesumar (2017). É especialista em Desenvolvimento de Sistema para Web pela Universidade Estadual de Maringá – UEM (2008). Graduado em Ciência da Computação pela Faculdades Adamantinenses Integrada – FAI. Pós-graduando em Docência no Ensino Superior: Tecnologias Educacionais e Inovação e Banco de Dados pela Unicesumar. Professor – Ensino Técnico pela SEED-PR desde 2009. Professor autor, formador e mediador do Ensino a Distância – EAD da Unicesumar, desde 2014, nos Cursos de TI: Sistemas para Internet, Análise e Desenvolvimento de Sistemas, Gestão da Tecnologia da Informação e Engenharia de Software. Professor presencial da Faculdade Unifamma, desde 2019, no curso de Engenharia de Software e Sistema de Informação.

<http://lattes.cnpq.br/7246554526271622>

PROGRAMAÇÃO BACK END II

Caro(a) estudante,

Atualmente, vivemos um novo normal com usuários mais consumistas de conteúdos da internet e, também, com a evolução e a migração de diversos sistemas para a web. Algumas empresas já possuem sistemas de gestão totalmente on-line, outras trabalham com sistemas híbridos — parte off-line na empresa e parte on-line para a integração com outros recursos e com a equipe.

As nossas necessidades dos sistemas de informações estão, cada vez mais, baseadas na web, por isso, devemos nos empenhar em como lidar com o Back End. A parte visual de um projeto pode ser relevante para o usuário, mas, caso o Back End apresente falhas de programação, isso será um problema considerável.

Este material foi pensado para você, para fornecer subsídios sobre a programação Back End PHP, trazendo diversos conceitos fundamentais da disciplina que serão úteis em seu dia a dia no futuro. Este livro está dividido em cinco unidades. Abordaremos desde os conceitos básicos do paradigma de Orientação a Objetos com PHP e recursos complementares, como o modelo MVC e alguns frameworks. O objetivo deste livro é fomentar em você o desejo de continuar com os estudos desse conteúdo, que é de grande valia para as empresas.

Logo na primeira unidade, estudaremos os princípios do paradigma de Orientação a Objetos, como interpretar um modelo UML, criar classes, identificar atributos e métodos, compreender os conceitos de métodos que possibilitam o acesso a determinados atributos, tudo isso aplicado na prática com a linguagem de programação PHP.

Na segunda unidade, aprofundaremos nossos estudos sobre as funcionalidades e oportunidades possíveis em se trabalhar com PHP OO, estudando os modificadores de acesso default, public, protected e private, compreendendo as características essenciais de qualquer classe, e, também, tudo isso sendo aplicado na prática e analisando um estudo de caso.

Ao chegarmos na terceira unidade, trabalharemos com o relacionamento das classes, visto que elas não trabalham sozinhas. Um destaque interessante para esta unidade seria sobre a oportunidade de poder trabalhar com outras classes e componentes para aproveitar conteúdos já existentes, assim aplicando um conteúdo muito interessante de Engenharia de Software: o reuso de módulos e funcionalidades, implementação que é muito utilizada no mercado de trabalho.

Em nossa quarta unidade, elevaremos o nosso conhecimento ao trabalharmos com um banco de dados relacional, afinal, para que ser um sistema, se este não é útil para manipular os dados? Nesta unidade, reveremos os conceitos de banco de dados relacionais e os aspectos da linguagem SQL, tudo sendo implementado com PHP e extensões nativas da linguagem como MySQLi e PDO.

Por fim, em nossa última unidade, compreenderemos que um sistema, seja ele off-line ou on-line, encontra-se em um ecossistema complexo, mas com diversas oportunidades, arquiteturas e ferramentas que podem ser utilizadas para auxiliar o desenvolvedor na construção do projeto de sistema. Desta forma, iniciamos a nossa busca por conhecimento, aplicando aspectos relevantes sobre o Back End de nossos sistemas Web.

Espero que aproveitem ao máximo o material. Bom estudo!

ÍCONES



pensando juntos

Ao longo do livro, você será convidado(a) a refletir, questionar e transformar. Aproveite este momento!



explorando ideias

Neste elemento, você fará uma pausa para conhecer um pouco mais sobre o assunto em estudo e aprenderá novos conceitos.



quadro-resumo

No fim da unidade, o tema em estudo aparecerá de forma resumida para ajudar você a fixar e a memorizar melhor os conceitos aprendidos.



conceituando

Sabe aquele termo ou aquela palavra que você não conhece? Este elemento ajudará você a conceituá-lo(a) melhor da maneira mais simples.



conecte-se

Enquanto estuda, você encontrará conteúdos relevantes on-line e aprenderá de maneira interativa usando a tecnologia a seu favor.



Quando identificar o ícone de QR-CODE, utilize o aplicativo **Unicesumar Experience** para ter acesso aos conteúdos on-line. O download do aplicativo está disponível nas plataformas  Google Play  App Store

CONTEÚDO

PROGRAMÁTICO

UNIDADE 01

10

PROGRAMAÇÃO
ORIENTADA A
OBJETOS

UNIDADE 02

44

PILARES DA
PROGRAMAÇÃO
ORIENTADA A
OBJETOS

UNIDADE 03

90

RELACIONAMENTOS

UNIDADE 04

140

MANIPULAÇÃO DE
DADOS ENTRE PHP
COM AS APIS (MYSQLI,
MYSQLI PREPARED E
PDO)

UNIDADE 05

201

MODELOS DE
APLICAÇÃO DO BACK
END

FECHAMENTO

230

CONCLUSÃO GERAL

UNIDADE



PROGRAMAÇÃO ORIENTADA A OBJETOS

PROFESSORES

Me. Rafael Alves Florindo

Esp. Carlos Danilo Luz

PLANO DE ESTUDO ▼

A seguir, apresentam-se as aulas que você estudará nesta unidade. • O Que é Programação Orientada a Objetos • Conceitos de Classe, Atributos, Métodos • Métodos Construtores e Destrutores • Modificadores de Acesso Public, Private e Protected • Aplicando na Prática.

OBJETIVOS DE APRENDIZAGEM ▼

Estudar conceitos pertinentes ao paradigma de Orientação a Objetos em relação com a linguagem PHP • Entender e implementar o que é uma classe, o que é atributo e métodos na linguagem PHP • Estudar os métodos que inicializam e encerram as classes • Compreender os conceitos de métodos que possibilitam o acesso a determinados atributos protegidos ao meio externo • Levar o aluno à prática de uma classe envolvendo os conceitos aprendidos nesta unidade.



Olá, prezado(a) aluno(a)! Seja bem-vindo(a) à nossa primeira unidade.

Geralmente, escutamos dizer que o principal fator responsável pela aceitação de um sistema ou por ele estar de acordo com a qualidade aceitável é o Front End, mas o Back End é que faz todo o trabalho pesado. Podemos abstrair e comparar um sistema, seja ele web ou não, a um carro: o Front End é toda a parte estética do veículo, e o Back End seria o motor e todas as demais equipamentos necessários para que o carro possa andar.

Sendo assim, podemos dizer que, para o sucesso de um projeto, depende-se muito da junção do Back End com o Front End. Nosso foco, nesta unidade, é trabalhar com os conceitos de linguagem de programação PHP, com a aplicação dos conceitos de Orientação a Objetos, visto que este paradigma de linguagem de programação se tornou muito forte no desenvolvimento de sistemas web.

Um fator relevante, nesse contexto, é o fato de que algumas pessoas adotam frameworks para o desenvolvimento de aplicações Web. Entretanto, um dos objetivos desta disciplina é trazer a você uma visão de programação PHP que poderá ser utilizada com o uso de um framework ou não. Por este motivo, nesta unidade, você será levado a compreender conceitos básicos sobre programação orientada a objetos, assim como os elementos básicos desse paradigma de linguagem de programação.

No decorrer desta unidade, serão apresentados a você os conceitos associados a aplicações práticas em linguagem de programação PHP, sendo possível assimilar como os conceitos apresentados são aplicados na prática. Trabalharemos desde os passos iniciais de sobre a composição de classes, métodos, modificadores de acesso, dentre outros conceitos básicos sobre a linguagem de programação orientada a objeto (OO) associada à linguagem Web PHP.

Acreditamos que esta unidade será um marco para sua carreira como desenvolvedor web, levando a você um aprofundamento sobre a aplicação e representação da linguagem OO nas atividades propostas, podendo dizer que será diferente de tudo que já viu e que, com certeza, gostará muito.

1 O QUE É PROGRAMAÇÃO ORIENTADA A OBJETOS



Olá, alunos(as)! Apresentaremos os conceitos introdutórios do paradigma de programação orientado a objetos, a qual é vista pelo mercado de trabalho com um método de desenvolvimento atual e extremamente usual pelas empresas de desenvolvimento de sistemas, aplicando-a no desenvolvimento de software para web.

Provavelmente, você já ouviu falar que existem diversos paradigmas de programação, sejam eles: funcional, eventos, estruturado, orientado a objetos, entre outros paradigmas existentes. Contudo, você sabe qual é a diferença entre eles? Neste livro, abordaremos brevemente o paradigma estruturado e o orientado a objetos.

Para explicar o que é um paradigma de programação, seja ele de forma estruturada ou orientada a objetos, precisamos definir e explicar o que é paradigma.

Para Bezerra (2007), o termo paradigma pode ser compreendido como um modelo ou padrão a ser seguido para a resolução de um problema. No desenvolvimento de software, não existe uma receita de bolo – você, com a sua expertise e o seu raciocínio lógico, construirá uma solução adequada para cada problema.

Na visão de Dall'Oglio (2009, p. 86), o termo paradigma “representa toda uma filosofia para a construção de sistemas”, buscando, assim, retratar a visão do mundo real como um sistema de objetos cooperativos e colaborativos. O mundo que o autor cita realmente é o mundo real, onde temos objetos que possuem uma forma, mas com características e comportamentos diferentes.

A partir dessas afirmações, podemos deduzir que o termo paradigma, no mundo computacional, significa uma forma de abordar um problema. Para cada problema a ser resolvido, podemos ter uma metodologia diferente, um paradigma diferente.

Como saber, porém, qual paradigma utilizar?

A resposta é: “depende”. Isso mesmo: para cada situação, é necessário analisar os benefícios e os problemas que podem surgir. Para exemplificar, imagine que você precise comprar um computador. Ao chegar na loja, o vendedor lhe perguntará: qual é a finalidade do computador para você? Dependendo da sua resposta, ele oferecerá uma configuração específica.

O desenvolvimento de software com a abordagem estrutural consiste na construção de um sistema sequencial. Assim, os programas são criados baseados em quais funções, procedimentos e variáveis são necessárias para resolver um problema. Esse paradigma possui uma estrutura básica com instruções, decisões e interações de forma sequencial. Nessa forma de programar, é possível ainda criar funções/módulo, ou seja, a divisão do programa em módulos menores (YOURDON, 1990).

Além de programar em módulo, a abordagem estrutural permite a reutilização de código, ou seja, não é necessário escrever os códigos dessas funções todas as vezes que for necessário, bastando apenas chamá-los para serem executados novamente. Mesmo assim, ainda obedece a ordem sequencial lógica do programa, ou seja, uma instrução após a outra.



pensando juntos

Ao aplicarmos o paradigma de Orientação a Objetos em um projeto, subentendemos que é possível implementar o conceito de reuso de software sobre as classes e métodos já utilizados em outros projetos.

Já o desenvolvimento de software com a abordagem orientada a objetos consiste na construção de módulos independentes ou objetos que podem ser facilmente substituídos, modificados e reutilizados. O programador deve conseguir abstrair do problema coisas que podem ser representadas como objetos, como, por exemplo: se deseja criar um programa para saber se um candidato está ou não aprovado em um concurso, o programa deverá criar objetos para representar o candidato e o concurso.

A Orientação a Objetos tem, como principal característica, a forma natural de tratar a realidade, pois considera que o mundo real é formado por objetos. Os objetos são definidos em classes – forma com características e comportamentos –, em que cada objeto criado é denominado de instância (SOMMERVILLE, 2011).

A partir do surgimento da programação orientada a objetos, houve uma mudança no enfoque para desenvolver um sistema, enquanto, na programação estruturada, havia, nitidamente, uma visão sequencial e bem dividida, como os programas que executam determinados processos e tratam determinados dados.

**conecte-se**

Para conhecer um pouco mais sobre o paradigma de linguagem orientada a objetos, acesse o texto disponível em:



A Orientação a Objetos é um estilo de programação que permite aos desenvolvedores agruparem tarefas semelhantes em classes. Passamos a visualizar classes como responsáveis por atributos contidos nelas, com operações criadas para tratá-los, e a execução das atividades contidas nas classes dos sistemas passam a depender da interação dessas classes (SOMMERVILLE, 2011).

Pois bem, acredito que este nosso tópico foi bem produtivo e revigorante. Agora que já sabemos os conceitos básicos sobre a programação orientada a objetos, podemos alegar que esse será um belo começo para nossos estudos e para sua carreira de desenvolvedor.



2 CONCEITOS DE CLASSE, ATRIBUTOS, MÉTODOS

Agora que já conhecemos o que é programação orientada a objetos e para o que ela serve, aprofundaremos nos conceitos em suas particularidades – objetos, classes, métodos e atributos – que a diferem dos outros paradigmas. Pensando em apresentar a você, aluno(a), os primeiros passos na compreensão e implementação dos conceitos de programação orientada a objetos, associado a diagramas UML, você será apresentado a estruturas básicas de uma classe em linguagem PHP OO. Venha conosco neste momento de estudos!

Classe

A classe é uma estrutura que definirá um tipo de dado, podendo conter atributos e, também, métodos para manipular os atributos da classe. Uma classe representa a abstração de um conjunto de objetos do mundo real que possui comportamentos e características comuns (DAL'OGGIO, 2009).

Podemos exemplificar uma classe como um conjunto de características de um ser humano, por exemplo: gênero, peso, altura, nome, e-mail, data de nascimento etc. Essas características se transformam em atributos da classe ou variáveis de classe. A classe é um modelo para criação dos objetos de um mesmo domínio, ou seja, cada classe é responsável por um assunto diferente e possui responsabilidade sobre si mesmo por meio de métodos.

A seguir, na Figura 1, veremos a representação gráfica de uma classe de acordo com o padrão UML.

Note que, ao modelar uma classe de acordo com o padrão UML, os membros da classes são declarados na ordem: nome da classe, depois, os atributo mediante o seu tipo e visibilidade e, em seguida, os métodos mediante o seu tipo e visibilidade e

tipo de retorno. Ao analisar a Figura 1, percebemos que, à frente de cada atributo e método, temos um sinal de +, isso identifica que tais elementos têm o tipo de visibilidade *Public*.

A seguir, apresentamos a sintaxe da construção da classe. Para declarar a classe, utilizaremos a palavra reservada *class* seguida do nome da classe, o abrir e fechar as chaves marcam o início e o fim da classe.

```
class NomeDaClasse
{
    //instruções;
}
```

Agora, veremos o que é atributo e como declarar os atributos da classe.

Atributo

Atributo é uma característica ou propriedade particular que os objetos de uma classe possuem, assumindo valores diferentes para cada objeto. Para declarar um atributo, colocaremos um identificador precedido da visibilidade (*public*, *private* e *protected*) do atributo. Como você já sabe, a linguagem PHP não exige do desenvolvedor que faça a declaração do tipo das variáveis ao utilizá-las, isso também vale para a Orientação a Objetos, não iremos tipá-las. Contudo, ao saber que aqueles atributos possuem um tipo, devemos realizar o tratamento em cada variável a fim de possibilitar uma melhor performance e robustez de nossa aplicação. Vamos às variáveis de classe.

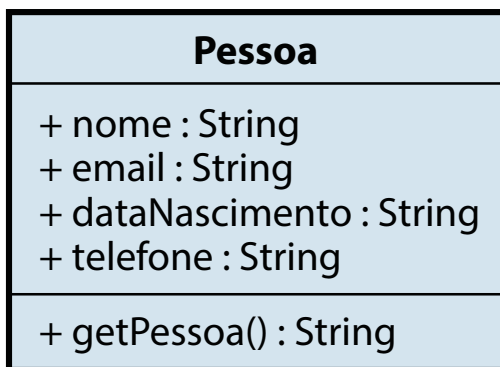


Figura 1 – Classe / Fonte: os autores.

```
class
{
public $identificador;
}
```

No exemplo a seguir, implementaremos a classe Pessoa com as suas quatro propriedades, ou seja, quatro variáveis de classe que existem dentro do contexto do objeto. Por enquanto, declararemos os atributos por meio da palavra-chave *public*, que significa visibilidade pública.



quadro-resumo

A identificação da visibilidade dos atributos em um diagrama UML é informado por meio de um símbolo.

Public: é visível de qualquer lugar do sistema. No diagrama de classes, é identificado pelo símbolo "+".

Private: é visível apenas de dentro da classe. Não pode ser acessado de fora da classe. No diagrama de classes, é identificado pelo símbolo "-".

Protected: é visível de dentro da classe e das subclasses herdadas dessa classe, mas não de fora. No diagrama de classes, é identificado pelo símbolo "#".

Abra um editor de código-fonte de sua escolha e digite o script em PHP conforme apresentado a seguir. Depois, para salvar, crie um pasta no diretório raiz do seu servidor e nomeie ela como **unidadeI**, dentro dela, crie uma outra pasta e coloque o nome de **classes**, por fim, salve esse arquivo como **Pessoa.php**.

Arquivo: Pessoa.php

```
<?php
class Pessoa
{
    public $nome;
    public $email;
    public $telefone;
    public $dataNascimento;
}
?>
```

Note que tanto o nome do arquivo quanto o nome da classe estão com a primeira letra em caixa alta, seguindo uma regra de boa prática de programação e atendendo ao padrão chamado pelo PHP-FIG de *StudlyCaps*. Os atributos da classe foram declarados em linhas separadas, a fim de melhor apresentá-los, mas podem ser declarados todos em apenas uma única linha, desde que todos os atributos tenham a mesma visibilidade.

Objeto

Um objeto é qualquer elemento concreto (físico) ou abstrato (não físico) que existe no mundo real. Entendemos objetos físicos por mesa, cadeira, carro, entre outros. Enquanto, para objetos abstratos, temos o cadastro de pessoa, uma faixa de uma música, um filme, ou seja, eu consigo ver, mas não consigo materializar. Este é o princípio da abstração de dados, ou seja, se a sua aplicação possui um nível de abstração maior, trará um aumento no nível de performance e robustez, além de possibilitar uma facilidade na manutenção, portabilidade e reuso.

Para Dall'Oglio (2009, p. 93), “um objeto é uma estrutura dinâmica originada com base em uma classe”. Isso significa que um objeto é dinâmico, ou seja, ele pode sofrer alterações no decorrer do seu curso de vida. O objeto é único e possui comportamentos e características próprios. Ele é fruto de uma instância de uma classe (BEZERRA, 2007).

Ainda de acordo Dall'Oglio (2009, p. 93), “o objeto é uma instância de uma classe, porque o objeto existe durante dado instante de tempo – da sua criação até a sua destruição”. Analisando este conceito, temos que um objeto é único no sistema, ou seja, durante a sua vida, ele possui uma referência de memória, garantindo a ele uma unicidade no sistema.

Para instanciar um objeto, primeiro, criaremos uma variável que receberá a instância da classe. Para instanciar uma classe, é utilizado o operador *new*, seguido do nome da classe, com abre e fecha parênteses.

```
$objeto = new NomeDaClasse();
```

No exemplo a seguir, fazemos o uso da classe Pessoa, criando um objeto (\$pessoa). Para termos o acesso aos atributos da classe, utilizaremos o nome do atributo precedido pelo nome do objeto.

Abra um novo arquivo, digite o script a seguir. Depois, para salvar, acesse a pasta **unidadeI** e, dentro dela, salve esse arquivo como **controlePessoa.php**.

Arquivo: controlePessoa.php

```
<?php
require_once('classes/Pessoa.php');

$ pessoa = new Pessoa();

$ pessoa->nome = "Rafael A. Florindo";
$ pessoa->email = "rafaelflorindo@hotmail.com";
$ pessoa->telefone = "44 3027-9390";
$ pessoa->dataNascimento = "2020-08-14";

echo "<div><strong>Dados da Pessoa</strong>";
echo "Nome: " . $ pessoa->nome;
echo "E-mail: " . $ pessoa->email;
echo "Telefone: " . $ pessoa->telefone;
echo "Data Nascimento: " . $ pessoa->dataNascimento;
echo "</div>";
?>
```

Para executar os arquivos, abriremos o navegador – já com o servidor web ativado – e rodaremos o arquivo **controlePessoa.php**. No nosso caso, será: *http://localhost/Unidade01/controlePessoa.php*.

Atenção: caso você tenha alterado algum nome, deve alterar aqui também.

Confira o resultado após executar o **controlePessoa.php**:

Dados da Pessoa

```
Nome: Rafael A. Florindo
Email: rafaelflorindo@hotmail.com
Telefone: 44 3027-9390
Data de Nascimento: 2020-08-14
```

Vejam, a seguir, a interpretação, passo a passo, da execução do script:

1. Notem que, na primeira linha, utilizamos o comando (`require_once('classes/Pessoa.php');`) para incluir a classe Pessoa que está dentro da pasta classes.

2. Na sequência, instanciamos o objeto \$pessoa com o comando (\$pessoa = new Pessoa();).
3. Após ter instanciado o objeto, adicionaremos e resgataremos os valores armazenados nos atributos da classe.
4. Para acessar os atributos fora da classe, devemos utilizar o próprio objeto seguido por uma seta (->), que aponta para a sua propriedade, setando ou resgatando o valor, por exemplo: (\$pessoa->nome = “Rafael A. Florindo”; echo “Nome: “ . \$pessoa->nome;).
5. Note que só é possível acessar esses atributos dessa forma porque eles estão com visibilidade pública. Nos tópicos seguintes, abordaremos as outras visibilidades.

Métodos

De acordo com Dall’Oglio (2009), um método é considerado um comportamento ou uma funcionalidade específica e única de uma classe. Isso significa que o método deve ser único, ou seja, deve possuir apenas uma única funcionalidade. É por esse motivo que os métodos são considerados de responsabilidades das classes. O método da classe em PHP segue os mesmos princípios de uma function, apenas possui, no início de sua declaração, a visibilidade do método. Confira a sintaxe a seguir:

```
public function nomeFuncao()  
{  
    //instruções  
    //  
}
```

Agora, vamos reescrever a classe Pessoa incluindo o método getPessoa(). Esse método imprimirá os valores dos atributos do objeto. Confira como ficou o código a seguir.

Arquivo: Pessoa.php

```
<?php  
class Pessoa  
{
```

```

    public $nome;
    public $email;
    public $telefone;
    public $dataNascimento;

    public function getPessoa()
    {
        echo "<br>Nome: " . $this->nome;
        echo "<br>Email: ". $this->email;
        echo "<br>Telefone: ". $this->telefone;
        echo "<br>Data de cimento: ". $this->dataNascimento;
    }
}

```

Note que, para referenciar uma de suas propriedades ou métodos dentro de uma classe, basta utilizar o ponteiro `$this` para acessá-lo. Esse ponteiro representa o próprio objeto que estará sendo manipulado.

Agora, basta chamar esse método no arquivo **controlePessoa.php** para que ele imprima os dados da pessoa.

Arquivo: controlePessoa.php

```

<?php
require_once('classes/Pessoa.php');

$pessoa = new Pessoa();

$pessoa->nome = "Rafael A. Florindo";
$pessoa->email = "rafaelflorindo@hotmail.com";
$pessoa->telefone = "44 3027-9390";
$pessoa->dataNascimento = "2020-08-14";

echo "<div><strong>Dados da Pessoa</strong>";
$pessoa->getPessoa();
echo "</div>";
?>

```

Confira o resultado após executar o **controlePessoa.php**:

Dados da Pessoa

Nome: Rafael A. Florindo

Email: rafaelflorindo@hotmail.com

Telefone: 44 3027-9390

Data de Nascimento: 2020-08-14

Note que retiramos as linhas que realizavam a impressão dos atributos e invocamos o método para realizar a sua responsabilidade.

AULA

3 MÉTODOS

CONSTRUTORES E DESTRUTORES

Em Orientação a Objetos, todas as classes possuem, nativamente, um método construtor e um destrutor que será executado no momento da instância de um objeto ou da destruição do objeto. Sendo assim, neste tópico, veremos como aplicar tais conceitos com programação OO, vamos lá!

Método Construtor

O método construtor é um método especial, executado na instância da classe pelo operador *new*, e esse método não produz um valor de retorno, pois estará retornando o próprio objeto. Todavia, o desenvolvedor pode implementar, em suas classes, o método construtor, e ele será chamado a cada objeto recém-criado. É adequado para qualquer inicialização que o objeto necessite antes de ser utilizado.



explorando Ideias

Para criar uma instância de uma classe, a instrução *new* deve ser utilizada. Um objeto sempre será criado exceto se a classe tiver um construtor definido que dispare uma exceção em caso de erro. Classes devem ser definidas antes de instanciadas – e, em alguns casos, isso é obrigatório.

Se uma string, contendo o nome da classe, é utilizada com *new*, uma nova instância da classe será criada.

Exemplo:

```
<?php
$instance = new SimpleClass();
// Também pode ser feito com uma variável:
$className = 'SimpleClass';
$instance = new $className(); // new SimpleClass()
?>
```

Fonte: PHP ([2020], on-line).

Voltando à nossa classe Pessoa, note que não temos o método construtor implementado. Nesse caso, no momento da instância do objeto `$pessoa = new Pessoa();`, o construtor não recebeu parâmetros e, também, não o implementamos. Dessa forma, todos os atributos receberam um valor *NULL* automaticamente. Edite o arquivo **controlePessoa.php** e, logo após a linha `$pessoa = new Pessoa();`, insira uma nova linha com o comando `var_dump($pessoa);`. Agora, confira o resultado após executar o **controlePessoa.php**:

```
object (Pessoa) #1 (4) {
    ["nome"]=>
    NULL
    ["email"]=>
    NULL
    ["telefone"]=>
    NULL
    ["dataNascimento"]=>
    NULL
}
```

Para declarar o método construtor em uma classe, seguiremos a sintaxe a seguir.

```
function __construct()
{
    //instruções
}
```

Vamos reescrever a nossa classe Pessoa() implementando um construtor que receberá, por parâmetros, os campos da classe para que, ao inicializar o objeto \$pessoa, ele já esteja com seus respectivos valores.

Arquivo: Pessoa.php

```
<?php
class Pessoa
{
    public $nome;
    public $email;
    public $telefone;
    public $dataNascimento;

    function __construct($nome, $email, $telefone, $dataNascimento)
    {
```

```

        $this->nome = $nome;
        $this->email = $email;
        $this->telefone = $telefone;
        $this->dataNascimento = $dataNascimento;
    }

    public function getPessoa()
    {
        echo "<br>Nome: " . $this->nome;
        echo "<br>Email: " . $this->email;
        echo "<br>Telefone: " . $this->telefone;
        echo "<br>Data de Nascimento: " . $this->dataNascimento;
    }
}

```

Note que o método construtor recebeu os parâmetros function __construct(\$nome, \$email, \$telefone, \$dataNascimento) e os setou nos seus respectivos atributos por meio do operador \$this.

Agora, confira o arquivo **controlePessoa.php**:

Arquivo: controlePessoa.php

```

<?php
require_once('classes/Pessoa.php');

$nome = "Rafael A. Florindo";
$email = "rafaelflorindo@hotmail.com";
$telefone = "44 3027-9390";
$dataNascimento = "2020-08-14";

$pessoa = new Pessoa($nome, $email, $telefone, $dataNascimento);
var_dump($pessoa);
?>

```


Note que foi necessário declarar variáveis com seus respectivos valores e passá-los por parâmetro no momento da instância do objeto \$pessoa. Ao instanciar a classe, lembre de manter a mesma ordem dos parâmetros.

Confira o resultado após executar o **controlePessoa.php**:

```
object(Pessoa)#1 (4) {
    ["nome"]=>
    string(18) "Rafael A. Florindo"
    ["email"]=>
    string(26) "rafaelflorindo@hotmail.com"
    ["telefone"]=>
    string(12) "44 3027-9390"
    ["dataNascimento"]=>
    string(10) "2020-08-14"
}
```

Método destrutor

O método destrutor é um outro método especial executado automaticamente quando o objeto é desalocado da memória. Esse processo pode ocorrer de forma natural, ou seja, quando terminar as chamadas do objeto à classe ou quando forcamos o PHP a liberar a referência do objeto pelo comando apresentado a seguir. Para implementar o método destrutor, siga a sintaxe a seguir:

```
function __destruct()
{
    //instrução
}
```

Vamos reescrever, mais uma vez, a nossa classe Pessoa().

```
<?php
class Pessoa
{
    /*
```

```
#####
conteúdo já escrito no exemplo anterior
- declaração dos atributos
- método construtor
- método getPessoa()
#####
*/

function __destruct()
{
    echo "Objeto finalizado!!!";
}
}
?>
```

Note que incluímos, no final do arquivo Pessoa.php, o método `__destruct()`. Confira o resultado após executar o **controlePessoa.php** com o método destrutor.

```
object(Pessoa)#1 (4) {
    ["nome"]=>
    string(18) "Rafael A. Florindo"
    ["email"]=>
    string(26) "rafaelflorindo@hotmail.com"
    ["telefone"]=>
    string(12) "44 3027-9390"
    ["dataNascimento"]=>
    string(10) "2020-08-14"
}
```

Objeto finalizado!!!

Pois bem, prezado(a) aluno(a), os primeiros passos para evoluir com a programação OO e PHP é saber como instanciar uma classe e trabalhar com ela, sendo assim, este tópico foi de grande aproveitamento. Vejo você em nossa próxima etapa. Até lá, bons estudos!

MODIFICADORES DE ACESSO *PUBLIC*, *PRIVATE* E *PROTECTED*

Até aqui, trabalhamos com a visibilidade *public* [notação UML (+)] dos atributos e métodos. Este é o nível de acesso mais permissivo, pois ele indica que o método ou atributo da classe é público, ou seja, pode ser acessado em qualquer outro ponto do código e por outras classes. Contudo, ainda temos mais duas visibilidades (*private* e *protected*). Com essas visibilidades, definimos se o atributo ou método pode ou não ser acessado fora da classe em que foi declarado.

A visibilidade *private* [notação UML (-)] significa que membros declarados com essa visibilidade só podem ser acessados de dentro da própria classe em que foram declarados. Desta forma, os membros não podem ser acessados de fora da classe em que eles estão contidos.

A visibilidade *protected* [notação UML (#)] significa que membros declarados com essa visibilidade só podem ser acessados pela própria classe ou classes filhas da classe principal – o conteúdo de herança será trabalhado nos próximos capítulos.

Confira um exemplo didático de nossa classe *Funcionario* na Figura 2.

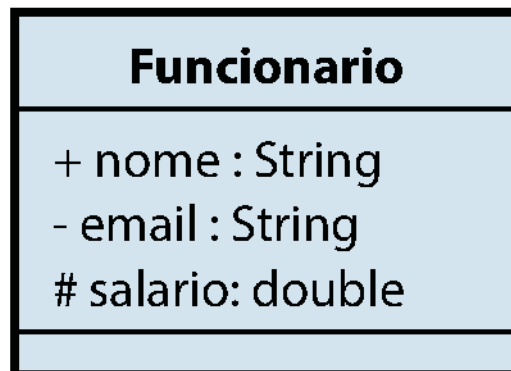


Figura 2 – Classe *Funcionario* / Fonte: os autores.

Para modificar a visibilidade de um atributo ou método, devemos proceder sua declaração de uma das palavras reservadas que representam o modificador para o atributo e método da seguinte forma:

```
modificador $atributo;
modificador function metodo()
{
    //instruções
}
```

Para que seja possível acessar os atributos *private* e *protected*, necessitamos de métodos públicos, conhecidos como modificadores de acesso **setters** e **getters**, que setam e resgatam valores, respectivamente.

Vejamos a sintaxe desses métodos.

```
modificador function setNomeDaVariavel(<parâmetro>)
{
    $this-><atributo> = <parâmetro>;
}
```

O método *setter* recebe, por parâmetro, o valor a ser armazenado no atributo da classe.

```
modificador function getNomeDaVariavel()
{
    return $this-><atributo>;
}
```

O método *getter* recebe, por parâmetro, o valor a ser resgatado no atributo da classe.

Agora que já conceituamos os modificadores de acesso, implementaremos a nossa classe **Funcionario**.

Para isso, crie um novo arquivo, digite o *script* a seguir e salve-o dentro da pasta **classes** com o nome de **Funcionario.php**.

Arquivo: Funcionario.php

```
<?php
class Funcionario
```

```
{  
    public $nome;  
    private $email;  
    protected $salario;  
  
    function Funcionario()  
    {  
  
    }  
    public function setNome($nome)  
    {  
        $this->nome = $nome;  
    }  
    public function setEmail($email)  
    {  
        $this->email = $email;  
    }  
    public function setSalario($salario)  
    {  
        $this->salario = $salario;  
    }  
    public function getNome()  
    {  
        return $this->nome;  
    }  
    public function getEmail()  
    {  
        return $this->email;  
    }  
    public function getSalario()  
    {  
        return $this->salario;  
    }  
}  
  
?>
```

Note que, para cada atributo, temos um método *set* e um *get* respectivo. O construtor da classe iniciará, de forma automática, com *NULL*, todos os atributos da classe.



pensando juntos

A esta altura, você pode estar se perguntando: “se a ideia da programação OO é o reuso, por que utilizar todos estes modificadores? Será que deixar tudo como *PUBLIC* não seria mais fácil?”. Este pensamento pode estar equivocado ao analisarmos os critérios de segurança da informação. Neste momento, devemos refletir sobre boas práticas de programação voltadas à segurança da informação.

Para testar a nossa classe, crie um novo arquivo com o nome de `controleFuncionario.php` na raiz da pasta `UnidadeI` e digite o código a seguir.

Arquivo: `controleFuncionario.php`

```
<?php
require_once('classes/Funcionario.php');

$funcionario = new Funcionario();
$funcionario->setNome("Carlos Danilo");
$funcionario->setEmail("carlosdanilo@xxx.xx");
$funcionario->setSalario(4500.00);
var_dump($funcionario);

echo "<br>Nome: ".$funcionario->getNome();
echo "<br>E-mail: ".$funcionario->getEmail();
echo "<br>Salário: ".$funcionario->getSalario();

?>
```

Note que incluímos o arquivo **Funcionario.php**, instanciamos o objeto `$funcionario` e invocamos os métodos *setters* para armazenar os valores dos campos. Além disso, invocamos os métodos *getters* para devolver os valores e serem impressos na tela.

Vejamos o resultado executando o arquivo **controleFuncionario.php**:

```
Nome: Carlos Danilo
E-mail: carlosdanilo@xxx.xx
Salário: 4500
```

Os modificadores de acesso são de extrema importância em um projeto, pois estão ligados diretamente a critérios de segurança e pontos de acesso ao código. Tais princípios estão ligados, também, a boas práticas em desenvolvimento. Acreditamos que o conteúdo apresentado será muito bem aplicado por você futuramente.

AULA

5 APLICANDO NA PRÁTICA

Nesta aula, colocaremos em prática o conhecimento adquirido nesta Unidade. Para isso, vamos a um pequeno estudo de caso.

José é um aluno do curso de SI e está cursando a disciplina de *Back End II*. Por ser aluno, José possui alguns atributos, como: um número de matrícula, suas quatro notas que são acessíveis apenas para o cadastro dele, ou seja, não será recuperado fora da classe. Além desses dois atributos, o seu nome é visível para todos os alunos. José também deseja obter uma relação de notas e a sua média.

Analisando o estudo de caso anterior, criaremos uma classe Aluno, pois podemos ter outros alunos. Nesta classe, incluiremos os atributos *matricula* e *notas[4]* com visibilidade *private* para que apenas o próprio aluno possa ter acesso aos dados, e o atributo *nome* como *public*. Incluiremos os métodos *obterMedia()* e *obterNotas()*. A seguir, temos a Figura 3.

Aluno
- <i>matricula</i> : int + <i>nome</i> : String - <i>notas[4]</i> : double
+ <i>obterMedia()</i> : double + <i>obterNotas()</i> : void

Figura 3 – Classe Aluno(). / Fonte: os autores.



explorando Ideias

O que faz um bom programador?

Muita gente se pergunta como se tornar um bom programador. Normalmente estão mais preocupados em qual linguagem aprender, qual curso fazer, quais livros técnicos ler. Porém podemos dizer que existem qualidades mais importantes a se levar em conta. [...].

Um bom programador é um pensador analítico.

Programadores precisam ser solucionadores de problemas.

O processo de programação requer que possamos quebrar problemas complicados, planejar e implementar soluções e encontrar pequenas inconsistências no código (*bugs*).

Pensamento analítico também se manifesta na habilidade de seguir e entender lógicas complicadas através de segmentos de código. Isso nos permite alcançar conceitos abstratos como metodologias de Orientação a Objetos e *design patterns* (padrões de projetos), implementando-os em prática.

Um bom programador tem suas prioridades claras.

Se eu lhe pedisse para ordenar os itens seguintes por prioridade, como você os ordenaria?

- Segurança
- Manutenibilidade
- Usabilidade
- Performance
- Quantidade de LOC (lines of code – linhas de código)

Fonte: Eu Faço Programas (O QUE FAZ..., 2019, on-line).

Mediante a classe anterior, abra um novo arquivo e salve-o como **Aluno.php** na pasta Classes.

Arquivo: Aluno.php

```
<?php
class Aluno {
    private $matricula;
    public $nome;
    private $notas;

    function __construct($matricula, $nome)
    {
        $this->notas = array();
        $this->matricula = $matricula;
        $this->nome = $nome;
    }
    public function setNotas($nota):int
    {
        if(count($this->notas) < 4)
        {
            $this->notas[]=$nota;
            return 1;
        }
        else
            return 0;
    }
    public function obterNotas()
    {
        foreach($this->notas as $pos => $nota)
        {
            echo "<br>Nota [" . ($pos+1) . "]= " . $nota;
        }
    }
    public function obterMedia():float
    {

```

```

        $soma = 0;
        for($i=0; $i<count($this->notas); $i++)
        {
            $soma += $this->notas[$i];
        }
        $media = $soma/count($this->notas);
        return $media;
    }
}

?>

```

Explicaremos o código por partes. Primeiro, declaramos os atributos da classe Aluno. Note que o atributo notas (private \$notas;), por enquanto, é um atributo comum da classe, contudo ele deverá ser um *array* de quatro posições.

Após declarar os atributos, implementamos o construtor da classe, o qual está recebendo, por parâmetro, o número da matrícula e o nome do aluno e setando eles em seus respectivos valores. Contudo, dentro do construtor, estamos inicializando um *array* dinâmico no atributo `$this->notas = array();`, para que seja possível adicionar as notas do aluno. Ainda, não limitaremos a quantidade de notas.

Na sequência, implementamos o método public function `setNotas($nota) : int`. Este método recebe a nota do aluno, realiza todo o processamento e devolve um número do tipo inteiro. Dentro do método, estamos fazendo uma condição `if(count($this->notas) < 4)` para verificar se a quantidade de notas não atingiu o limite de quatro alunos. Caso não tenha atingido, setamos o valor dentro do *array*, e, como ele é dinâmico, o próprio PHP trata de controlar o seu índice. Caso já tenha atingido o limite de quatro notas, retornamos um valor 0.

Continuando na classe Aluno, implementamos o método public function `obterNotas()`. Nele, fizemos um *foreach* que percorre o *array* e imprime as notas.

Por fim, implementamos o método public function `obterMedia() : float`, que retornará um valor *float* (real). Esse método, por meio de um laço de repetição, percorre o *array* e acumula a soma das notas do aluno. Terminando o laço de repetição, calcula-se a média do respectivo aluno.

Agora que explicamos a classe Aluno, vamos abrir um novo arquivo, digitar este *script* e salvar na raiz da pasta **UnidadeI** com o nome de **controleAluno.php**.

Arquivo: controleAluno.php

```

<?php
require_once('classes/Aluno.php');

$aluno = new Aluno(1234, "Jose Fernandes");

echo ($aluno->setNotas(6.5))? "<br>Nota incluída
com sucesso": "<br>Erro ao Incluir Nota";
echo ($aluno->setNotas(5.5))? "<br>Nota incluída
com sucesso": "<br>Erro ao Incluir Nota";
echo ($aluno->setNotas(6.5))? "<br>Nota incluída
com sucesso": "<br>Erro ao Incluir Nota";
echo ($aluno->setNotas(5.5))? "<br>Nota incluída
com sucesso": "<br>Erro ao Incluir Nota";

echo "<div><br>Notas do aluno";
echo "<br>Nome: " . $aluno->nome;

$aluno->obterNotas();
echo "<br>Média: " . $aluno->obterMedia();
echo "</div>"
?>

```

Dentro deste arquivo, incluímos a classe e a instanciamos, passando dois valores para o método construtor.

Na sequência, fizemos um *if* ternário para cada nota com uma impressão, ou seja, se a condição for satisfeita, imprime a primeira mensagem, caso contrário, imprime a segunda. A condição de *if* vem do retorno da chamada da função `$aluno->setNotas(6.5)`. Se a nota do aluno for incluída, teremos o retorno 1, caso contrário, teremos o retorno 0. Essa chamada se repete por mais três vezes.

Continuando: imprimimos o nome do aluno, as suas notas, bem como a sua média.



conecte-se

Um padrão de escrita para código em PHP é chamado de PSR. Acesse o link a seguir para conhecer mais sobre este assunto:



Agora, confira o resultado depois de ter executado o arquivo **controleAluno.php**.

```
Nota incluída com sucesso
Nota incluída com sucesso
Nota incluída com sucesso
Nota incluída com sucesso
```

```
Notas do aluno
Nome: Jose Fernandes
Nota [1] = 6.5
Nota [2] = 5.5
Nota [3] = 6.5
Nota [4] = 5.5
Média: 6
```

Conforme Aristóteles dizia: “é fazendo que se aprende a fazer aquilo que se deve aprender a fazer”. Nosso conteúdo final reflete justamente sobre isso, em levar até você, aluno(a), a prática dos conteúdos teóricos apresentados, trabalhando com todo o conhecimento adquirido e o colocando em prática. Compreendemos que este momento é fundamental e necessário para um bom aprofundamento de seus estudos. Nos vemos em breve! Até lá, bons estudos!

CONSIDERAÇÕES FINAIS

A linguagem de programação PHP é uma das mais utilizadas no *Back End* em projetos Web. Isso se dá pela sua dimensão de conteúdos que podem ser facilmente encontrados e que auxiliam muitos desenvolvedores e empresas web, além de ser uma linguagem de programação com uma curva de aprendizado simples. Isso significa que, com poucos momentos de estudos, você já consegue desenvolver e entregar alguma aplicação em PHP rodando, podendo, também, ser facilmente trabalhada com os paradigmas de linguagem estrutural e Orientação a Objetos, as quais são requisitadas pelo mercado de trabalho.

Um ponto de atenção é que a linguagem PHP sobre o paradigma de Orientação a Objetos continua a atuar apenas com o *Back End*, mesmo que suas ações sejam refletidas e possuam uma interação com o *Front End*. Isso quer dizer que os seus conhecimentos sobre HTML, CSS, JavaScript ou demais artefatos de *Front End* continuam sendo necessários.

A programação orientada a objetos traz diversas vantagens sobre a reusabilidade e aumento da qualidade no projeto final, entretanto compreenda, prezado(a) aluno(a), a programação orientada a objetos não é tão fácil e exige maior concentração na análise e no projeto. Para que ela tenha êxito, o desenvolvedor e o usuário precisam trabalhar em conjunto. A adoção da programação orientada a objetos em aplicações web é uma realidade viável para desenvolvedores e empresas. A tendência é que os projetos se tornem cada vez mais complexos, conforme a versatilidade de aplicações de programas.

Nesta unidade, você estudou os conceitos iniciais sobre o paradigma de Orientação a Objetos (OO), abordamos os conceitos básicos desse paradigma para entender, por meio de elementos, a estrutura do sistema, analisando diagramas UML, sendo possível associar tais observações com a programação PHP com OO.

No decorrer de nossa unidade, foi possível compreender e aplicar os conceitos teóricos sobre OO com a linguagem de programação PHP. Você pôde conhecer e aprofundar os conhecimentos sobre a linguagem de programação PHP. Identificamos, também, os atributos e os métodos de cada uma das classes, bem como os seus relacionamentos.

Acreditamos que teremos momentos muito especiais nas próximas unidades. Preparado(a) para seguir em frente? Bons estudos! Nos vemos em breve!



1. Normalmente, o acesso a **arrays** é feito utilizando laços de repetição. Seria muito trabalhoso escrever o índice de cada elemento no código. Podemos inicializar um **array** com 50 elementos facilmente com um laço FOR. A variável do laço seria utilizada como índice do **array**. Observe o código a seguir:

```
<?php
class Pessoas {
    public function idade(){
        $somaIdades = 0;
        for($j = 0; $j < 50; $j=$j+2){
            $somaIdades= $j*2;
        }
        return $somaIdades;
    }
}

$foo = new Pessoas();
$funcname = "idade";
echo $foo->$funcname();

?>
```

A alternativa que mostra corretamente o que será impresso na tela é:

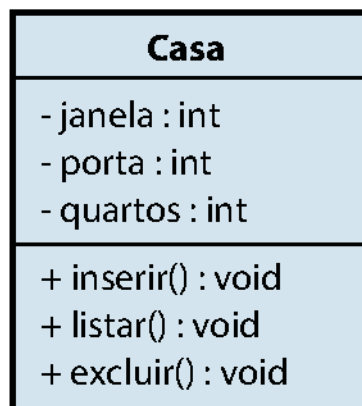
- a) 0.
- b) 53.
- c) 96.
- d) 128.
- e) 236.



2. É comum encontrarmos, na linguagem OO, termos como *"public"*, *"private"*, *"protected"* etc. Esses são modificadores de acesso. São utilizados para configurarmos o acesso que o membro ou classe terá perante o sistema.

De acordo com o conceito de modificadores de acesso da linguagem PHP, assinale a alternativa que expressa corretamente o funcionamento do modificador *private*.

- a) Atributos e métodos não podem ser acessados.
 - b) Atributos e métodos são acessados somente pela própria classe, independente do pacote em que esteja.
 - c) Atributos e métodos podem ser acessados e modificados de qualquer classe de projeto a partir da instância do objeto.
 - d) Atributos e métodos podem ser acessados e modificados de qualquer classe, dentro do mesmo pacote, ou tenham herdado características de uma classe dentro do mesmo pacote.
 - e) Atributos e métodos podem ser acessados e modificados de qualquer classe dentro do mesmo pacote, mas não de classes que tenham herdado características da classe no mesmo pacote e que estejam em pacotes diferentes.
3. A devida interpretação de diagramas UML é um processo comum na vida de um desenvolver, sendo assim, analise o diagrama a seguir:





De acordo com a imagem apresentada, analise as afirmativas a seguir de forma que indique a classe, atributos e métodos.

- a) Classe: Casa.
Atributos: janela, portas e quartos.
Método: inserir(), listar() e excluir().
 - b) Classe: Casa.
Atributos: inserir(), listar() e excluir().
Método: janela, portas e quartos.
 - c) Classe: Casa.
Atributos: int.
Métodos: Void.
 - d) Classe: Casa.
Atributos: int, int, int.
Método: janela, portas e quartos.
 - e) Classe: Casa.
Atributos: janela, portas e quartos.
Método: void, void, void.
4. A base para a Orientação a Objetos está nas classes, que possuem atributos e métodos. Explique o que são as visibilidades de atributos e métodos, quais são as opções de visibilidade e qual é a diferença entre elas.
5. Na Orientação a Objetos, é importante sabermos que os objetos representam qualquer coisa concreta ou abstrata do mundo real e que pode ser individualizada, já uma classe representa, de forma abstrata, um conjunto de objetos do mundo real, com comportamentos e características comuns. Já a ocorrência de um objeto formado a partir de uma classe é uma instância.

Dentro do paradigma da Orientação a Objetos, considere a descrição a seguir sobre as ações de um personagem de um jogo:

“O gato pode andar, correr, pular. O gato come ração. O gato tem 2 anos”.

Como você representaria toda a descrição apresentada em um diagrama de classes que poderia ser, posteriormente, utilizado para a criação de um script?



NOSQL NO DESENVOLVIMENTO DE APLICAÇÕES WEB COLABORATIVAS

O grande volume de dados gerado por aplicações Web, juntamente com os requisitos diferenciados destas aplicações, como a escalabilidade sob demanda e o elevado grau de disponibilidade, têm contribuído para o surgimento de novos paradigmas e tecnologias. As redes sociais, por exemplo, requerem o gerenciamento de grandes quantidades de dados não estruturados, os quais são gerados diariamente por milhões de usuários em busca do compartilhamento de informações, conhecimentos e interesses.

Neste contexto, surgiu uma nova categoria de Banco de Dados, chamada NoSQL (Not Only SQL), que foi proposta com o objetivo de atender aos requisitos de gerenciamento de grandes volumes de dados, semi-estruturados ou não estruturados, que necessitam de alta disponibilidade e escalabilidade. A necessidade de uma nova tecnologia de BD surgiu como consequência da ineficiência dos bancos de dados relacionais em lidar com esta tarefa. [...]

Os bancos de dados NoSQL têm sido amplamente adotados em empresas como Facebook, Amazon e Google com o intuito de atender às suas demandas de escalabilidade, alta disponibilidade e dados não estruturados. Além disso, atualmente, diversos bancos de dados NoSQL de código livre estão disponíveis, como: Cassandra, Hypertable, MongoDB, Redis, CouchDB e Dynamo.

Tendo em vista o crescente interesse na adoção desta tecnologia, bem como os novos desafios gerados pelo uso do NoSQL, torna-se fundamental o conhecimento de seus principais conceitos e sua utilização. Em particular, estes conhecimentos são de grande relevância para aqueles que têm interesse em aplicações Web colaborativas, as quais, na maioria das vezes, necessitam de uma tecnologia que ofereça suporte ao gerenciamento e escalabilidade de grandes volumes de dados, de maneira simples e eficiente. [...]

Fonte: adaptado de Lóscio, Oliveira e Pontes (2011).



eu recomendo!



livro

PHP: Programando com Orientação a Objetos

Autor: Pablo Dall'Oglio

Editora: NovaTrec

Sinopse: o PHP é uma das linguagens mais utilizadas no mundo. Sua popularidade se deve à facilidade em criar aplicações dinâmicas com suporte à maioria dos bancos de dados existentes e ao conjunto de funções que, por meio de uma estrutura flexível de programação, permitem desde a criação de simples portais até complexas aplicações de negócio.

O uso da Orientação a Objetos, juntamente com o emprego de boas práticas de programação, possibilita-nos manter um ritmo sustentável no desenvolvimento de aplicações.





PILARES DA PROGRAMAÇÃO orientada a objetos

PROFESSORES

Me. Rafael Alves Florindo

Esp. Carlos Danilo Luz

PLANO DE ESTUDO ▼

A seguir, apresentam-se as aulas que você estudará nesta unidade: • Abstração • Encapsulamento • Herança e Classes Abstratas • Polimorfismo • Aplicando na Prática.

OBJETIVOS DE APRENDIZAGEM ▼

Compreender as características essenciais de um contexto qualquer, ignorando os menos importantes. Entender o conceito de encapsulamento. Estudar os modificadores de acesso default, public, protected e private. Controlar o acesso a métodos, atributos e construtores por meio de modificadores. Compreender os modificadores static, abstract e final. Ampliar o conhecimento do aluno sobre a especialização de comportamentos de um método em comum com a superclasse. Levar o aluno à prática de dois pilares da Orientação a Objetos, envolvendo os conceitos aprendidos nesta unidade.



Olá, prezado(a) aluno(a), é com muita alegria que damos continuidade aos nossos estudos nesta unidade, a qual o levará a compreender como trabalhar com os atributos das classes, abordando os conceitos essenciais para as boas práticas de programação. Devo lembrar que os conceitos de Programação Orientada a Objetos vistos neste livro podem ser aplicados não apenas em linguagem PHP, mas qualquer outra que possua o paradigma OO.

Um dos primeiros assuntos a ser abordado é a abstração, visto que um software, seja ele web ou não, é uma abstração do mundo real para compreensão do computador. Muitos analistas de TI têm dificuldade em abstração de ideias e funcionalidades a serem criadas, muitos acham o processo inicial do software muito complexo pelo fato de terem que abstrair informações de situações e casos-problemas. Veremos no decorrer de nossa unidade que a abstração faz parte itinerante da programação OO.

Com o aumento de complexidade dos problemas a serem tratados por computadores, isso também exige a criação de uma codificação mais refinada. Sendo assim, também abordaremos as estruturas de controle de atributos e métodos, visibilidade e abstração das classes. Essas estruturas de controle são uma parte-chave de quase toda linguagem de programação OO, pois podemos fazer as definições de acesso sobre trecho de código. Neste caso, estamos falando do encapsulamento, um método simples, mas eficaz para a proteção de conteúdos do código.

A Programação Orientada a Objetos tem como particularidade a ideia de reuso de código e a integração de classes. Isso é possível com base nos conceitos de herança e polimorfismo, princípios que dão autonomia para trabalhar com as classes. O estudo desses conceitos lhe levará para outro nível no paradigma OO. Novamente, trabalharemos com as modelagens UML de nossos exemplos. A UML (Unified Modeling Language — Linguagem de Modelagem Unificada) é uma linguagem visual utilizada para modelar sistemas computacionais orientados a objeto, conforme já descrito na unidade anterior.

Com base em toda nossa conversa inicial, convido-lhe a nos acompanhar em mais um passo rumo ao sucesso. O que estamos esperando? Vamos lá!

1 ABSTRAÇÃO



Olá, alunos(as)! Trabalharemos com um dos quatros conceitos dos pilares da Programação Orientada a Objetos. Esse pilar é muito importante para um bom entendimento da complexidade que esse tipo de programação utiliza. Vejamos a seguir a representação de todos os pilares.

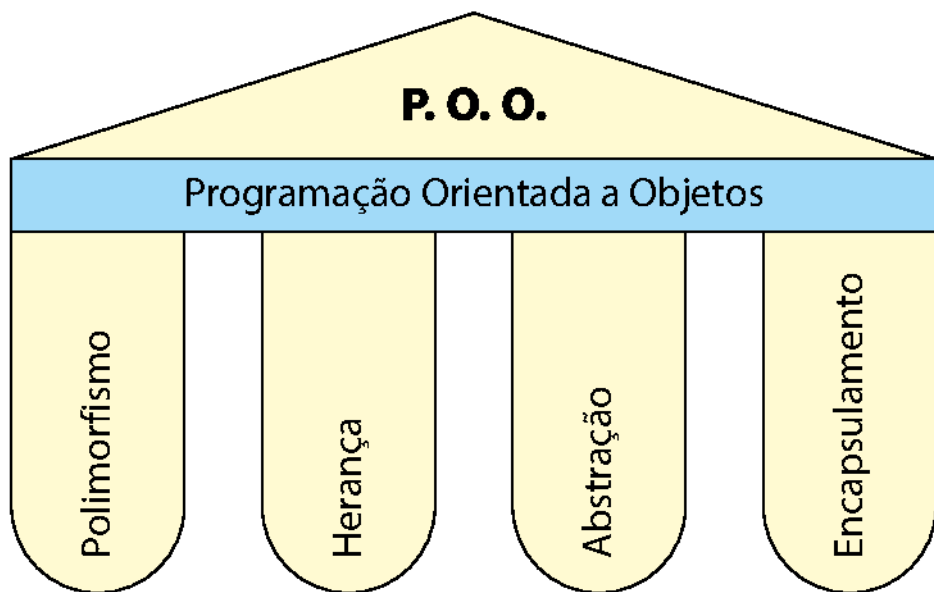


Figura 1 - Quatro pilares da Programação Orientada a Objetos / Fonte: os autores.

O uso da abstração dentro da Orientação a Objetos é um ponto vital dentro do desenvolvimento de sistemas. Esse fato se dá pois lidamos diretamente com a representação de um objeto seja ele real ou abstrato, além disso, temos que imaginar o que esse objeto realizará dentro de nosso sistema.

Nós, seres humanos, trabalhamos com agrupamentos de objetos para que possamos gerenciar a complexidade de entender as coisas do mundo real, como, por exemplo, uma mesa, uma cadeira, um cachorro.

Para Bezerra (2007, p. 8), o conceito de abstração é definido como “um processo mental pelo qual nós seres humanos nos atemos aos aspectos mais importantes (relevantes) de alguma coisa, ao mesmo tempo que ignoramos os aspectos menos importantes”. Vamos a um exemplo.

Quando você pensa em uma mesa, logo, vem à sua mente uma mesa de madeira quadrada, com quatro pernas na altura de sua cintura. Pode ser que, em algum dia, você veja duas mesas, uma mais baixa que a outra, uma retangular, a outra quadrada, uma de madeira, outra de vidro, uma com quatro pernas, e a outra com três pernas, e por aí vai. No entanto, você ainda terá a certeza de estar diante de duas mesas. Você deve estar se perguntando “como isso é possível?”. Calma, isso é possível pois você já tem uma ideia de mesa formada em sua mente. Independente das pequenas diferenças que possam haver entre os objetos, ela sempre será uma mesa.



pensando juntos

O processo de abstração também pode ser associado ao uso de sua imaginação, pois você não necessariamente pode ver a situação ou tocar um objeto, mas você consegue, com sua imaginação, compreender os fatos.

Ainda para Bezerra (2007, p. 8), “abstração de algo é dependente da perspectiva (contexto) sobre a qual uma coisa é analisada: o que é importante em um contexto pode não ser importante em outro”. Esse exercício é importante para que possamos gerenciar a complexidade e extrair as características essenciais do objeto e entender os outros pilares.

Desta forma, podemos considerar que uma abstração é uma classe, pois ambas possuem características e comportamentos, o que se difere é como utilizar os seus valores. Sendo assim, ao modelar uma classe, pense que apenas um único subconjunto de características pode ser relevante para o domínio, ou seja, uma classe representa apenas uma abstração. Vamos exemplificar.

Quando projetamos uma classe, reunimos características e comportamentos inerentes àquela classe, ou seja, construiremos um molde a partir desses atributos. Esse molde só caberá naquele escopo de domínio.

Outro exemplo: quando você compra um aparelho celular, você olha para as suas características e as suas performances, mas não se preocupa a respeito de como ele foi construído, quais são os seus componentes, como ele funciona, ou seja, você está empregando a abstração desse objeto.

Para Dall'Oglio (2009, p. 103),



[...] construir um sistema orientado a objetos, não devemos projetar o sistema como sendo uma grande peça monolítica; devemos separá-lo em partes, concentrando-nos nas peças mais importantes e ignorando os detalhes (em um primeiro momento), para podermos construir peças bem-definidas que possam ser reaproveitadas mais tarde, formando uma estrutura hierárquica.

Na programação, isso ocorre diversas vezes, principalmente, na Orientação a Objetos, independente da linguagem. Agora, vamos falar da linguagem PHP. Essa linguagem possui diversas funções e classes nativas, em que as chamamos, passamos parâmetros, e elas nos devolvem um resultado que já é reconhecido.

Bom, você deve estar se perguntando novamente “vou ter que aprender sobre todas as funções e classes nativas do PHP?”. A resposta é depende, pois você precisa entender a lógica que está por trás da abstração e, depois, procurar por manuais, fóruns da comunidade e livros.



2 ENCAPSULAMENTO

A partir de agora, estudaremos o segundo pilar da Programação Orientada a Objetos, o encapsulamento.

De acordo com Dall'Oglio (2009, p. 107), o “encapsulamento é um mecanismo que provê proteção de acesso aos membros internos de um objeto”. Como já estudamos na unidade anterior, uma classe é formada por propriedades (atributos da classe) e métodos (comportamentos da classe), e o objeto instanciado tem acesso ao domínio dessa classe. Contudo, nem sempre é conveniente deixá-los visíveis, por esse motivo, o uso do encapsulamento é fundamental, ou seja, proteger meus atributos significa proteger meus dados, e proteger seus comportamentos ocultando o que pode ser realizado, deixando visível apenas para quem tem acesso.

Para o autor Bezerra (2007), toda a comunicação entre objetos e classes se dá pela troca de mensagem entre objeto remetente, que solicita a execução de alguma operação ou valor, e objeto destinatário, que os devolve ao objeto remetente por meio da mensagem. Porém, o objeto remetente não precisa conhecer a forma pela qual a operação requisitada é realizada, tudo o que importa a esse objeto remetente é obter a operação realizada, não importando como.

Nem toda classe deve permitir que seus atributos ou métodos devam ser acessados de forma externa, ou seja, o acesso deve ser apenas interno da própria classe ou por suas classes herdeiras. Assim, é necessário ter métodos específicos para tratar esses atributos de uma forma correta. Se uma classe permite que seus atributos sejam acessados diretamente fora da classe, ela está perdendo a responsabilidade sobre eles.

Contudo, como restringir atributos e métodos?

Uma forma de restringi-los é utilizando as visibilidades *public*, *private* e *protected*. A visibilidade define a forma como essas propriedades devem ser acessadas de forma externa.

Contudo, como acessar esses atributos ou métodos?

Os atributos que possuem a visibilidade *public* podem ser acessadas por todos os objetos externos de forma direta, isso é um risco para a sua aplicação.

Os atributos que possuem a visibilidade *private* só podem ser acessados internamente à classe, e, para serem acessados de forma externa, é necessário utilizar um método que possua a visibilidade *public*.

Os atributos que possuem a visibilidade *protected* só podem ser acessados internamente à classe ou em classes que dela herdaram, e, da mesma forma que a visibilidade *private*, para serem acessados de forma externa, é necessário utilizar um método que possua a visibilidade *public*.

Normalmente, esses métodos são chamados de métodos acessores, que vão setar ou resgatar o valor do atributo. Esses métodos são os *setters* e *getters*, conteúdo que já estudamos, lembra?

No entanto, nada impede você de construir um método que seta todos os atributos e um método que resgate todos os atributos, pois, dessa forma, você está ocultando as características dos atributos, que é um dos requisitos do encapsulamento. Por exemplo, se você tem um atributo que é *private*, você concorda que ele tem um valor significativo e que não pode ser compartilhado de qualquer forma? Seja para entrada ou saída de dados. Então, se você der acesso a esse atributo por meio do método acessor *setter*, estará perdendo a responsabilidade dele, pois pode ser alterado, ou o *getter*, que pode ser resgatado.



pensando juntos

Será que deixar todos os métodos e atributos públicos não seria mais fácil para o uso deles durante o desenvolvimento do sistema? E o critério de segurança? Esses pensamentos são realmente um dilema da Programação Orientada a Objetos.

Você deve estar se perguntando: uso ou não uso os métodos acessores? A resposta você já conhece: depende. Normalmente, quando implementamos um método *setter*, realizaremos algum tratamento especial dentro dele, evitando, assim, valores indesejados no atributo.

Exemplificaremos o conceito de abstração e de encapsulamento para que você possa internalizar esses conhecimentos.

A seguir, temos a Figura 2, que ilustra uma classe Mesa; nesta, temos apenas três atributos: `quantidadePernas` do tipo `int`, e os atributos `tipo` e `formaGeometrica` do tipo `string`, todos com visibilidade `public`. Temos o método `adicionarMesa`, que recebe os 3 atributos por parâmetro e, ao seu final, retorna um `boolean`. Por fim, o método `mostrarMesa`, que retorna um `String`.

Mesa
+ quantidadePernas : int + tipo : String + formaGeometrica : String
+ adicionarMesa(qtdPernas : int, tipo : String, forma : String) : boolean + mostrarMesa() : String

Figura 2 - Classe Mesa / Fonte: os autores.

Implementaremos essa classe agora. Para isso, crie uma nova pasta com o nome de `Unidade02` e, dentro desta pasta, crie uma outra pasta com o nome de `classes` e, por fim, abra um arquivo novo e salve-o com o nome de `Mesa.php`.

Arquivo: Mesa.php

```
<?php
class Mesa
{
    public $quantidadePernas, $tipo, $formaGeometrica;

    function __construct(){ }

    public function adicionarMesa($qtdPernas, $tipo, $forma):bool{
        if($this->setQuantidadePernas($qtdPernas) &&
            $this->setTipo($tipo) &&
```

```

        $this->setForma($forma))
        return true;
    else
        return false;
}

public function mostrarMesa():string{
    return "Mesa ". $this->formaGeometrica . " com " .
    $this->quantidadePernas . " pernas, do tipo " . $this->tipo;
}

private function setQuantidadePernas($qtdPernas):bool{
    if(is_int($qtdPernas)){
        $this->quantidadePernas = $qtdPernas;
        return true;
    }
    else{
        return false;
    }
}

private function setTipo($tipo):bool{
    if(is_string($tipo)){
        $this->tipo = $tipo;
        return true;
    }
    else{
        return false;
    }
}

private function setForma($forma):bool{
    if(is_string($forma)){
        $this->formaGeometrica = $forma;
        return true;
    }
}

```

```

    }
    else{
        return false;
    }
}
}
?>

```

Calma, explicaremos o código! Inicialmente, declaramos os atributos `public $quantidadePernas, $tipo, $formaGeometrica`; e, na sequência, implementamos o método construtor sem parâmetros, pois existe um método que realizará a inserção de valores.

Note que o método `adicionarMesa($qtdPernas, $tipo, $forma) : bool` devolve um valor do tipo *boolean*, de acordo com o solicitado na modelagem. Isso só é possível a partir do php na versão 7. Para que ele retorne um valor booleano, devemos incluir o comando *return* seguido de *true* ou *false*. Neste caso, implementamos uma condição que validará as chamadas dos métodos acessores. Se todos retornarem *true*, essa função retornará *true* também; caso algum método acessor devolva *false*, essa função retornará *false* também.



conceituando

Vale lembrar que as variáveis, atributos ou métodos que possuem como declaração ou retorno “booleano” assumem que esse retorno possa ser apenas “0 ou 1” ou “*true* ou *false*”.

Na sequência, implementamos o método `public function mostrarMesa() : string`, que está retornando uma *string* de acordo com o solicitado na modelagem, contendo todas as informações necessárias.

E, por fim, implementamos os métodos acessores. Contudo, note que os deixamos com a visibilidade *private*, ou seja, apenas a própria classe pode consumi-lo. Aqui, temos uma boa prática do encapsulamento. Além dessa visibilidade, os métodos retornam um valor booleano, pois cada um deles realiza uma validação do valor que recebe.

Note que, no método `setQuantidadePernas`, o tipo do dado recebido é um *int* e, nos outros, é *string*, sendo assim, cada um está utilizando as funções de validação de tipos, sendo `is_int()` para o tipo inteiro, e o `is_string()` para o tipo *string*. Essa validação é importante, pois o php não é tipado, cabe ao desenvolvedor realizar as validações, caso necessário.

Agora, abra um novo arquivo, salve-o com o nome de `controlerMesa.php` dentro da pasta **Unidade02**.

Arquivo: `controlerMesa.php`

```
<?php
require_once('classes/Mesa.php');

$mesa1 = new Mesa();
if($mesa1->adicionarMesa(4, "Madeira Rústica", "Retangular"))
{
    echo "<b>Mesa cadastrada:</b><br>".$mesa1->mostrarMesa();
}
else
    echo "Ocorreu um erro ao cadastrar a mesa!!!";
?>
```

Note que requisitamos a inclusão da classe `Mesa` e já instanciamos na sequência, em um objeto com o nome de **\$mesa1**. Como o método `adicionarPessoa()` retorna um valor booleano, implementamos uma condição para testar se o mesmo foi cadastrado; caso o retorno tenha sido *true*, chamamos o método `mostrarMesa()`, caso contrário, imprimiremos uma mensagem de erro.

Ao executar esse script, temos o seguinte resultado no navegador:

Mesa cadastrada:

Mesa Retangular com 4 pernas, do tipo Madeira Rústica

Agora, é hora de você praticar os conceitos internalizados de abstração e encapsulamento antes de avançar. Sugiro colocar mais atributos e métodos que sejam pertinentes a esse domínio, como, por exemplo, incluir cadeiras na mesa.

Entendendo a diferença dos modificadores *Static*, *Abstract* e *Final*

O **modificador Static** é usado para definir que um método ou atributo em uma classe é estático. Nesse caso, o valor estático pertence à classe e não às instâncias, mas podem ser usados dentro da classe via `self`. Isso é muito interessante para valores que devem ficar disponíveis para toda a aplicação, ou seja, valores constantes. Podemos acessá-las fora da classe, e, neste caso, basta colocar o nome da classe seguido de `::` e o nome do atributo ou método.

Veja o exemplo a seguir. Para isso, abra um novo arquivo na pasta classe e salve-o com o nome **ExemploStatic.php**

Arquivo: ExemploStatic.php

```
<?php
class ExemploStatic
{
    public static $voltage = 220;

    public function valorEstatico() {
        return self::$voltage;
    }
}
```

Note que temos a variável `$voltage`, que é declarada como *static* e inicializada com o valor 220, ou seja, um valor constante. Dentro do método `valorEstatico()`, acessamos o atributo estático por meio da palavra reservada `self`. O `self` serve para acessar propriedades da classe dentro dela mesma, ou seja, para todas as instâncias, só existirá um único valor, já que a propriedade é de classe.

Agora, abra um novo arquivo e salve com o nome de **controlerStatic.php**.

Arquivo: controlerStatic.php

```
<?php
include('classes/ExemploStatic.php');

echo "<br>Acessando direto o atributo " .
ExemploStatic::$voltage;

$teste = new ExemploStatic();
```

```
echo "<br>Acessando o atributo por meio da classe " .
$teste->valorEstatico();
```

Vejam que, quando executo a instrução `ExemploStatic::$voltagem;`, acessamos, de forma direta, a variável estática, dessa forma, não é necessário instanciar a classe. Já no segundo caso, precisamos instanciar a classe e acessar o método, e, dentro dele, tem-se acesso à variável estática.

O **modificador *Final*** previne que classes filhas possam sobrescrever um método da superclasse que esteja marcada com o operador `final`. Se a própria classe estiver definida como `final`, ela também não pode ser estendida, ou seja, ela não pode ser superclasse (DALL'OGGIO, 2009). Vejamos um exemplo a seguir. Confirmam a sintaxe do php:

```
<?php
class Teste1{
    final public function metodo(){}
}
?>

<?php
final class teste{
}
?>
```

Note que, nesse, temos duas classes, sendo que, na primeira, temos um método com o modificador `final`, e, na segunda, a classe tem o modificador `final`.

O **modificador *Abstract*** nada mais é do que um método que consiste na definição de uma assinatura na classe abstrata. Neste caso, a assinatura do método terá um (;) no seu fim. Esse método deverá ter a sua implementação realizada na classe que a estender. Veremos um exemplo desse modificador nas próximas aulas.

Confirmam a sintaxe do php:

```
<?php
abstract class Teste1{
    abstract public function metodo1();
    public function metodo2(){
        //instruções
    }
}
?>
```

3 HERANÇA E CLASSES ABSTRATAS



Trabalharemos com o pilar da herança que visa a reutilização de código. A herança ou generalização é uma outra forma de abstração de dados de alta complexidade, pois o objetivo desse pilar é o agrupamento comum de diversos objetos, possibilitando ao desenvolvedor um reaproveitamento de código.

Segundo Bezerra (2007, p. 11), “a generalização pode ser vista como um nível de abstração acima da encontrada entre classes e objetos. Na generalização, classes semelhantes são agrupadas em uma hierarquia”. Nesse caso, temos classes separadas de acordo com o seu domínio, utilizando-se, para isso, do conceito de hierarquia.

De acordo com Dall’Oglio (2009, p. 98),



[...] a utilização da herança a objetos e do encapsulamento do código em classes nos orienta em direção a uma maior organização, mas um dos maiores benefícios que encontramos na utilização desse paradigma é o reuso.

Resumindo, a herança se utiliza de uma classe denominada classe filha, que tem os mesmos atributos e responsabilidades de outra classe chamada de classe mãe ou classe pai e os seus próprios atributos e responsabilidades. As classes inferiores da hierarquia, automaticamente, herdam todas as propriedades e os métodos das classes superiores, chamadas de superclasses, confira a figura a seguir.

Na figura anterior, temos uma classe no topo da hierarquia com o nome SuperClasse, logo abaixo, temos duas classes, Filho1 e Filho2, herdando de SuperClasse. Abaixo da classe Filho1, temos a classe FilhodeFilho1, herdando de Filho1. A herança é ilustrada na forma de uma seta que sai da classe filho e aponta para a classe pai.

A herança em PHP é representada pela palavra reservada *extends*, seguida do nome da classe que está sendo herdada. Nesse caso, Filho1 e Filho2 herdam de SuperClasse, e FilhodeFilho1 herda de Filho1. Confira a sintaxe a seguir.

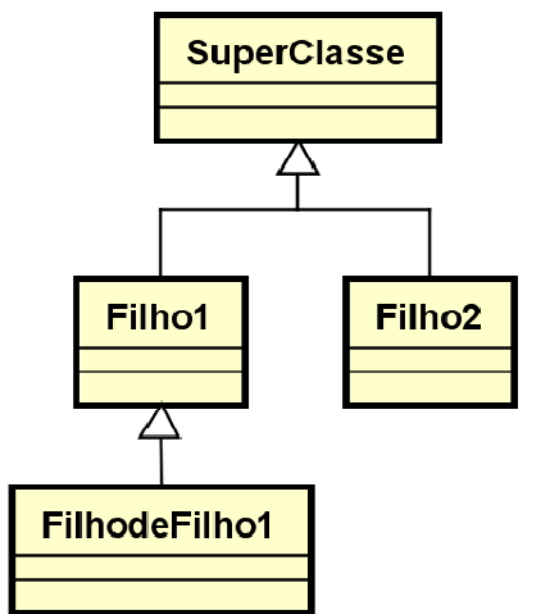


Figura 3 - Exemplo de herança / Fonte: os autores.

```

class SuperClasse{

}

-----

class Filho1 extends SuperClasse{

}

-----

class Filho2 extends SuperClasse{

}

-----

class FilhodeFilho1 extends Filho1{

}
  
```

Faremos um exemplo prático, confira a modelagem a seguir.

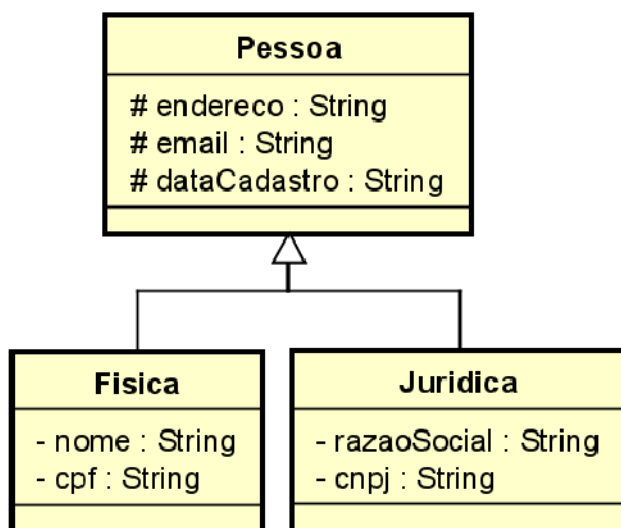


Figura 4 - Exemplo de herança
Fonte: os autores.

Primeiro, vamos entender a modelagem. A classe **Pessoa** é a classe que está no topo da hierarquia, assim, ela será considerada superclasse. Nela, os atributos (`endereco`, `email` e `dataCadastro`) estão com a visibilidade `protected`, ou seja, apenas as classes filhas possuem acesso a ela.

As duas classes que estão abaixo: **Fisica** e **Juridica**, são as classes filhas, e elas possuem seus atributos (`nome` e `cpf`) (`razaoSocial` e `cnpj`), respectivamente, com visibilidade `private`, e, pelo relacionamento de herança, elas possuirão os atributos da superclasse. Agora, vamos ao código.



conecte-se

Para elucidar melhor os conceitos sobre herança em Programação Orientada a Objetos, vejamos o link a seguir, o qual apresenta alguns exemplos da aplicação do conceito POO (Programação Orientada a Objetos).



Abra um novo arquivo e salve-o na pasta **Classes**, com o nome de **Pessoa.php**.

Arquivo: Pessoa.php

```
<?php
```

```

class Pessoa
{
    protected $endereco, $email, $dataCadastro;

    function __construct($endereco, $email, $dataCadastro)
    {
        $this->setEndereco($endereco);
        $this->setEmail($email);
        $this->setDataCadastro($dataCadastro);
    }
    private function setEndereco($endereco):bool{
        if(is_string($endereco)){
            $this->endereco = $endereco;
            return true;
        }
        else return false;
    }
    private function setEmail($email):bool{
        if(is_string($email)){
            $this->email = $email;
            return true;
        }
        else return false;
    }
    private function setDataCadastro($dataCadastro):bool{
        if(is_string($dataCadastro)){
            $this->dataCadastro = $dataCadastro;
            return true;
        }
        else return false;
    }
}

?>

```

Note que, da forma com que implementamos classe, utilizando parâmetros no construtor, e os métodos *setters* como privado, a única forma de entrada é pelo método construtor.

Agora, implementaremos a classe Física. Abra um novo arquivo e salve-o na pasta Classes, com o nome de **Fisica.php**.

Arquivo: Fisica.php

```
<?php
require_once('Pessoa.php');
class Fisica extends Pessoa
{
    private $nome, $cpf;
    function __construct($nome, $cpf, $endereco, $email,
        $dataCadastro)
    {
        parent::__construct($endereco, $email, $dataCadastro);
        $this->setNome($nome);
        $this->setCpf($cpf);
    }
    private function setNome($nome):bool{
        if(is_string($nome)){
            $this->nome = $nome;
            return true;
        }
        else return false;
    }
    private function setCpf($cpf):bool{
        if(is_string($cpf) && (strlen($cpf)==11)){
            $this->cpf = $cpf;
            return true;
        }
        else return false;
    }
}

?>
```

Note que foi necessário incluir a classe **Pessoa.php** nesse arquivo devido ao recurso de herança. Quando declaramos a classe **Fisica**, estendemos ela de **Pessoa** por meio da palavra reservada `extends`.

No método **construtor**, ele recebe todos os atributos de **Pessoa** mais os seus atributos por meio da assinatura `function __construct($nome, $cpf, $endereco, $email, $dataCadastro)`.

Dentro desse método, chamamos o construtor da classe pai utilizando a palavra-chave especial `parent::`, que é utilizada para acessar propriedades e métodos dentro de uma definição de classe.

Neste caso, ao utilizar o `parent::__construct($endereco, $email, $dataCadastro);`, passamos, para o construtor da classe pai, os seus respectivos atributos.

No método `setCpf`, fizemos uma validação a mais, pois, como sabemos, o campo CPF tem 11 dígitos, então o número passado deve ser exatamente 11.

Agora, faremos o arquivo da pessoa jurídica, para isso, abra um novo arquivo e salve-o na pasta **Classes** com o nome **Juridica.php**.

Arquivo: Juridica.php

```
<?php
require_once('Pessoa.php');
class Juridica extends Pessoa
{
    private $razaoSocial, $cnpj;
    function __construct($razaoSocial, $cnpj, $endereco, $email,
        $dataCadastro)
    {
        parent::__construct($endereco, $email, $dataCadastro);
        $this->setRazaoSocial($razaoSocial);
        $this->setCnpj($cnpj);
    }
    private function setRazaoSocial($razaoSocial):bool{
        if(is_string($razaoSocial)){
            $this->razaoSocial = $razaoSocial;
            return true;
        }
        else return false;
    }
}
```



```

    }
    private function setCnpj($cnpj):bool{
        if(is_string($cnpj) && (strlen($cnpj)==14)){
            $this->cnpj = $cnpj;
            return true;
        }
        else return false;
    }
}

?>

```

A única diferença entre as duas classes filhas, neste caso, são os atributos, e, para o método `setCnpj`, validamos o campo com 14 caracteres.

Agora, crie um arquivo novo e salve-o com o nome de `controlerPessoa.php`.

Arquivo: controler.php

```

<?php
require_once('classes/Fisica.php');

$pessoaFisica = new Fisica("Rafael Florindo", "12345678909",
"Rua 1", "rafaelflorindo@emai.com", "2010-05-24");
print_r($pessoaFisica);

?>

```

Após executar esse script, teremos a resposta:

```

Fisica Object
(
    [nome:Fisica:private] => Rafael Florindo
    [cpf:Fisica:private] => 12345678909
    [endereco:protected] => Rua 1
    [email:protected] => rafaelflorindo@emai.com
    [dataCadastro:protected] => 2010-05-24
)

```

Agora, você deverá fazer o mesmo para a pessoa Jurídica.

Classes abstratas

Estudamos e aplicamos o conceito de herança, agora, entenderemos como funciona o conceito de *abstract*. As classes definidas como abstratas não podem ser instanciadas, e qualquer classe que contenha, ao menos, um método abstrato também deve ser abstrata. Contudo, se temos uma classe que não pode ser instanciada, o que pode ser feito com ela? Só podemos entendê-la.

Uma classe abstrata serve como estrutura ou modelo para outras classes, por esse motivo, utilizamos ela com a herança. Dessa forma, marcaremos a classe super como *abstract*, e o interpretador do php não permitirá que se instancie um objeto de uma classe marcada com *abstract*.

Voltamos à nossa classe **Pessoa.php**. Realize as alterações.

```
<?php
abstract class Pessoa
{
    /*
    #####
    conteúdo já escrito no exemplo anterior
    - declaração dos atributos
    - método construtor e setters
    #####
    */
    abstract public function exibirDados();
}
?>
```

Note que colocamos o *abstract* antes do class e declaramos a assinatura de um método `exibirDados()` com o modificador `abstract`.

Agora, volte à classe **Fisica.php** e realize as alterações.

```
<?php
require_once('Pessoa.php');
final class Fisica extends Pessoa
{
    /*
```

```
#####
conteúdo já escrito no exemplo anterior
- declaração dos atributos
- método construtor e setters
#####
*/

public function exibirDados() {
    $dados = array("nome"=>$this->nome,
        "cpf"=>$this->cpf,
        "endereco"=>$this->endereco,
        "email"=>$this->email,
        "dataCadastro"=>$this->dataCadastro
    );
    return json_encode($dados);
}
}
?>
```

Note que, nesta classe, foi colocado o operador **final** antes da palavra *class*. E, também, fomos obrigados a implementar o método `exibirDados()` da classe *Pessoa*. Agora, caso não implemente esse método, o interpretador do PHP acusará um erro fatal dizendo que a classe **Fisica** deve implementar um método abstrato.

Fatal error: Class Fisica contains 1 abstract method and must therefore be declared abstract or implement the remaining methods (Pessoa::exibirDados) in **C:\xampp\htdocs\LivroBackEndII\UnidadeII\classes\Fisica.php** on line 3

Note que temos um comando novo, pois não estamos realizando impressão dentro dos métodos, assim precisamos retornar um *array*. Lembra do conceito de *array*?



quadro-resumo

Um array no PHP é na verdade um mapa ordenado. Um mapa é um tipo que relaciona valores a chaves. Este tipo é otimizado para vários usos diferentes: ele pode ser tratado como um array, uma lista (vetor), hashtable (que é uma implementação de mapa), dicionário, coleção, pilha, fila e provavelmente mais. Assim como existe a possibilidade dos valores do array serem outros arrays, árvores e arrays multidimensionais.

Um array pode ser criado com o construtor de linguagem `array()`. Ele leva qualquer quantidade de pares separados por vírgula chave => valor como argumentos.

Fonte: PHP ([2020], on-line).

Contudo, avançaremos e trabalharemos com o retorno de um **JSON** (*JavaScript Object Notation* ou *Notação de Objetos em JavaScript*), que nada mais é do que um *array* com uma formatação de dados muito leve, usada bastante na troca de informações entre aplicações, independente da linguagem.

Para isso, primeiro montamos o *array* `$dados`, com chave e valor ("chave" => "valor"), e, depois, encodamos no JSON por meio do comando `json_encode($dados)`. Essa função espera receber um parâmetro, que é a variável que será convertida em uma *string* JSON.

Agora, faremos o mesmo para a classe **Juridica.php**.

```
<?php
require_once('Pessoa.php');
final class Juridica extends Pessoa
{
    /*
    #####
    conteúdo já escrito no exemplo anterior
    - declaração dos atributos
    - método construtor e setters
    #####
    */
    public function exibirDados(){
        $dados = ["razaoSocial"=>$this->razaoSocial,
            "cnpj"=>$this->cnpj,
            "endereco"=>$this->endereco,
```

```

        "email"=>$this->email,
        "dataCadastro"=>$this->dataCadastro
    ];
    return json_encode($dados);
}

}

?>

```

Note que, nesta classe, colocou-se o operador **final** antes da palavra **class**, garantindo que nossa classe não será herdada por outras classes, e incluímos o método **exibirDados()** com suas características.

Aqui, alteramos as últimas três linhas, invocando o método e armazenando o retorno do JSON na variável `$dados` e, depois, dando um `echo` neste JSON.

Agora, vamos alterar o arquivo **controlerPessoa.php**.

```

<?php
require_once('classes/Fisica.php');

$pessoaFisica = new Fisica("Rafael Florindo", "12345678909",
"Rua 1", "rafaelflorindo@emai.com", "2010-05-24");

$dados = $pessoaFisica->exibirDados();
echo $dados;

?>

```

Executando esse arquivo, agora, teremos:

```

{
    "nome": "Rafael Florindo",
    "cpf": "12345678909",
    "endereco": "Rua 1",
    "email": "rafaelflorindo@emai.com",
    "dataCadastro": "2010-05-24"
}

```

4 POLIMORFISMO



Agora, estudaremos mais um dos pilares da Programação Orientada a Objetos, o polimorfismo. Esse pilar tem a capacidade de possibilitar ao desenvolvedor reutilizar partes de um método de sua classe-mãe, especializando com novas instruções. Contudo, as assinaturas devem ser idênticas para que o conceito possa ser validado como polimorfismo, caso contrário, podemos confundir esse conceito como aproveitamento de códigos, o qual também é muito utilizado no desenvolvimento.

Para Dall'Oglio (2009, p. 101), ele define polimorfismo como:



Polimorfismo em orientação a objetos é o princípio que permite que classes derivadas de uma mesma superclasse tenham métodos iguais (com a mesma nomenclatura e parâmetros), mas comportamentos diferentes, redefinidos em cada uma das classes-filhas.

Para Bezerra (2007, p. 11), “polimorfismo indica a capacidade de abstrair várias implementações diferentes em uma única interface”.

Resumindo, um método de uma classe herdeira que possui a mesma assinatura pode fazer a operação da superclasse e mais alguma instrução própria. Contudo, como fazer isso em PHP?

Neste caso, utilizaremos o comando `parent::` seguido do nome do método. Vamos a um exemplo novo. Analisando a Figura 5, temos a classe `Conta`, com os atri-

butos `agencia`, `numeroConta` e `titular`, todas com visibilidade `protected` e tipo `String`, e o atributo `saldo`, também `protected` e tipo `double`. Ainda na classe `Conta`, temos dois métodos (`depositar` e `retirar`), ambos esperam receber o parâmetro `valor` do tipo `double`. Ao término, as duas classes devem retornar um valor `boolean`.

Além dessa classe `Conta`, temos a classe `ContaCorrente`, que herda de `Contas`. Na classe `ContaCorrente`, há o atributo `limite` com o tipo `double` e visibilidade `protected`. Ainda na classe `Conta`, temos o método (`retirar`), que espera receber o parâmetro `valor` do tipo `double`. Ao término do método, deve ser retornado um valor `boolean`. Nesse ponto, identificamos o polimorfismo, pois, em ambas as classes, o método está com a mesma nomenclatura.

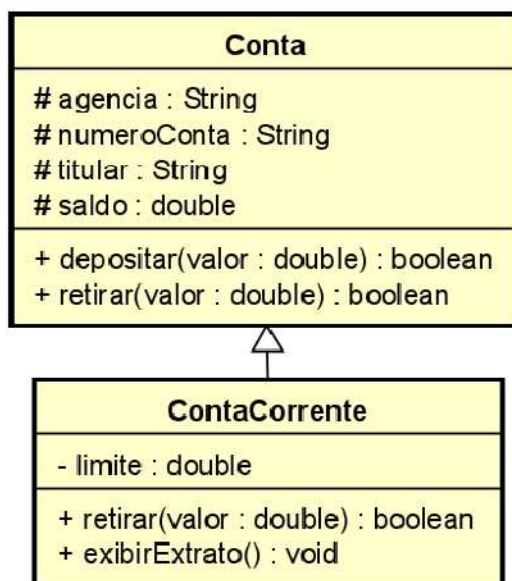


Figura 5 - Polimorfismo / Fonte: os autores.

Agora que já analisamos a modelagem, crie as classes conforme os códigos a seguir e as salve na pasta `classes`.

Confira o código da classe `Conta`.

Arquivo: `Conta.php`

```

<?php
class Conta{
    protected $agencia, $numeroConta, $titular, $saldo;

```

```

        function __construct($agencia, $numeroConta, $titular,
$saldo)
        {
            $this->setAgencia($agencia);
            $this->setNumeroConta($numeroConta);
            $this->setTitular($titular);
            $this->setSaldo($saldo);
        }

public function depositar($valor):bool{
    if(is_float($valor) > 0){
        $this->setSaldo($this->getSaldo() + $valor);
        return true;
    }
    else
        return false;
}

public function retirar($valor):bool{
    if(is_float($valor) > 0){
        if($this->getSaldo() > 0){
            $this->setSaldo($this->getSaldo() - $valor);
            return true;
        }
        else
            return false;
    }else
        return false;
}

/*
    implementar os métodos acessores,todos com visibilidade
    public e com os teste e retorno do tipo String
    */
}
?>

```


Note que, tanto no método `depositar` quanto no método `retirar`, iniciamos validando o tipo do valor recebido `is_float($valor)` e se ele for maior que 0. Caso o resultado seja satisfeito, será executada a instrução que seta o resgate do saldo, adicionando o valor recebido `$this->setSaldo($this->getSaldo() + $valor)`; e, por fim, devolvemos um *boolean* com `true`, caso contrário, retornamos `false`. No método `retirar`, o processo é o mesmo, porém, em vez de adicionar, será subtraído o valor recebido.

Agora, confira o código da classe `ContaCorrente`.

Arquivo: ContaCorrente.php

```
<?php
require_once('Conta.php');

class ContaCorrente extends Conta{
    private $limite;

    function __construct($agencia, $numeroConta, $titular,
        $saldo, $limite)
    {
        parent::__construct($agencia, $numeroConta, $titular,
            $saldo);

        $this->setLimite($limite);
    }

    public function retirar($valor):bool{
        if (($this->getSaldo()+$this->getLimite()) > is_float($valor)){
            parent::retirar($valor);
            if (true) return true;
            else return false;
        }else
            return false;
    }

    public function exibirExtrato(){
        $dados = ["Titular"=>$this->getTitular(),
            "Agencia"=>$this->getAgencia(),
            "Conta"=>$this->getNumeroConta(),
            "Saldo"=>$this->getSaldo()
        ];
    }
}
```

```

        return json_encode($dados);
    }

    /*
    implementar os métodos acessores,todos com visibilidade
    public e com os teste e retorno do tipo String
    */
}
?>

```

Note que, no método `retirar`, iniciamos validando o tipo se a soma do saldo + o limite é maior que do valor recebido `if (($this->getSaldo()+$this->getLimite()) > $valor);`. Caso o resultado seja satisfeito, será executado a instrução `parent::retirar($valor);`, que chama o método `retirar` e passa para ele o valor, nesse ponto, ocorre o polimorfismo. E, por fim, devolvemos um *boolean* com `true`, caso contrário, retornamos `false`. Por fim, temos o método `exibirExtrato()`, que retorna um **JSON** com todos os atributos.

Agora, confira o código de controleConta.

```

Arquivo: ControlerConta.php

<?php
require_once('classes/ContaCorrente.php');
$cliente1 = new ContaCorrente("3284-0","12456-8","José",
1500.05,500.00);
$cliente1->depositar(500.25);
$results = $cliente1->exibirExtrato();
$cliente1->retirar(1000.25);
$results = $cliente1->exibirExtrato();
echo $results;
?>

```

5 APLICANDO NA PRÁTICA

Agora, chegou a hora de praticarmos os conteúdos internalizados nesta unidade do livro. Para isso, analisaremos o diagrama de classe a seguir:

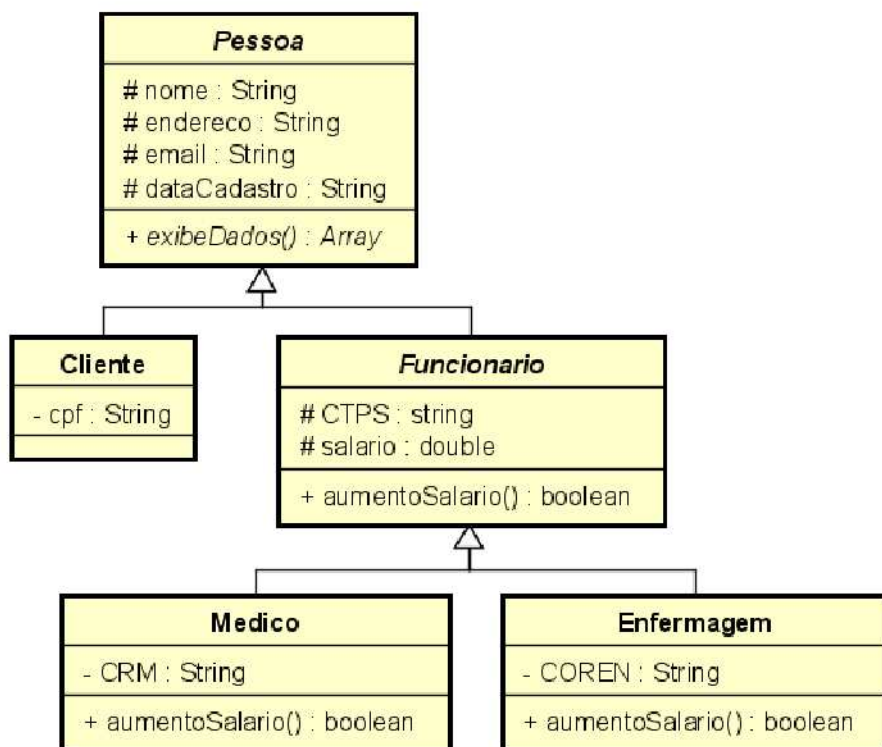


Figura 6 - Modelagem do diagrama de classe / Fonte: os autores.

Neste diagrama, temos duas classes abstratas (Pessoa e Funcionário) e três classes concretas (Cliente, Médico e Enfermagem). É possível notar que possuímos o relacionamento de herança. Vejamos: as classes Cliente e Funcionario herdam de Pessoa, e as classes Medico e Enfermagem herdam de Funcionário.

- Na classe Pessoa, foram atribuídos os atributos: nome, endereço, email e dataCadastro, todos do tipo String e com visibilidade Protected. Ainda na classe Pessoa, temos um método abstrato `exibirPessoa()`.
- Na classe Cliente, temos apenas o atributo cpf do tipo String e com visibilidade private.
- Na classe Funcionario, temos os atributos CPTS e salario, ambos do tipo String e com visibilidade protected. Ainda na classe Funcionario, temos um método abstrato `aumentoSalario()`, e este, por sua vez, retorna um Boolean.
- A classe Medico possui um atributo CRM do tipo String e com visibilidade private. Ainda na classe Medico, temos um método abstrato `aumentoSalario()`, e este, por sua vez, retorna um Boolean.
- A classe Enfermagem possui um atributo COREN do tipo String e com visibilidade private. Ainda na classe Enfermagem, temos um método abstrato `aumentoSalario()`, e este, por sua vez, retorna um Boolean.

Agora, chegou a hora de criarmos os códigos. Para isso, abra novos arquivos para as classes `Pessoa.php`, `Funcionario.php`, `Cliente.php`, `Medico.php` e `Enfermagem.php` e salve-as na pasta `classes`.

Arquivo: Pessoa.php

```
<?php
abstract class Pessoa
{
    protected $nome, $endereco, $email, $dataCadastro;
    function __construct($nome, $endereco, $email, $dataCadastro)
    {
        $this->setNome($nome);
        $this->setEndereco($endereco);
```

```

        $this->setEmail($email);
        $this->setDataCadastro($dataCadastro);
    }
    private function setNome($nome):bool{
        if(is_string($nome)){
            $this->nome = $nome;
            return true;
        }
        else return false;
    }
    private function setEndereco($endereco):bool{
        if(is_string($endereco)){
            $this->endereco = $endereco;
            return true;
        }
        else return false;
    }
    private function setEmail($email):bool{
        if(is_string($email)){
            $this->email = $email;
            return true;
        }
        else return false;
    }
    private function setDataCadastro($dataCadastro):bool{
        if(is_string($dataCadastro)){
            $this->dataCadastro = $dataCadastro;
            return true;
        }
        else return false;
    }
    abstract public function exhibirDados();
}
?>

```

Na classe `Pessoa`, temos a declaração dos atributos, os métodos de acesso, o construtor e a declaração da assinatura método abstrato `exibeDados()`.

Arquivo: `Cliente.php`

```
<?php
require_once('Pessoa.php');
class Cliente extends Pessoa{
    private $cpf;

    function __construct($nome, $endereco, $email, $dataCadastro,
$cpf)
    {
        parent::__construct($nome, $endereco, $email,
$dataCadastro);
        $this->setCpf($cpf);
    }
    private function setCpf($cpf):bool{
        if(is_string($cpf) && (strlen($cpf)==11)){
            $this->cpf = $cpf;
            return true;
        }
        else return false;
    }
    public function exibirDados(){
        $dados = ["nome"=>$this->nome,
            "cpf"=>$this->cpf,
            "endereco"=>$this->endereco,
            "email"=>$this->email,
            "dataCadastro"=>$this->dataCadastro
        ];
        return json_encode($dados);
    }
}
?>
```

Na classe `Cliente`, temos a declaração dos atributos, os métodos de acesso, o construtor e a implementação do método `exibeDados()`.

Arquivo: Funcionario.php

```
<?php
require_once('Pessoa.php');
class Funcionario extends Pessoa{
    protected $ctps, $salario;

    function __construct($nome, $endereco, $email, $dataCadastro,
        $ctps, $salario)
    {
        parent::__construct($nome, $endereco, $email,
            $dataCadastro);
        $this->setCtps($ctps);
        $this->setSalario($salario);
    }
    private function setCtps($ctps):bool{
        if(is_string($ctps)){
            $this->ctps = $ctps;
            return true;
        }
        else return false;
    }
    private function setSalario($salario):bool{
        if(is_float($salario)){
            $this->salario = $salario;
            return true;
        }
        else return false;
    }
    public function exibirDados(){
    }
    public function aumentoSalario($valor):bool{
        if(is_float($valor) > 0){
            $this->setSalario($this->salario + $valor);
            return true;
        }
    }
}
```

```

        }
        else
            return false;
    }
}
?>

```

Na classe `Funcionario`, temos a declaração dos atributos, os métodos de acesso, o construtor e a implementação do método `exibirDados()` sem instrução, pois os implementaremos nas classes filhas, e, ainda, temos a implementação do método `aumentoSalario`, que recebe o valor do aumento, verifica se ele é um número real e se ele é maior que zero, para, depois, acrescentá-lo no atributo `salario`. Após acrescentar, devolve um `boolean`.

Arquivo: `Medico.php`

```

<?php
require_once('Funcionario.php');
class Medico extends Funcionario{
    protected $crm;

    function __construct($nome, $endereco, $email, $dataCadastro,
        $ctps, $salario, $crm)
    {
        parent::__construct($nome, $endereco, $email,
            $dataCadastro, $ctps, $salario);
        $this->setCrm($crm);
    }

    private function setCrm($crm):bool{
        if(is_string($crm)){
            $this->crm = $crm;
            return true;
        }
        else return false;
    }

    public function exibirDados(){
        $dados = ["nome"=>$this->nome,
            "endereco"=>$this->endereco,
            "email"=>$this->email,

```



```

        "ctps"=>$this->ctps,
        "crm"=>$this->crm,
        "salario"=>$this->salario,
        "dataCadastro"=>$this->dataCadastro
    ];
    return json_encode($dados);
}

public function aumentoSalario($valor):bool{
    if(parent::aumentoSalario($valor))
        return true;
    else
        return false;
}
}
?>

```

Na classe `Medico`, temos a declaração do atributo, os métodos de acesso, o construtor e a implementação do método `exibeDados()`, e, ainda, temos a implementação do método `aumentoSalario`, que recebe o valor do aumento e invoca o método da classe `Funcionário`. Após acrescentar, devolve um boolean.

Arquivo: Enfermagem.php

```

<?php
require_once('Funcionario.php');
class Enfermagem extends Funcionario{
    protected $coren;

    function __construct($nome, $endereco, $email, $dataCadastro,
        $ctps, $salario, $coren)
    {
        parent::__construct($nome, $endereco, $email,
            $dataCadastro, $ctps, $salario);
        $this->setCoren($coren);
    }

    private function setCoren($coren):bool{
        if(is_string($coren)){
            $this->coren = $coren;
            return true;
        }
    }
}

```

```

        }
        else return false;
    }

    public function exhibirDados(){
        $dados = ["nome"=>$this->nome,
            "endereco"=>$this->endereco,
            "email"=>$this->email,
            "ctps"=>$this->ctps,
            "coren"=>$this->coren,
            "dataCadastro"=>$this->dataCadastro
        ];
        return json_encode($dados);
    }

    public function aumentoSalario($valor):bool{
        if(parent::aumentoSalario($valor))
            return true;
        else
            return false;
    }
}
?>

```

Na classe Enfermagem, temos a declaração do atributo, os métodos de acesso, o construtor e a implementação do método `exibeDados()`, e, ainda, temos a implementação do método `aumentoSalario`, que recebe o valor do aumento e invoca o método da classe `Funcionário`. Após acrescentar, devolve um `boolean`.

Arquivo: controlerCliente.php

```

<?php
require_once('classes/Cliente.php');

$paciente = new Cliente("Rafael Florindo", "Rua 1",
"rafaelflorindo@emai.com", "2010-05-24", "12345678909");
$results = $paciente->exibirDados();
echo $results;

```

```
?>
```

Executando o arquivo, teremos:

```
{
    "nome": "Rafael Florindo",
    "cpf": "12345678909",
    "endereco": "Rua 1",
    "email": "rafaelflorindo@emai.com",
    "dataCadastro": "2010-05-24"
}
```

Arquivo: controlerMedico.php

```
<?php
require_once('classes/Medico.php');

$medico = new Medico("Rafael Florindo", "Rua 1", "rafaelflorindo@
emai.com", "2010-05-24", "ctps 123", 4500.85, "CRM 45789");
$results = $medico->exibirDados();
echo $results;

$medico->aumentoSalario(1200.25);
echo "Novo Salário: " . $medico->getSalario();

?>
```

Ao executar o arquivo, teremos como resultado:

```
{
    "nome":"Rafael Florindo",
    "endereco":"Rua 1",
    "email":"rafaelflorindo@emai.com",
    "ctps":"ctps 123",
    "crm":"CRM 45789",
    "salario":4500.85,
    "dataCadastro":"2010-05-24"
}
```

Novo Salário = 5701.1

Arquivo: controlerEnfermagem.php

```
<?php
require_once('classes/Enfermagem.php');

$enfermagem = new Enfermagem("Rafael Florindo", "Rua 1",
"rafaelflorindo@emai.com", "2010-05-24", "ctps 123", 4500.85,
"COREN 45789");

$results = $enfermagem->exibirDados();
echo $results;

$enfermagem->aumentoSalario(200.25);
echo "Novo Salário: " . $enfermagem->getSalario();
?>
```

Ao executar o arquivo, teremos como resultado:

```
{
  "nome":"Rafael Florindo",
  "endereco":"Rua 1",
  "email":"rafaelflorindo@emai.com",
  "ctps":"ctps 123",
  "coren":"COREN 45789",
  "salario":4500.85,
  "dataCadastro":"2010-05-24"
}
```

Novo Salário = 4701.1

Esperamos que você tenha conseguido acompanhar todo o nosso raciocínio do conteúdo abordado. Caso tenha ficado com alguma dúvida, sugiro voltar ao início da unidade e rever o conteúdo, além de refazer os códigos.

CONSIDERAÇÕES FINAIS

Como vimos anteriormente, o paradigma de Orientação a Objetos traz como vantagens a reusabilidade, extensibilidade, o aumento da qualidade e, por consequência, vantagens financeiras. Nesta unidade, abordamos aspectos fundamentais do paradigma OO e como o conceito de abstração está ligado diretamente a ele. A compreensão inicial de um software está se tornando cada vez mais complexa; transformar as ideias em componentes de um sistema não é uma tarefa fácil. Nesse cenário, o controle da complexidade se torna uma questão de sobrevivência para a evolução e manutenção de sistemas de controle e informação.

Trabalhamos, também, os conceitos sobre encapsulamento, um processo identificado como boas práticas de programação e, também, associado à proteção de código e segurança de acesso. Em relação ao nível de acesso, temos um dilema muito grande: deixar tudo público ou proteger tudo. Compreenda, prezado(a) aluno(a), que temos que encontrar um meio termo, nem tudo aberto não é muito restrito.

Outro conceito que analisamos que é tido como um dos principais do paradigma OO é o de herança. Por meio desse processo, aplicamos a ideia de reuso de código e extensão de métodos, contexto fundamental para o uso de Programação PHP com OO. Associado a isso, estudamos sobre polimorfismo em que é possível permitir que classes derivadas de uma mesma superclasse tenham métodos iguais, mas sem os instanciar.

Tudo isso parece confuso, por isso, preparamos um tópico especial com um material que possa enriquecer seus conhecimentos, levando você a analisar a aplicação prática desses conceitos, partindo da visão de um diagrama UML para a construção de diversos arquivos PHP, aplicando os conceitos vistos na unidade.

Ficamos muito felizes em poder compartilhar com você um pouco mais de conhecimento. Vemo-nos em nossa próxima unidade. Até lá, bons estudos!



1. O Paradigma de Orientação a Objetos, atualmente, é o mais difundido entre todos. Isso acontece porque se trata de um padrão que tem evoluído muito, principalmente, em questões voltadas para segurança e reaproveitamento de código, o que é muito importante no desenvolvimento de qualquer aplicação moderna.

De acordo com o contexto anterior, associe as duas colunas, relacionando os pilares do Paradigma Orientação a Objetos com o seu significado.

Coluna 1

- 1 - Abstração
- 2 - Encapsulamento
- 3 - Herança
- 4 - Polimorfismo

Coluna 2

- () Trata-se de um dos elementos que adicionam segurança à aplicação em uma Programação Orientada a Objetos pelo fato de esconder as propriedades.
 - () A complexidade de um objeto é escondida "por trás" de suas operações.
 - () É a capacidade pela qual duas ou mais classes derivadas da mesma superclasse podem invocar métodos que têm a mesma identificação e assinatura, mas que possuem comportamentos distintos.
 - () O reuso de código é uma das grandes vantagens da Programação Orientada a Objetos entre pai e filho.
- a) 1, 2, 3, 4.
 - b) 2, 1, 4, 3.
 - c) 3, 4, 1, 2.
 - d) 4, 2, 3, 1.
 - e) 4, 3, 2, 1.



2. Analise a classe a seguir:

Conta
- saldo : float - titular : String - cpfTitular : String - numConta : int
+ sacar(valorSaque : float) : void + depositar(valorDep : float) : void + verificarSaldo() : void + transferir(numContaDestino : int, valor : float) : float

Diante da modelagem anterior, podemos afirmar que os atributos do objeto atendem ao controle de acesso (proteção dos dados) referente ao conceito de encapsulamento caso a classe seja acessada por outro objeto, de uma outra classe, dentro do mesmo pacote?

- Sim, atende. Um objeto de outra classe acessa os atributos de forma pública.
- Atende parcialmente. Se implementar os métodos *set* e *get* para cada atributo, garantirá o acesso restrito, ou seja, os atributos não poderão ser acessados diretamente.
- Não, para que haja uma proteção do controle de acesso aos atributos, o desenvolvedor deve colocar os atributos como *private*, garantindo, assim, o encapsulamento.
- Não, para que haja uma proteção do controle de acesso aos atributos, o desenvolvedor deve colocar os atributos como *default*, garantindo, assim, o encapsulamento.
- Não, para que haja uma proteção do controle de acesso aos atributos, o desenvolvedor deve colocar os atributos como *protected*, garantindo, assim, o encapsulamento.



3. A Orientação a Objetos nada mais é do que uma forma de programar em que se visa modelar os processos de programação, aproximando-se da realidade. Essa forma de programar trata cada componente de um programa como um objeto, que, por sua vez, possuirá suas próprias características e funcionalidades. Um objeto é a instância de uma classe, e ele é composto de métodos (comportamentos) e atributos. Além disso, é possível utilizar de recursos de classes concretas e abstratas, recurso de interface, polimorfismo, entre outros recursos da Orientação a Objetos.

Diante do contexto anterior, avalie as afirmações a seguir sobre o uso de classes abstratas em Java.

I - Uma classe abstrata pode conter atributos e métodos abstratos.

II - Uma classe abstrata pode ser herdada e implementar uma interface.

III - Uma classe abstrata pode conter métodos e atributos de classe.

IV - Uma classe abstrata pode ser estendida por uma classe concreta.

V - Uma classe abstrata pode ser instanciada.

É correto o que se afirma em:

- a) I, apenas.
 - b) I e II, apenas.
 - c) II e IV, apenas.
 - d) I, II, III e IV, apenas.
 - e) I, II, III, IV e V.
4. Uma das características da Programação Orientada a Objetos está relacionada com a proteção dos atributos internos dos objetos contra modificações diretas. As alterações dos atributos devem ocorrer por meio de métodos adequados, criados para acesso e modificação desses atributos.

Dentro do contexto anterior, essa característica é conhecida como:

- a) Encapsulamento.
- b) Generalização.
- c) Herança.
- d) Polimorfismo.
- e) Sobrecarga de operador.

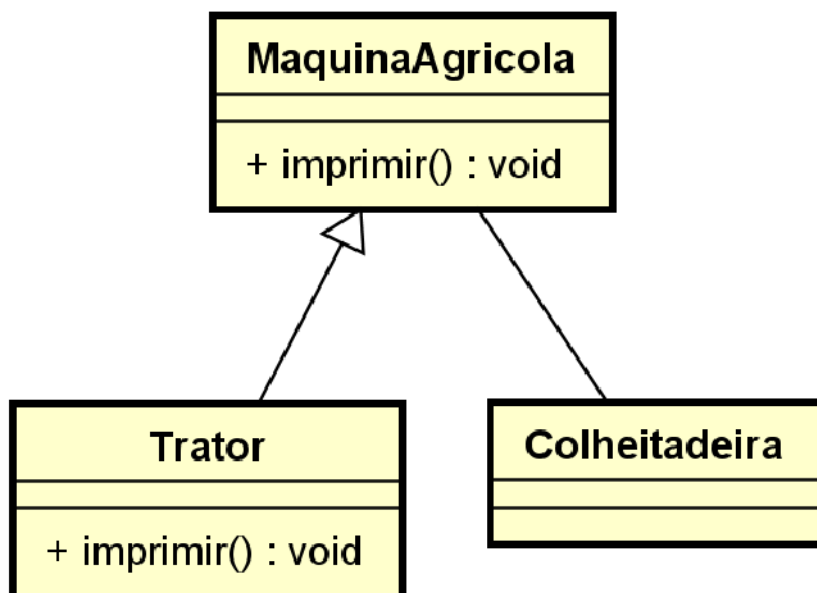


5. “O encapsulamento, na maioria das linguagens de programação orientada a objetos, é praticado utilizando métodos especiais conhecidos por getters e setters, onde o método setter é responsável por alterar ou inserir valores nos dados do objeto, e o método getter é utilizado para recuperar valores encapsulados no objeto e pode ser implementado em três níveis de acesso: Privado, Público e Protegido”.

PERSEGUINE, V. R. **Modelagem de Software**. Maringá: Unicesumar, 2018. p. 75.

Com base no exposto, explique o tipo de modificador utilizado para que um atributo seja visível apenas dentro da classe.

6. Classes mães e herdeiras podem compartilhar métodos com o mesmo nome, mas que executem operações diferentes. Isso é o conceito de polimorfismo, ou seja, um método de uma classe herdeira poder fazer a mesma operação da classe mãe e mais alguma coisa. Analise a imagem a seguir:



De acordo com o contexto de polimorfismo, temos o método `imprimir()` nas classes `MaquinaAgricola` e `Trator`, desta forma, implemente apenas o método `imprimir` da classe `Trator` de forma que aplique o conceito de polimorfismo.



PROPOSTA DE UMA ARQUITETURA DE SOFTWARE E FUNCIONALIDADES PARA IMPLEMENTAÇÃO DE UM AMBIENTE INTEGRADO DE DESENVOLVIMENTO PARA A LINGUAGEM PHP

A concepção de sistemas para Web confiáveis e de alta qualidade, requer, fundamentalmente, a adoção de uma linguagem de programação e de uma ferramenta de desenvolvimento adequadas. O presente trabalho propõe o estudo dos principais IDE existentes para a linguagem PHP e suas metodologias de implementação. Alguns IDE e editores, livres ou proprietários, implementados sob os paradigmas de programação estruturada ou orientada a objetos, são apresentados, expondo as principais características de cada um.

A utilização de um Ambiente de Desenvolvimento Integrado (do inglês *Integrated Development Enviroment* - IDE) adequado para o desenvolvimento de software é de suma importância em um desenvolvimento de sucesso, principalmente quando o desenvolvimento é focado para Web, que geralmente apresenta uma maior complexidade que os sistemas *stand alone*.

Uma das linguagens mais utilizadas atualmente para o desenvolvimento de sites Web é a PHP. Entretanto, pela inexistência de um IDE eficiente para uso desta linguagem, um desenvolvedor PHP faz uso de uma série de ferramentas no processo de desenvolvimento de software. A alternância entre um aplicativo e outro costuma diminuir o rendimento do programador. Uma solução eficiente para este problema é encontrada quando todas, ou pelo menos a maioria das ferramentas necessárias, encontram-se num mesmo ambiente.

Atualmente, existem editores e IDE para PHP, mas estes não são completos. Cada um deles trata o desenvolvimento de aplicações de forma particular, porém análoga.

Este trabalho tem como objetivo analisar de forma exploratória alguns ambientes de desenvolvimento de software para a linguagem PHP, do ponto de vista do desenvolvedor, bem como da metodologia de desenvolvimento, a fim de propor uma série de requisitos funcionais e não funcionais para construção de um IDE. Também uma arquitetura de software para implementação do mesmo é proposta.



eu recomendo!



livro

Construindo uma Aplicação Web Completa com PHP e MySQL

Autor: William Pereira Alves

Editora: NOVATEC

Sinopse: este livro oferece ao leitor o conhecimento necessário para desenvolver aplicações para a internet utilizando o servidor Web Apache, o servidor de banco de dados MySQL e a linguagem PHP. Com uma linguagem simples e objetiva e capítulos repletos de figuras e exercícios, a obra proporciona fácil entendimento acerca dos fundamentos de gestão de projeto, dos conceitos de HTML e do processo de instalação do Apache, MySQL e PHP no ambiente Windows. Você será capaz de desenvolver um projeto de aplicação para uma rede de hotéis, da concepção à implantação.



livro

PHP Para Quem Conhece PHP: recursos avançados para a criação de websites dinâmicos

Autor: Juliano Niederauer

Editora: NOVATEC

Sinopse: apresenta recursos avançados desta poderosa linguagem de programação para a Web. Aborda diversos assuntos úteis ao desenvolvedor, como cookies e sessões, upload de arquivos, geração de imagens e gráficos, arquivos PDF, templates, abstração de bancos de dados, entre outros. Além disso, contém uma abrangente revisão sobre PHP para aqueles que tiveram pouco contato com a linguagem. Assim como nas demais obras do autor, neste livro, a didática também é o ponto forte, o que permite ao leitor o rápido entendimento de assuntos aparentemente complexos. Para facilitar a compreensão dos conteúdos, são apresentados vários exemplos de programas e, ao final, um estudo de caso envolvendo diversos recursos da linguagem.





RELACIONAMENTOS

PROFESSORES

Me. Rafael Alves Florindo

Esp. Carlos Danilo Luz

PLANO DE ESTUDO ▼

A seguir, apresentam-se as aulas que você estudará nesta unidade: • Associação com Multiplicidades. • Agregação e Composição • Interface e Componentes • Classes Associativas • Aplicando na Prática.

OBJETIVOS DE APRENDIZAGEM ▼

Identificar quais os tipos de relacionamentos e suas multiplicidades entre classes • Diferenciar os relacionamentos de agregação e composição e quando utilizá-los na programação • Compreender quais são os pontos de ligação no back end do sistema e seus componentes de implementação • Apresentar ao aluno como implementar as classes associativas • Levar o aluno à prática de uma classe envolvendo os conceitos aprendidos nesta unidade.



Seja bem-vindo(a), aluno(a)! É um imenso prazer contar com você novamente. Anteriormente, estudamos os conceitos de encapsulamento, modificadores de acesso, conceitos de herança e polimorfismo. Acredito que, para alguns, esse conteúdo possa ser um pouco confuso, mas, com o tempo, você verá que se torna conceitos aplicáveis em nosso cotidiano de desenvolvedores. Nesta unidade, aprimoraremos nossos saberes ao conhecer sobre multiplicidade, agregação, composição, interface e componentes.

O primeiro aspecto que devemos compreender é que, dificilmente, uma classe atua de modo isolado em um sistema. Geralmente, é associada a diversas outras. Basicamente, a herança, a qual possibilita que classes com características e comportamentos semelhantes possam ser implementadas de forma agrupada, seria uma boa representação de associação entre classes.

Por termos diversas classes sendo interligadas, aplicamos uma notação visual em um diagrama de classe de UML, que indica o grau de execução com que aquela classe poderá ser instanciada. Assim, temos um entendimento de como o sistema poderá operar, isso se refere à multiplicidade.

Ao atuarmos com as notações de agregação e composição, pode-se dizer que seria a aplicação da ideia de herança, com a diferença de que elementos de agregação dependem da classe principal para coexistir. Já os de composição, mesmo se a sua classe principal não existir, poderiam continuar instanciados e sendo acessíveis.

Por fim, trabalharemos a estruturação da programação PHP OO com a utilização de interfaces e componentes e, também, com classes associativas. Deve-se ter uma atenção especial para as classes associativas, pois elas têm como função o ponto central de ligação de duas classes ou mais, sendo elas as concentradoras das informações das demais.

Para mostrar a você como utilizar esses recursos, apresentaremos uma aplicação prática deles para melhor absorção dos conhecimentos. Trabalharemos com diagramação UML para as devidas interpretações, a fim de mostrar como aplicar em um desenvolvimento web com PHP OO.

1

ASSOCIAÇÃO COM
MULTIPLICIDADES

De acordo com Dall'Oglio (2009, p. 117), temos que associação pode ser definida como “relação mais comum entre dois objetos, de modo que um possui uma referência à posição da memória onde o outro se encontra, podendo visualizar seus atributos ou mesmo acionar uma de suas funcionalidades”.

Segundo Melo (2004), a associação é um relacionamento que conecta duas ou mais classes, demonstrando a colaboração entre as instâncias de cada classe. Pode, também, haver um relacionamento de uma classe com ela mesma, e, nesse caso, tem-se uma associação unária ou reflexiva.

Na Figura 1, apresentamos um exemplo de uma associação entre classes. Nesta, temos as classes `Equipamento` e `TipoEquipamento` interligadas por uma linha sólida. Note que essa linha não possui setas em suas extremidades, isso significa que devemos amarrar o relacionamento entre as classes, ou seja, uma classe deve enxergar a outra. Para que isso seja possível, será necessário criar um atributo em cada classe que armazenará a referência do objeto da outra classe.





Figura 1 - Exemplo de associação / Fonte: os autores.

Ainda na Figura 1, a classe `Equipamento` possui os atributos `codigoEquipamento` do tipo inteiro, `descricao` e `dataAquisicao` do tipo *string*. Enquanto a classe `TipoEquipamento` possui os atributos `codigoTipoEquipamento` do tipo inteiro, e `descricao` do tipo *string*.

Antes de continuarmos, é necessário recordarmos dos conceitos de multiplicidades. A multiplicidade de uma classe é o número de instâncias possíveis que uma classe pode ter considerando uma única instância da outra classe à qual ela é associada. O quadro a seguir exemplifica os tipos comuns de multiplicidades.

Tipos	Significa
0..1	Zero ou uma instância. A notação <i>n..m</i> indica <i>n</i> para <i>m</i> instâncias. Os objetos não precisam estar relacionados, porém, se houver relacionamento, deve ser de, no máximo, 1.
0..* ou *	Não existe limite para o número de instâncias.
1..1	Exatamente uma instância.
1..*	Ao menos uma instância.

Quadro 1 - Notações possíveis / Fonte: os autores.

A seguir, apresentamos os tipos de associações.

Associação Unária ou Reflexiva

Segundo Guedes (2011), a associação unária ocorre quando existe um relacionamento de uma instância de uma classe com instâncias da mesma classe, conforme ilustra a figura a seguir:

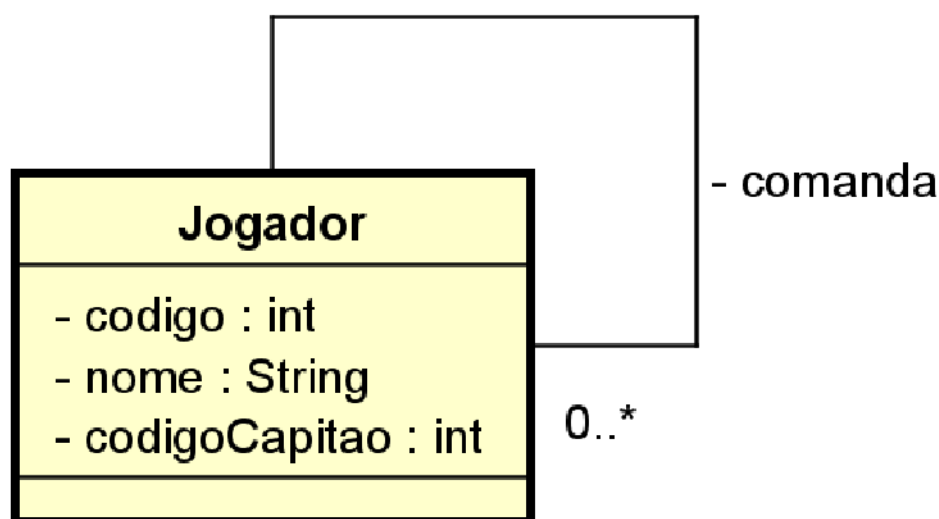


Figura 2 - Exemplo de associação unária / Fonte: os autores.

No exemplo apresentado, temos a classe *Jogador*, cujos atributos são código, nome e o código do possível capitão do jogador. Pelo fato de que esse capitão também é um jogador, a associação chamada *comanda* indica uma possível relação entre uma ou mais instâncias da classe *Jogador* com outras instâncias da mesma classe, ou seja, tal associação determina que um jogador pode ou não comandar outros jogadores. Além disso, essa associação faz com que a classe possua o atributo *codigoCapitao* para armazenar o código do jogador, que é o responsável pela instância do jogador em questão. Desse modo, após consultar uma instância da classe *Jogador*, pode-se utilizar o atributo *codigoCapitao* da instância consultada para pesquisar por outra instância da classe (GUEDES, 2011).

Vamos à implementação:

Arquivo: Jogador.php

```

<?php

class Jogador
{
    private $codigo, $nome, $codigoCapitao;

    public function __construct($codigo, $nome)
    {
        $this->setCodigo($codigo);
        $this->setNome($nome);
        $this->codigoCapitao=null;
    }
    public function __toString()
    {
        $dados = [
            "codigo"=>$this->getCodigo(),
            "nome"=>$this->getNome(),
            "Capitao"=>$this->getCodigoCapitao()
        ];
        return json_encode($dados);
    }
}
/*
Implementar os demais métodos de acesso (setters e getters)
*/

public function setCodigoCapitao(Jogador $codigoCapitao)
{
    $this->codigoCapitao = $codigoCapitao;
}
public function getCodigoCapitao()
{
    return $this->codigoCapitao->getNome();
}

```

Para garantir a associação unária, no método `setCodigoCapitao(Jogador $codigoCapitao)`, recebemos um objeto de capitão e armazenamos em seu respectivo atributo. Em seguida, no método `getCodigoCapitao()`, retornamos o conteúdo do objeto com o comando `return $this->codigoCapitao->getNome();`.

Arquivo: `controllerJogador.php`

```
<?php

require_once('classes/Jogador.php');

$jogador1 = new Jogador(10, "Rafael");
$jogador2 = new Jogador(12, "Carlos");
$jogador3 = new Jogador(8, "Flávia");

$jogador1->setCodigoCapitao($jogador3);
$jogador2->setCodigoCapitao($jogador3);

echo $jogador2;
```

Associação binária

A associação binária conecta duas classes por meio de uma reta que liga uma classe à outra. Lembra da Figura 1? Voltaremos a ela agora. Veja que, nesta figura, temos a ligação entre duas classes, perfazendo a associação binária:



No exemplo, um objeto da classe Equipamento pode se relacionar com uma e apenas uma instância da classe TipoEquipamento, conforme demonstra a multiplicidade **1**. Entretanto, um objeto da classe TipoEquipamento pode se relacionar com zero ou muitas instâncias da classe Equipamento, conforme demonstra a multiplicidade **0..***.

Vamos implementar o nosso código?

Arquivo: Equipamento.php

```
<?php

class Equipamento
{
    public $codigoEquipamento, $descricao, $dataAquisicao;
    public $tipoEquipamento;

    public function __construct($codigoEquipamento, $descricao,
    $dataAquisicao, TipoEquipamento $tipoEquipamento)
    {
        $this->codigoEquipamento = $codigoEquipamento;
        $this->descricao = $descricao;
        $this->dataAquisicao = $dataAquisicao;
        $this->tipoEquipamento = $tipoEquipamento;

        $tipoEquipamento->equipamentos[] = $this;
    }
}
```

Notem que, na classe Equipamento, além dos atributos explícitos na modelagem, devemos incluir mais um atributo que armazenará a referência do objeto da classe TipoEquipamento, garantindo, assim, a multiplicidade e a associação entre as classes. No construtor da classe, recebemos os atributos \$codigoEquipamento, \$descricao, \$dataAquisicao mais o objeto da classe TipoEquipamento por meio do parâmetro TipoEquipamento \$tipoEquipamento, ou seja, estamos forçando o php a receber um objeto e não um valor comum.

Ainda no método construtor, após setar todos os atributos, armazenamos, no atributo `equipamentos[]` da classe `TipoEquipamento`, o próprio objeto da classe `Equipamento` pelo comando `$this`.

Arquivo: TipoEquipamento.php

```
<?php

class TipoEquipamento
{
    public $codigoTipoEquipamento, $descricao;
    public $equipamentos;

    public function __construct($codigoTipoEquipamento,
$descricao)
    {
        $this->codigoTipoEquipamento= $codigoTipoEquipamento;
        $this->descricao=$descricao;
        $this->equipamentos = array();
    }
}
```

Na classe `TipoEquipamento`, além dos atributos explícitos na modelagem, devemos incluir mais um atributo que armazenará a referência do objeto da classe `Equipamento`, garantindo, assim, a multiplicidade e a associação entre as classes. No construtor da classe, recebemos por parâmetro os atributos `$codigoTipoEquipamento`, `$descricao` e setando em suas respectivas variáveis de classe, e já inicializamos o *array* dinâmico que armazenará as referências da classe `Equipamento`.

Arquivo: controllerEquipamento.php

```
<?php

require_once('classes/Equipamento.php');
require_once('classes/TipoEquipamento.php');

$midia = new TipoEquipamento(12, "Video");

$monitor1 = new Equipamento(10, "Monitor 15 pol.", date("d-
m-Y"), $midia);
print_r($monitor1);

$monitor2 = new Equipamento(20, "Monitor 17 pol.", date("d-
m-Y"), $midia);
$monitor3 = new Equipamento(30, "Monitor 21 pol.", date("d-
m-Y"), $midia);

print_r($midia);
```

Notem que, neste arquivo, instanciamos um objetos midia para a classe TipoEquipamento e três objetos para a classe Equipamento. Em todos os objetos da classe Equipamento, passamos a referência do objeto midia.



explorando Ideias

Os diagramas de classe são usados no desenvolvimento de um modelo de sistema orientado a objetos para mostrar as classes de um sistema e as associações entre essas classes. Em poucas palavras, uma classe de objeto pode ser pensada como uma definição geral de um tipo de objeto do sistema. Uma associação é um link entre classes que indicam algum relacionamento entre essas classes. Consequentemente, cada classe pode precisar de algum conhecimento sobre sua classe associada. Quando você está desenvolvendo modelos, durante os estágios iniciais do processo de engenharia de software, os objetos representam algo no mundo real, como um paciente, uma receita médica, um médico etc. Enquanto uma aplicação é desenvolvida, geralmente é necessário definir objetos adicionais de implementação que são usados para fornecer a funcionalidade requerida do sistema.

Fonte: Sommerville (2011, p. 90).



2 AGREGAÇÃO E COMPOSIÇÃO

A agregação é o tipo de relação entre objetos conhecida como todo/parte. Neste tipo de associação, um objeto agrega outro objeto, ou seja, torna um objeto externo parte de si mesmo. Nesse caso, temos que o objeto pai poderá utilizar funcionalidades dos objetos agregados, ou seja, um objeto *todo* pode agregar um ou mais objetos *parte*.

Para exemplificarmos, confira a modelagem na Figura 3 logo a seguir.

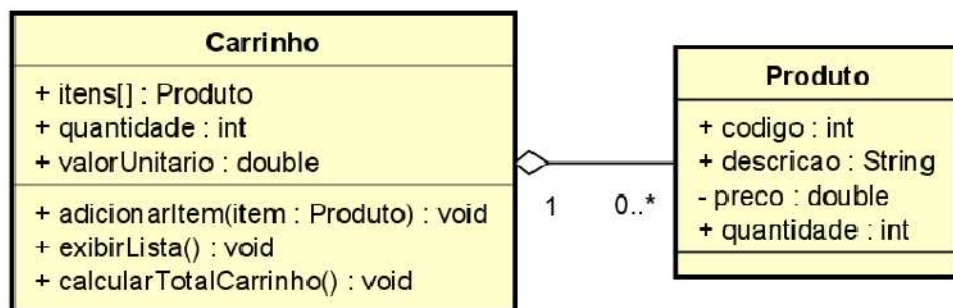


Figura 3 - Agregação / Fonte: os autores.

Na figura, temos duas classes: *Carrinho* e *Produto*, ambas ligadas por uma linha sólida, e, na extremidade da classe **todo**, há um **losango vazio**. Neste caso, a classe *Carrinho* é o **todo**, e a classe *Produto* é a **parte**. Notem que, na classe *Carrinho*, temos os atributos `itens[]` como um *array* que receberá as referências dos objetos da classe *Produto*.

Além dos atributos, temos os métodos: `adicionarItem()`, que recebe o parâmetro `item` do tipo `Produto`; o método `exibirLista()`; e, por fim, o método `calcularTotalCarrinho()`. Na classe `Produto`, temos os atributos `codigo` e `quantidade` do tipo inteiro, `descricao` do tipo *string*, e `preco` do tipo *double*.

Agora, vejamos como implementar essas classes:

Arquivo: Carrinho.php

```
<?php

class Carrinho
{
    public $itens;

    public function __construct()
    {
        $this->itens = array();
    }
    public function adicionarItem(Produto $itens)
    {
        $this->itens[] = $itens;
    }
    public function exibirLista()
    {
        return $this->itens;
    }
    public function calcularTotalCarrinho():float
    {
        $total = 0;
        for($i = 0; $i < count($this->itens); $i++){
            $total += $this->itens[$i]->preco;
        }
        return $total;
    }
}
```

Arquivo: Produto.php

```
<?php

class Produto
{
    public $codigo, $descricao, $preco, $quantidade;

    public function __construct($codigo, $descricao, $preco,
$quantidade)
    {
        $this->codigo = $codigo;
        $this->descricao = $descricao;
        $this->preco = $preco;
        $this->quantidade = $quantidade;
    }
    public function __toString()
    {
        $dados = [
            "codigo"=>$this->codigo,
            "descricao"=>$this->descricao,
            "preco"=>$this->preco,
            "quantidade"=>$this->quantidade,
        ];
        return json_encode($dados);
    }
}
```

Arquivo: ControllerProduto.php

```
<?php

require_once("classes/Carrinho.php");
require_once("classes/Produto.php");

$produto1 = new Produto(12, "Arroz", 25.78, 104);
$produto2 = new Produto(2, "Feijão", 5.78, 1400);
```



```

$produto3 = new Produto(125, "Café", 9.78, 714);

$carrinhoJunior = new Carrinho();
$carrinhoJunior->adicionarItem($produto1);
$carrinhoJunior->adicionarItem($produto2);
$carrinhoJunior->adicionarItem($produto3);

$results = $carrinhoJunior->exibirLista();
print_r($results);
echo "Total: " . $carrinhoJunior->calcularTotalCarrinho();

```

Resultado:

Array

```

(
    [0] => Produto Object
        (
            [codigo] => 12
            [descricao] => Arroz
            [preco] => 25.78
            [quantidade] => 104
        )

    [1] => Produto Object
        (
            [codigo] => 2
            [descricao] => Feijão
            [preco] => 5.78
            [quantidade] => 1400
        )

    [2] => Produto Object
        (
            [codigo] => 125
            [descricao] => Café
            [preco] => 9.78
            [quantidade] => 714
        )

```

```

    )
    )
    Total: 41.34

```

Composição

A composição também é uma relação **todo/parte**, contudo existe uma grande diferença em relação à agregação. Na composição, o objeto todo é responsável pela criação e destruição de suas partes. Enquanto, na agregação, o objeto todo possui agregações de partes (DALL’OGLIO, 2009).

Na agregação, ao destruir o objeto todo, os objetos partes ainda existem, ou seja, são independentes do objeto todo, diferentemente da composição em que, o objeto todo e o objeto parte são totalmente dependentes. Vejamos o exemplo na Figura 4 a seguir.

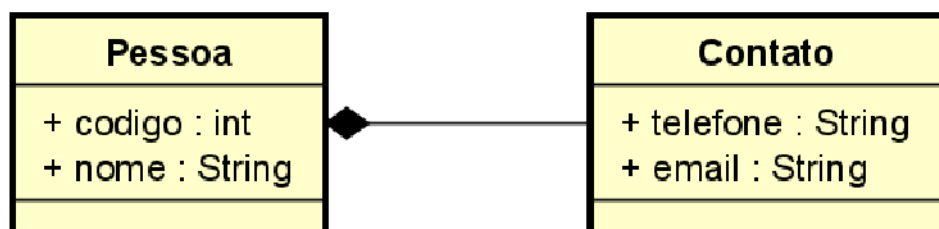


Figura 4 - Composição / Fonte: os autores.

Agora, analisaremos a modelagem anterior. Na Figura 4, temos duas classes: a primeira, chamada *Pessoa*, possui os atributos *codigo* do tipo *int* e *nome* do tipo *string*, ambos com visibilidade *public*. Enquanto a classe *Contato* possui os atributos *telefone* e *email*, ambos com o tipo *string* e visibilidade *public*. Agora, implementaremos as classes.

Arquivo: Pessoa.php

```
<?php
```

```
require_once('Contato.php');
```

```
class Pessoa
```

```
{
```

```
    public $codigo, $nome;
```

```
    public $contato;
```

```
    public function __construct()
```

```
    {
```

```
        $this->codigo=0;
```

```
        $this->nome="";
```

```
        $this->contato = new Contato();
```

```
    }
```

```
    public function setContato($codigo, $nome, $email, $telefone)
```

```
    {
```

```
        $this->codigo=$codigo;
```

```
        $this->nome=$nome;
```

```
        $this->contato->setContato($telefone, $email);
```

```
    }
```

```
    public function getContato()
```

```
    {
```

```
        return json_encode(
```

```
            [
```

```
                "Código"=>$this->codigo,
```

```
                "Nome"=>$this->nome,
```

```
                "Contato"=>$this->contato
```

```
            ]
```

```
        );
```

```
    }
```

```
}
```

Notem que declaramos os atributos explícitos da classe Contato e, para armazenar a referência da classe a ser instanciada, declaramos o atributo contato. O método construtor inicializa os atributos para garantir que não venha com lixo de sistema, e já instanciamos a classe Contato, garantindo, assim, a composição entre ambas.

O método `setContato($codigo, $nome, $email, $telefone)` recebe todos os atributos necessários para setar todos os atributos de ambas as classes. Notem que, após povoar os atributos `codigo` e `nome`, invocamos o método `setContato` da classe Contato pelo objeto `contato` da classe Pessoa e passamos a ele os atributos `telefone` e `email`. É neste método que a mágica acontece, ou seja, um objeto interno de uma classe **todo** chama a outra classe **parte** para compor sua classe **todo**.

Terminando a classe Pessoa, temos o método `getContato`, que devolve um JSON do objeto **todo**.

Arquivo: Contato.php

```
<?php

class Contato
{
    public $telefone, $email;

    public function __construct()
    {
        $this->telefone="";
        $this->email="";
    }
    public function setContato($telefone, $email)
    {
        $this->telefone=$telefone;
        $this->email=$email;
    }
    public function getContato()
    {
        return json_encode(
            [
                "Telefone"=>$this->telefone,
```

```

        "E-mail"=>$this->email
    ]
    );
}
}

```

Notem que, na classe `Contato`, a implementação é bem simples, seguimos a mesma premissa de inicialização dos atributos pelo método construtor e, na sequência, temos a implementação do método `setContato`, que recebe os dados de telefone e e-mail.

Na sequência, temos o método `getContato`, que retorna um JSON dos dados do contato. É válido saber que essa classe está atrelada ao objeto da classe `todo`.

Arquivo: `controllerComposicao.php`

```

<?php

require_once('classes/Pessoa.php');

$jose = new Pessoa();
$jose->setContato(10, "Jose de Abreu", "43 2568-7895", "jose@jsoe.com");

echo $jose->getContato();

```

No controller, basta apenas instanciar um objeto de `Pessoa`, povoar os atributos e resgatar os valores.

Resultado:

```

{
  "Código": 10,
  "Nome": "Jose de Abreu",
  "Contato": {
    "telefone": "jose@jsoe.com",
    "email": "43 2568-7895"
  }
}

```

3 INTERFACES E COMPONENTES

Abordaremos os conceitos de interfaces (contratos) no desenvolvimento e na criação de componentes web.

Interface

A interface é um recurso de POO muito utilizado pelos desenvolvedores de sistemas, principalmente, quando se necessita trabalhar com uma grande equipe de desenvolvedores e manter uma estrutura concisa e coesa. Dessa forma, temos que, na etapa de projeto, você pode definir conjuntos de métodos que determinadas classes do nosso sistema deverão implementar incondicionalmente. Uma interface possui apenas assinatura de métodos, ou seja, métodos prototipados, e não possuem atributos, por isso, compreende-se como um contrato que o desenvolvedor assina com o projetista.

Interfaces de objetos permitem a criação de códigos que especificam quais métodos uma classe deve implementar, sem definir como esses métodos serão tratados. Interfaces são definidas da mesma forma que classes, mas com a palavra-chave `interface` substituindo `class` e com nenhum dos métodos tendo seu conteúdo definido. Todos os métodos declarados em uma interface devem ser **públicos**, essa é a natureza de uma interface.

Para implementar uma interface, o operador `implements` é utilizado. Todos os métodos na interface devem ser implementados na classe; não fazê-lo resultará em um erro fatal. As classes podem implementar mais de uma interface se assim for desejado, separando cada interface com uma vírgula.

Vamos a um exemplo de interface, confira a Figura 5.

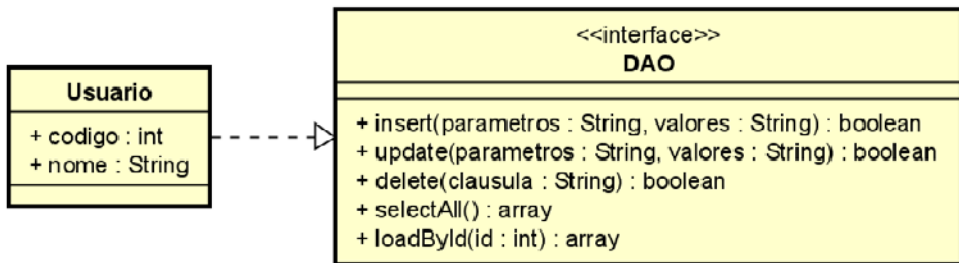


Figura 5 - Interface / Fonte: os autores.

Na figura, temos a classe `Usuario`; nesta, temos apenas dois atributos: `codigo` do tipo `int` e `nome` do tipo `String`, ambos com visibilidade `public`. Temos, também, a interface `DAO`; nesta, temos apenas métodos, sendo eles: `insert(parametros: String, valores: String): boolean`, `update(parametros: String, valores: String): boolean`, `delete(clausula: String): boolean`, `selectAll(): array`, e `loadById(id: int): array`.

Vamos ao código:

Arquivo: `InterfaceDao.php`

```

<?php
interface InterfaceDao
{
    public function insert($parametros, $valores): bool;
    public function update($parametros, $valores): bool;
    public function delete($clausula): bool;
    public function selectAll(): array;
    public function loadById($id): array;
}
  
```

No código anterior, implementamos a interface DAO e, nela, prototiparemos os métodos, ou seja, declaramos as suas assinaturas seguidas de ponto e vírgula no fim da linha.

Arquivo: Usuario.php

```
<?php
require_once('InterfaceDAO.php');

class Usuario implements InterfaceDao
{
    public function insert($parametros, $valores): bool
    {
        return true;
    }
    public function update($parametros, $valores): bool
    {
        return true;
    }
    public function delete($clausula): bool
    {
        return true;
    }
    public function selectAll(): array
    {
        return [];
    }
    public function loadById($id): array
    {
        return [];
    }
}
```


O código anterior implementa a interface DAO e, conseqüentemente, deverá implementar todos os seus métodos, mesmo que sejam sem instruções dentro de seus métodos. No nosso caso, como não implementaremos o conteúdo dos métodos, colocou-se apenas o comando *return*, pois esses métodos devem retornar um valor.

Componentes

Um componente é, basicamente, um elemento que encapsulará uma série de funcionalidades. Esse componente pode ser facilmente encaixado ao ser incorporado à programação, podemos associá-lo a uma “peça” com funções específicas, combináveis e reusáveis. Um componente é uma unidade independente, ou seja, não teria dependência de outras classes ou componentes para ser executado, sendo possível ser incorporado a qualquer momento e a outros componentes para formar um sistema mais complexo.

O uso de componentes de software possibilita encapsular complexidades técnicas e multidisciplinares, e o desenvolvedor não precisa conhecer os detalhes do funcionamento interno dos componentes para montar os sistemas. O baixo grau de acoplamento entre componentes e as interfaces bem definidas favorecem o desenvolvimento distribuído e a alocação de diferentes desenvolvedores para partes específicas do sistema, o que é desejável porque dificilmente uma única pessoa detém o conhecimento para desenvolver todo um sistema colaborativo (GEROSA; STEINMACHER, 2011).

Geralmente, um componente de software é um módulo preparado para reusar suas funcionalidades prontas e testado em múltiplos sistemas e contextos, o que favorece a redução do tempo de desenvolvimento e o aumento da qualidade do sistema. Podemos associar um componente ao uso de uma biblioteca, em vez de criar uma classe, é possível simplesmente incluir esse componente com o código já criado para fazer o que se deseja.

Na programação PHP, um componente seria um arquivo completo com todas as funcionalidades necessárias, sendo possível incorporar tal componente usando os recursos do PHP:

	Descrição
Include	Efetua a inclusão de um “arquivo”. Caso o arquivo em questão não exista, na saída, mostrar-se-ão avisos sobre o arquivo ausente, mas podendo continuar a execução do arquivo principal.
Require	Efetua a inclusão de um “arquivo”. Caso o arquivo em questão não exista, então um erro fatal aparecerá, nenhuma saída será mostrada, e a execução será interrompida.
Include_once()	Efetua a inclusão com as mesmas propriedades do Include, mas com a garantia de que o arquivo não será incluído novamente ao decorrer do arquivo. Caso aconteça, apresentará uma mensagem de erro.
Require_once	Efetua a inclusão com as mesmas propriedades do Require, mas com a garantia de que o arquivo não será incluído novamente ao decorrer do arquivo. Caso aconteça, apresentará uma mensagem de erro.

Quadro 2 - Funções de inclusão de componentes / Fonte: os autores.

As quatro possibilidades farão a incorporação do componente, então, em qualquer utilização, teremos um resultado satisfatório. Entretanto, devo chamar a atenção em sempre fazer a inclusão de componentes no início do código. Mesmo sendo possível fazer a chamada em qualquer momento, podemos ter problemas caso tenhamos várias iguais. Vejamos os códigos a seguir.

Arquivo: Carros.php

```
<?php
    public $marca = "Ford";
    public $tipo = "SUV";
    public $cor = "Preto";
```

No código anterior, declaram-se algumas variáveis com características predefinidas, as quais podemos ou não utilizar.

Arquivo: VendaCarros.php

```
<?php
include('Carros.php');
    public $tipo = "Passeio";
    public $cor = "Branco";
    echo "Venda: $marca $tipo $cor";
// será impresso Venda: Ford Passeio Branco

include('Carros.php');

    echo "Venda: $marca $tipo $cor";
// será impresso Venda: Ford SUV Preto
```

No código anterior, observe que utilizamos a função **Include** para chamar o componente. Como essa chamada aceita a inclusão do mesmo componente mais de uma vez, podemos ter problemas com subscrita, sendo aconselhável utilizar a função **Require_once**, assim, evitando problemas de múltiplas inclusões e a parada do sistema para identificação de possíveis erros.



4 CLASSES ASSOCIATIVAS

Utilizamos classes associativas quando ocorrem associações entre duas classes que possuem multiplicidade muitos (*) em ambas as extremidades. Essa classe é necessária para armazenar atributos transmitidos pela associação de ambas as classes, o que não impede que a classe associativa possua atributos próprios, ou seja, armazena os atributos transmitidos pela associação.

De acordo com Sommerville (2011), quando uma associação possui atributos próprios, pode-se criar uma classe associativa. Essas classes são úteis quando queremos armazenar o histórico de uma associação (relacionamentos que ocorrem e interessam ser salvos).

A associação é representada por uma reta tracejada partindo do meio da associação até uma classe. Veja na Figura 6.

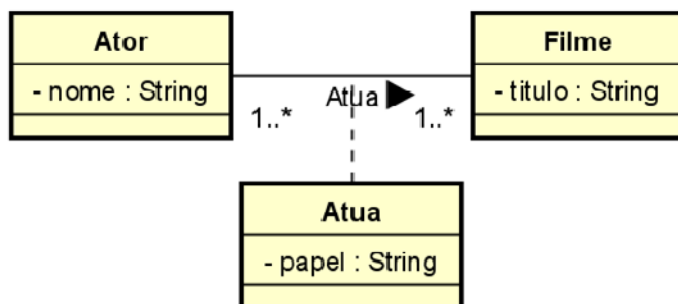


Figura 6 - Classe associativa / Fonte: os autores.

No modelo da figura, podemos observar que a multiplicidade em ambos os lados da associação é “*” (várias), o que implica dizer que, em um filme, podem haver vários atores, e um ator pode atuar em diversos filmes. Como resultado dessa associação, temos uma classe associativa que permite armazenar cada uma das atuações, juntamente com o papel, em uma classe separada.

Na classe `Ator`, temos o atributo `nome` do tipo `string` e visibilidade `private`. Na classe `Atua`, temos o atributo `papel` do tipo `string` e visibilidade `private`. E, na classe `Filme`, temos o atributo `titulo` do tipo `string` e visibilidade `private`.

As classes associativas podem ser substituídas por classes normais, chamadas Classes Intermediárias, conforme ilustrado na Figura 7.



Figura 7 - Classe Intermediária / Fonte: os autores.

Agora, vejamos a implementação dessas classes.

Arquivo: `Ator.php`

```
<?php
```

```

class Ator
{
    public $nome;
    public $atua = array();

    public function __construct($nome)
    {
        $this->nome=$nome;
        $this->atua;
    }
}
  
```

A classe `Ator` necessita ter o atributo `nome`, para armazenar os nomes dos atores, e o atributo `atua`, para armazenar as referências das atuações do ator. Neste caso, como ele deve atender à multiplicidade muitos, ele será um *array*.

Arquivo: `Atua.php`

```
<?php

class Atua
{
    public $filme;
    public $ator;
    public $papel;

    public function __construct()
    {
        $this->filme = "";
        $this->ator = "";
        $this->papel = "";
    }

    public function setAtua(Filme $filme, Ator $ator, $papel){
        $this->papel = $papel;
        $this->ator = $ator;
        $this->filme = $filme;

        $ator->atua[]=$this;
        $filme->atua[]=$this;
    }
}
```

A classe `Atua` possui um método que armazenará os atributos `filme`, `ator` e `papel`. Os atributos `filme` e `ator` recebem, respectivamente, um objeto como parâmetro, e isso pode ser confirmado com a colocação do tipo do objeto no parâmetro da função `setAtua(Filme $filme, Ator $ator,...)`. Note, ainda, que, neste método, recebemos os objetos `$filme` e `$ator`. Podemos utilizá-los na classe, desta forma, armazenaremos a referência do objeto da classe `Atua`, que está no `$this` na classe `Ator` e `Filme`.

Arquivo: Filme. php

```
<?php

class Filme
{
    public $titulo;
    public $atua;

    public function __construct($titulo)
    {
        $this->titulo = $titulo;
        $this->atua = array();
    }
}
```

A classe Filme é parecida com a classe Ator, diferenciando apenas o atributo titulo, que armazenará o nome do filme.

Arquivo: controllerAtua. php

```
<?php

require_once('classes/Ator.php');
require_once('classes/Atua.php');
require_once('classes/Filme.php');

$ator1 = new Ator("Rafael A. Florindo");
$ator2 = new Ator("Carlos Danilo Luz");

$filme1 = new Filme("Armagedon");

$atua = array();

$ator1Atua = new Atua();
$ator1Atua->setAtua($filme1, $ator1, "Cinegrafista");
$atua[] = $ator1Atua;

$ator2Atua = new Atua();
```

```

$ator2Atua->setAtua($filme1, $ator2, "Dramaturgico");
$atua[] = $ator2Atua;

print_r($ator1);
print_r($atua);

```

No arquivo de controle, instanciamos dois atores e um filme. Note que criamos um *array* com o nome *atua* e, em cada posição deste *array*, armazenamos a instância da atuação dos atores *ator1Atua* e *ator2Atua*.

```

Ator Object
(
    [nome] => Rafael A. Florindo
    [atua] => Array
        (
            [0] => Atua Object
                (
                    [filme] => Filme Object
                        (
                            [titulo] => Armagedon
                            [atua] => Array
                                (
                                    [0] => Atua Object *RECURSION*
                                    [1] => Atua Object
                                        (
                                            [filme] => Filme Object
                                                *RECURSION*
                                            [ator] => Ator Object
                                                (
                                                    [nome] =>
Carlos Danilo Luz
                                                    [atua] =>
Array
                                                (
                                                    [0]

```



```

=> Atua Object
  *RECURSION*
    )
      )
        [papel] => Dramaturgico
        )
      )
    )
  [ator] => Ator Object
  *RECURSION*
    [papel] => Cinegrafista
    )
  )
)

```

Notem que o ator Rafael participa do filme Armagedon, no papel de Cinegrafista, e, neste filme, ainda temos a participação do ator Carlos participando como Dramaturgico. Desta forma, mostramos o relacionamento de muitos para muitos.



5 APLICANDO NA PRÁTICA

Nesta prática, aplicaremos os conceitos de agregação em uma modelagem de passagens aéreas. Notem que é um exemplo didático, assim, foram omitidos alguns atributos complementares de todas as classes. Confira a figura a seguir.

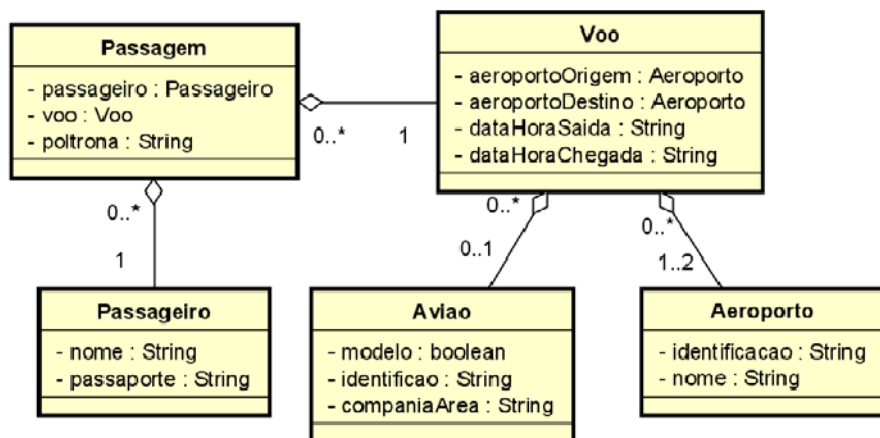


Figura 8 - Prática Agregação / Fonte: os autores.

Agora, analisaremos a imagem da modelagem anterior conforme os atributos, métodos e relacionamentos que estão explícitos. Notem que temos cinco classes: Aeroporto, Avião, Passagem, Passageiro e Voo.

Agora, vejamos os atributos que estão na modelagem:

- Na classe `Aeroporto`, temos os atributos `identificacao` e `nome`, ambos do tipo `string` e com visibilidade `private`.
- Na classe `Aviao`, temos os atributos `modelo`, `identificacao` e `companiaAerea`, ambos do tipo `string` e com visibilidade `private`.
- Na classe `Passagem`, temos os atributos `passageiro` do tipo `Passageiro`, `voo` do tipo `Voo` e `poltrona` do tipo `String`, todos com visibilidade `private`.
- Na classe `Passageiro`, temos os atributos `nome` e `passaporte`, ambos do tipo `String`, todos com visibilidade `private`.
- Na classe `Voo`, temos os atributos `aeroportoOrigem` e `aeroportoDestino` do tipo `Aeroporto`, `dataHoraSaida` e `dataHoraChegada` do tipo `String`, todos com visibilidade `private`.

Agora, vejamos os relacionamentos.

- A classe `Passagem` possui uma agregação com as classes `Passageiro` e `Voo`; em ambas, a classe `Passagem` é o **“todo”**, e as classes `Passageiro` e `Voo` são as **“partes”**, ou seja, é o agregado. Realizando a leitura dos relacionamentos, temos que, em uma instância de `Passagem`, só pode conter uma instância de `Passageiro`. Contudo, uma instância `Passageiro` pode se relacionar com muitas instâncias de `Passagem`. Por fim, uma instância de `Passagem` pode se relacionar com uma instância de `Voo`, e uma instância de `Voo` pode se relacionar com muitas instâncias de `Passagem`.
- A classe `voo` possui uma agregação com as classes `Aviao` e `Aeroporto`; em ambas, a classe `voo` é o **“todo”**, e as classes `Aviao` e `Aeroporto` são as **“partes”**, ou seja, é o agregado. Realizando a leitura dos relacionamentos, temos que, em uma instância de `Voo`, só pode haver uma instância de `Aviao`. Contudo, uma instância `Aviao` pode se relacionar com muitas instâncias de `Voo`. Por fim, uma instância de `Voo` pode se relacionar com duas instância de `Aeroporto`, e uma instância de `Aeroporto` pode se relacionar com muitas instâncias de `Voo`.

Analisada a modelagem, agora, realizaremos o desenvolvimento dos códigos. Iniciaremos pela classe `Avião`. Neste caso, crie um arquivo para a classe `Aviao`. Feito isso, necessitaremos declarar os atributos necessários, o método construtor e um método que será utilizado para a entrada de valores.

Arquivo: Aviao.php

```
<?php

class Aviao
{
    private $identificacao, $modelo, $companiaArea;
    private $voo;

    public function __construct()
    {
        $this->identificacao = "";
        $this->modelo = "";
        $this->companiaArea = "";
        $this->voo = array();
    }

    public function setAviao($identificacao, $modelo,
$companiaArea) {
        $this->identificacao = $identificacao;
        $this->modelo = $modelo;
        $this->companiaArea = $companiaArea;
    }
}
```

Notem que, no método construtor, foi necessário inicializar um *array* dinâmico `$this->voo = array();` que ficará responsável por armazenar as referências que esse avião terá com a classe `Voo`. O método `setAviao(...)` é um método comum, dessa forma, dispensa explicação.

Agora, implementaremos a classe `Aeroporto`.

Arquivo: aeroporto.php

```
<?php

class Aeroporto
{
    private $identificacao, $nome;
```

```

private $voo;

public function __construct()
{
    $this->identificacao = "";
    $this->nome = "";
    $this->voo = array();
}

public function setAeroporto($identificacao, $nome){
    $this->identificacao = $identificacao;
    $this->nome = $nome;
}

}

```

Notem que, de forma idêntica à classe anterior, declaramos o atributo `voo` e inicializamos nele um *array* dinâmico, para que armazene os voos que saem ou chegam neste aeroporto. O método `setAeroporto(...)` é um método comum, dessa forma, dispensa explicação.

Agora, implementaremos a classe `Passageiro`.

Arquivo: Passageiro.php

```
<?php
```

```

class Passageiro
{
    private $passaporte, $nome;
    private $passagens;

    public function __construct()
    {
        $this->passaporte = "";
        $this->nome = "";
        $this->passagens = array();
    }
}

```

```

        public function setPassageiro($passaporte, $nome){
            $this->passaporte = $passaporte;
            $this->nome = $nome;
        }
    }
}

```

Notem que, de forma idêntica à classe anterior, declaramos o atributo *passagens* e inicializamos nela um *array* dinâmico para que armazene as passagens do passageiro. O método `setPassageiro(...)` é um método comum, dessa forma, dispensa explicação.

Agora, implementaremos a classe *Passagem*.

Arquivo: **Passagem.php**

```

<?php

class Passagem
{
    private $voo, $passageiro, $poltrona;

    public function __construct()
    {
        $this->voo = "";
        $this->passageiro = "";
        $this->poltrona = "";
    }

    public function setPassagem(Voo $voo, Passageiro $passageiro,
    $poltrona){
        $this->voo = $voo;
        $this->passageiro = $passageiro;
        $this->poltrona = $poltrona;

        $voo->passagens[]=$this;
        $passageiro->passagens[]=$this;
    }
}

```

No método **setPassagem(Voo \$voo, Passageiro \$passageiro, \$poltrona)**, recebemos dois objetos \$voo e \$passageiro que comporão a passagem junto com o atributo \$poltrona. Nesse caso, como os objetos \$voo e \$passageiro possuem acesso aos seus atributos **\$voo->passagens[]** e **\$passageiro->passagens[]**, armazenaremos a referência do objeto passagem, que está no \$this. para voo e passageiro: **\$voo->passagens[]=\$this** e **\$passageiro->passagens[]=\$this**.

Agora, implementaremos a classe Voo.

Arquivo: Voo.php

```
<?php

class Voo
{
    private $aeroportoOrigem, $aeroportoDestino, $dataHoraSaida,
    $dataHoraChegada;

    private $passagens;

    private $aviao;

    public function __construct()
    {
        $this->aeroportoOrigem = "";
        $this->aeroportoDestino = "";
        $this->dataHoraSaida = "";
        $this->dataHoraChegada = "";
        $this->passagens="";
        $this->passagens=array();
    }

    public function setaeroportoOrigem(Aeroporto
    $aeroportoOrigem, Aeroporto $aeroportoDestino, $dataHoraSaida,
    $dataHoraChegada, Aviao $aviao){
        $this->aeroportoOrigem = $aeroportoOrigem;
        $this->aeroportoDestino = $aeroportoDestino;
        $this->dataHoraSaida = $dataHoraSaida;
        $this->dataHoraChegada = $dataHoraChegada;
```

```

        $this->aviao = $aviao;

        $aeroportoDestino->voo = $this;
        $aeroportoOrigem->voo = $this;
        $aviao->voo[] = $this;
    }
}

```

No método `setaeroportoOrigem(Aeroporto $aeroportoOrigem, Aeroporto $aeroportoDestino, $dataHoraSaida, $dataHoraChegada, Aviao $aviao)`, recebemos três objetos (`aeroportoOrigem`, `$aeroportoDestino` e `$aviao`) que farão parte da agregação. E, por sua vez, por meio dos objetos, armazenaremos as referências (`$aeroportoDestino->voo = $this; $aeroportoOrigem->voo = $this; $aviao->voo[] = $this;`). Tudo isso para garantir a multiplicidade das classes.

Depois de implementadas as classes, implementaremos o nosso *controller* para gerenciar toda a modelagem.

Arquivo: Controller.php

```

<?php

require_once('Aeroporto.php');
require_once('Aviao.php');
require_once('Passageiro.php');
require_once('Passagem.php');
require_once('Voo.php');

$aeroporto1 = new Aeroporto();
$aeroporto2 = new Aeroporto();

$aviao1 = new Aviao();

```



```

$passageiro1 = new Passageiro();
$passageiro2 = new Passageiro();
$passageiro3 = new Passageiro();

$passagem1 = new Passagem();
$passagem2 = new Passagem();
$passagem3 = new Passagem();

$voo1 = new Voo();

$aeroporto1->setAeroporto("SBMG", "Aeroporto Regional de
Maringá");
$aeroporto2->setAeroporto("GRU", "Aeroporto Internacional de
São Paulo/Guarulhos");

$aviao1->setAviao("PR-XMG", "B737-800", "GOL Linhas Aéreas
S.A.");
$passageiro1->setPassageiro("CS-123456", "José Lima");
$passageiro2->setPassageiro("CS-198476", "Fernando Reis");
$passageiro3->setPassageiro("CS-145899", "Cassandra Costa");

$voo1->setaeroportoOrigem($aeroporto1, $aeroporto2, "2020-10-
24 15:40:00", "2020-10-24 16:40:00", $aviao1);

$passagem1->setPassagem($voo1, $passageiro1, "D1");
$passagem2->setPassagem($voo1, $passageiro2, "D2");
$passagem3->setPassagem($voo1, $passageiro3, "D3");

print_r($passagem1);

```

No início do `controller`, incluímos as classes e instanciamos alguns objetos dessas classes, sendo: dois aeroportos, um avião, três passageiros, um voo e três passagens. Notem que todos eles estão interligados nas chamadas das funções que setam valores nas classes.

Para realizar as impressões, você pode utilizar laços de repetições que percorrerão os vetores, mas, como teste, o código rodará apenas um comando de impressão `print_r($passagem1)`; . Notem que é possível ver claramente os relacionamentos. O objeto `passagem` possui um `voo` com `aeroporto` de destino e chegada e, por fim, apresenta as `passagens` e seus respectivos passageiros. Lógico: se você alterar o comando de impressão `print_r($voo)`; , este resultado será alterado.

```

Passagem Object
(
    [voo:Passagem:private] => Voo Object
        (
            [aeroportoOrigem:Voo:private] => Aeroporto Object
                (
                    [identificacao:Aeroporto:private] => SBMG
                    [nome:Aeroporto:private] => Aeroporto
Regional de Maringá
                    [voo] => Voo Object
                    *RECURSION*
                )

            [aeroportoDestino:Voo:private] => Aeroporto
Object
                (
                    [identificacao:Aeroporto:private] => GRU
                    [nome:Aeroporto:private] => Aeroporto
Internacional de São Paulo/Guarulhos
                    [voo] => Voo Object
                    *RECURSION*
                )
        )

```

```

                                [dataHoraSaida:Voo:private] => 2020-10-24
15:40:00
                                [dataHoraChegada:Voo:private] => 2020-10-24
16:40:00
                                [passagens] => Array
                                    (
                                        [0] => Passagem Object
*RECURSION*
                                        [1] => Passagem Object
                                            (
                                                [voo:Passagem:private] => Voo
Object
*RECURSION*
                                                [passageiro:Passagem:private]
=> Passageiro Object
                                                    (
                                                        [passaporte:Passageiro:private]
=> CS-198476
                                                            [nome:Passageiro:private]
=> Fernando Reis
                                                                [passagens] => Array
                                                                    (
                                                                        [0] => Passagem
Object
*RECURSION*
                                                                    )
                                                                )
                                                            )
                                                        )
                                                    )
                                                )
                                            )
                                        )
                                    )
                                [poltrona:Passagem:private] => D2
                                )

```

```

[2] => Passagem Object
(
    [voo:Passagem:private] => Voo
Object
    *RECURSION*
    [passageiro:Passagem:private]
=> Passageiro Object
(
    [passaporte:Passageiro:private]
=> CS-145899
    [nome:Passageiro:private]
=> Cassandra Costa
    [passagens] => Array
    (
        [0] => Passagem
Object
    *RECURSION*
    )
    )
    [poltrona:Passagem:private] => D3
)

)

[aviao] => Aviao Object
(
    [identificacao:Aviao:private] => PR-XMG
    [modelo:Aviao:private] => B737-800
    [companiaArea:Aviao:private] => GOL
Linhas Aéreas S.A.
    [voo] => Array
    (
        [0] => Voo Object
    *RECURSION*

```

```

    )

    )

    )

    [passageiro:Passagem:private] => Passageiro Object
    (
        [passaporte:Passageiro:private] => CS-123456
        [nome:Passageiro:private] => José Lima
        [passagens] => Array
            (
                [0] => Passagem Object
*RECURSION*
            )
        )

    )

    [poltrona:Passagem:private] => D1
    )

```

Neste tópico, aplicamos alguns conceitos, mas agora está nas suas mãos deixar essa aplicação mais dinâmica. Como fazer isso?

CONSIDERAÇÕES FINAIS

Pois bem, prezado(a) aluno(a), chegamos ao fim de mais uma unidade de nosso material de estudos. Nesta unidade, pudemos acompanhar como aplicar o conceito de associação, visto que, em um sistema completo, geralmente, uma classe não trabalha de modo isolado, mas sim atrelada a outras, mesmo que seja por herança, associação, agregação, composição ou classes associativas.

Você aprendeu que a multiplicidade, além de ser uma notação importante a ser vista nos diagramas UML, é uma informação relevante ao trabalhar com a associação de classes. Seu entendimento, num primeiro momento, pode ser um pouco complexo, mas é necessário segui-lo para um bom desenvolvimento de sistema no paradigma de orientação objetos.

Trabalhamos, nesta unidade, com os conhecimentos de agregação e composição, processos que dizem respeito à interligação e à criação de classes, em que temos a agregação, que se refere ao tipo de relação entre objetos conhecida como todo/parte, sendo que, neste tipo de associação, um objeto agrega outro objeto, ou seja, torna um objeto externo parte de si mesmo. A composição segue a ideia do todo/parte, entretanto o objeto todo é responsável pela criação e destruição de suas partes; assim, caso a classe principal seja destruída, a parte também será destruída.

Outro processo visto foi o de interface e componentes, ambos associados ao conceito de reuso de software, pois são processos que se baseiam em utilizar recursos e funcionalidades de outras classes e arquivos já existentes. As interfaces são úteis ao se trabalhar com grandes equipes ou sistemas complexos em que podemos definir conjuntos de métodos que determinam as classes do nosso sistema e que deverão implementar incondicionalmente. Já a ideia de trabalhar com componentes seria integrar ao sistema módulos interiores que podem ser acoplados a qualquer momento, sendo estes independentes de outras classes, assim se aproveitando de todas as funcionalidades e métodos já existentes nesses arquivos.

Por fim, conforme já vimos nas unidades anteriores, lavamos você, prezado(a) aluno(a), a aprimorar seus conhecimentos sobre aplicações práticas dos conteúdos estudados na unidade. Espero que tenham aproveitado ao máximo o nosso material com bons estudos.

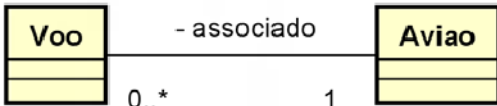
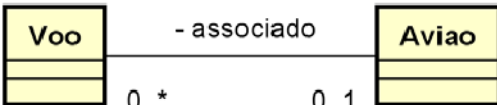
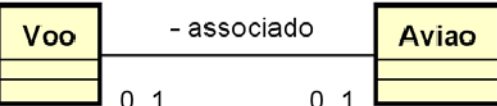
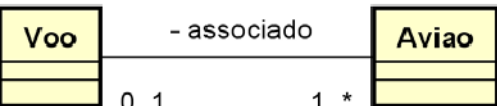
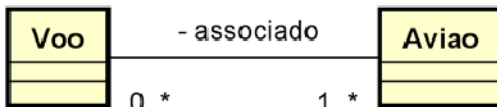


1. Segundo Rando (2018), a multiplicidade especifica quantas instâncias de uma classe se relacionam a uma instância de uma classe associada.

RANDO, D. R. **Análise e Projeto Orientado a Objetos**. Maringá: Unicesumar, 2018.

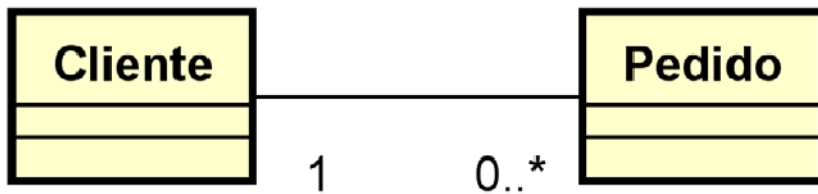
Imaginemos que você foi contratado por uma companhia de aviação para desenvolver um software orientado a objetos com a seguinte especificação: “um voo sempre está associado a um avião específico. Um voo sempre está associado a um único avião, no entanto, é possível ter um voo no qual nenhum avião ainda tenha sido designado. Um avião que atende à companhia pode não estar associado a qualquer voo, assim como pode ser utilizado em um número infinito de voos”.

Com base nessa explicação, assinale a alternativa que corresponde à associação e multiplicidade especificadas.

- a)
- 
- b)
- 
- c)
- 
- d)
- 
- e)
- 



2. Analise a modelagem a seguir:



Dentro do contexto da modelagem apresentada, qual seria a leitura da multiplicidade das classes?

- I - A leitura dessa associação nos informa que pode haver um objeto da classe Cliente que esteja associado a vários objetos da classe Pedido (isso é representado pela parte * do símbolo 0..*).
 - II - A leitura nos informa que pode haver um objeto da classe Cliente que não esteja associado a pedido algum (isso é representado pela parte 0 do símbolo 0..*).
 - III - O diagrama também representa a informação de que um objeto da classe Pedido está associado a um, e somente um, objeto da classe Cliente.
 - IV - O Diagrama apresenta que podem ter múltiplos clientes para múltiplos pedidos.
- É correto o que se afirma em:
- a) I, apenas.
 - b) II, apenas.
 - c) I, II e III, apenas.
 - d) II e IV, apenas.
 - e) I, II, III e IV.
3. Uma associação é um relacionamento entre dois objetos, como classes ou casos de uso, que têm por objetivo representar os motivos e as regras que conduzem os objetos ao relacionamento. A agregação é um caso especial de associação, utilizada para representar relacionamentos de pertinência do tipo “parte-todo” ou “uma parte de”. Assinale a alternativa que define corretamente uma agregação hierárquica.



- a) Uma forma forte de um relacionamento todo/parte.
 - b) Uma forma fraca de um relacionamento todo/parte.
 - c) Uma forma forte de uma hierarquia generalização/especialização.
 - d) Uma forma fraca de uma hierarquia generalização/especialização.
 - e) Uma forma fraca de um relacionamento de generalização/especialização.
4. A utilização de interfaces deve garantir ao projeto um padrão de implementação e deveria ser algo tão comum como qualquer outro recurso, como abstração em bons projetos. Analise o código a seguir e informe o retorno que será obtido após a sua execução.

```
<?php
```

```
interface PrimeiraInterface {  
    public function teste();  
}
```

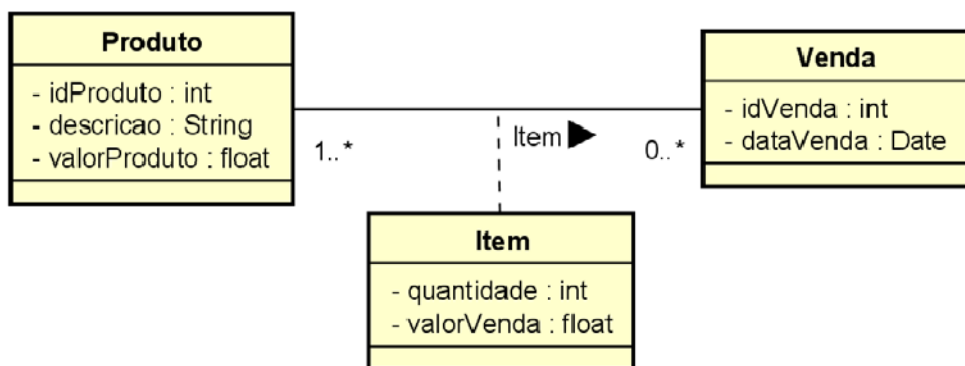
```
class Teste implements PrimeiraInterface{  
    public function teste($n1) {  
        return $n1 * 0.5;  
    }  
}
```

```
$teste = new Teste();  
echo $teste->teste(20);
```

- a) 0.5.
- b) 10.
- c) 20.
- d) 5.
- e) Erro de PHP.



5. Você, como um(a) excelente desenvolvedor(a), foi incumbido(a) de criar um modelo conceitual utilizando a notação do diagrama de classes da UML para a seguinte situação: “representantes trabalham em empresas. Cada empresa usa o seu depósito particular para armazenar um conjunto de produtos. O sistema deseja saber quais representantes estão associados a determinado produto”. Ilustre o diagrama que atenda à descrição anterior com a classe associativa.
6. As classes associativas são úteis quando queremos armazenar o histórico de uma associação. Observe o diagrama de classes a seguir e explique a multiplicidade envolvida nessa associação.





WEB COMPONENTS NA PRÁTICA

De acordo com a especificação da W3C, os Web Components consistem em “um conjunto de cinco tecnologias: Templates, Shadow DOM, Custom Elements, HTML Imports e Decorators”. Sendo que essa última, diferente das demais, ainda não possui uma especificação e tem sido bastante omitida pela comunidade, por esses motivos os exemplos disponíveis nesse artigo não farão menção aos decoradores.

Para facilitar o entendimento do conjunto de tecnologias podemos defini-las da seguinte forma:

Templates: Templates representam uma sub árvore no DOM que permanece inerte e é enxertada na árvore principal. O trecho de código contido no template serve para a criação de componentes dinâmicos, possibilitando que o desenvolvedor instancie e manipule tal código sob demanda. Devido a inércia dos templates o navegador não realiza a requisição das imagens e não executa os scripts contidos nos mesmos até que sejam enviados para a página via JavaScript.

Shadow Dom: O Shadow DOM realiza o isolamento de elementos do DOM em trechos independentes evitando que estilos CSS e comportamentos JavaScript dos elementos alterem as características uns dos outros.

Custom Elements: O Custom Element permite que o desenvolvedor crie tags diferentes das convencionais. O intuito é encapsular as demais tags numa personalizada, reutilizável, manutenível e manuseável. Essa tecnologia auxilia bastante na diminuição dos enormes aninhamentos de divs, conhecidos como *div hell*. Custom Elements também possibilitam que os elementos personalizados herdem características e propriedades de tags nativas específicas como a tag `<button>`, por exemplo.



HTML Import: Com o HTML Import podemos importar documentos HTML inteiros para dentro de nossas páginas. De uma maneira semelhante à importação de um arquivo JavaScript ou CSS, o HTML Import cria uma nova requisição ao servidor para realizar o download do arquivo. Essa tecnologia auxilia no encapsulamento dos componentes em arquivos externos possibilitando importá-los posteriormente. Antes do HTML Imports, as poucas alternativas de importarmos documentos HTML eram através de elementos iframes e requisições AJAX, não muito bem vistas do ponto de vista do código limpo.

Decorators: Os decoradores aplicam os templates com base em seletores CSS e JavaScript para criar mudanças visuais e comportamentais. O elemento “content” inserido dentro do template será substituído com o conteúdo do elemento de decoração. E através do atributo “select” é possível especificar a posição exata do elemento particular na marcação que será produzida. Com o CSS em conjunto com as Media Queries seria possível criar uma poderosa ferramenta de Responsive Web Design utilizando decoradores.

Fonte: DevMedia ([2021], on-line).



eu recomendo!



livro

Aprendendo PHP: introdução amigável à linguagem mais popular da web

Autor: David Sklar

Editora: Novatec

Sinopse: se você deseja conhecer PHP, este livro é essencial. O autor David Sklar (PHP Cookbook) o guiará por aspectos da linguagem necessários para a construção de sites dinâmicos no lado do servidor. Examinando recursos do PHP 5.x e as excelentes melhorias da versão mais recente, PHP 7, você aprenderá como trabalhar com servidores web, navegadores, bancos de dados e web services. Exercícios no fim dos capítulos o ajudarão a fixar as lições. Seja você um curioso tentando construir sites dinâmicos, um desenvolvedor front-end pronto para adicionar programas no lado do servidor ou um programador experiente querendo se atualizar nos meandros da linguagem, essa introdução amigável também aborda aspectos do moderno PHP 7, como a internacionalização, o uso de PHP a partir da linha de comando e o gerenciamento de pacotes.



livro

Aprendendo Padrões de Projeto em PHP: Programação Orientada a Objetos

Autor: William Sanders

Editora: O'Reilly

Sinopse: desenvolva aplicações executadas em servidores de modo mais eficiente — e melhore suas habilidades em programação PHP no decorrer do processo — aprendendo a usar padrões de projeto em seu código. Este livro mostra como aplicar vários padrões Orientados a Objetos por meio de exemplos simples e demonstra vários desses padrões em aplicações funcionais completas. Saiba como esses padrões reutilizáveis podem ajudar você a solucionar problemas complexos, organizar código Orientado a Objetos e fazer revisões em projetos de grande porte, alterando somente partes pequenas do projeto. Aprendendo padrões de projeto em PHP, você aprenderá a adotar um estilo de programação mais sofisticado e a reduzir dramaticamente o tempo de desenvolvimento. Aprenda os conceitos de padrões de projeto, incluindo como selecionar padrões para lidar com problemas específicos. Adquira uma visão geral sobre conceitos de programação Orientada a Objetos, tais como composição, encapsulamento, polimorfismo e herança.





MANIPULAÇÃO DE DADOS ENTRE PHP COM AS APIS (mysqli, mysqli prepared e pdo)

PROFESSORES

Me. Rafael Alves Florindo

Esp. Carlos Danilo Luz

PLANO DE ESTUDO ▼

A seguir, apresentam-se as aulas que você estudará nesta unidade: • Conceitos Básicos de Banco de Dados Relacionais • Diferenciando as APIs • Conectando PHP às APIs e Executando Comandos SQL em um Programa PHP (INSERT e SELECT) • Executando Comandos SQL em um Programa PHP (UPDATE, DELETE) • Aplicação Prática dos Comandos SQL.

OBJETIVOS DE APRENDIZAGEM ▼

Resgatar os comandos de SQL que serão utilizadas no decorrer do livro • Conceituar e apresentar as APIs de conexão com o banco de dados em PHP • Apresentar as conexões do PHP com as API e aplicar os comando de inserção e seleção simples de registros utilizando a API PDO Object • Aplicar os comandos de alteração e deleção de registros utilizando a API PDO Object • Levar o aluno à prática de uma classe envolvendo os conceitos aprendidos nesta unidade.



Olá! É um imenso prazer tê-lo novamente conosco em busca de conhecimento. Conforme vimos em nossas unidades anteriores, o paradigma de Orientação a Objetos aplicada a PHP se demonstra altamente adaptável, além de proporcionar uma fácil implementação. Até o momento, observamos como implementar os aspectos de um sistema baseado em PHP OB, mas compreendemos que qualquer sistema, do mais simples ao mais avançado, utiliza-se de um banco de dados e componentes externos para o seu bom desenvolvimento.

Devemos lembrar que um sistema faz parte de um ecossistema mais elaborado e complexo, por isso, ter o conhecimento dos subsistemas, componentes e armazenamento dos dados de um sistema que será desenvolvido e implementado torna-se relevante no processo de desenvolvimento de qualquer tipo de aplicação. Por isso, nesta unidade, relembremos os princípios de banco de dados e do Sistema Gerenciador de Banco de Dados (SGBD) relacional, como, também, a execução das consultas em Standard Query Language (SQL), as quais utilizam a integração de um sistema com os dados armazenados. Tais conceitos serão apresentados com as APIs de conexão com o banco de dados em PHP, dando condições para futuros desenvolvimentos com essas integrações.

Como o nosso livro é baseado no paradigma de Orientação a Objetos, trabalharemos com a API PDO Object. Tal API dá suporte na integração de banco de dados e comandos SQL em um programa PHP.

Atualmente, existem diversos SGBD no mercado. Para os exemplos desta unidade, basear-nos-emos no MySQL, um dos principais SGBDs utilizados em aplicações web e PHP. O que devemos lembrar é que estudaremos os comandos SQL e que eles são 95% compatíveis com os demais SGBDs relacionais disponíveis para armazenamento de dados.

Espera-se que, ao final desta unidade, você, prezado(a) aluno(a), seja capaz de desenvolver aplicações web com maior eficiência. Para isso, aplicaremos, de modo prático, o uso dos recursos e das classes envolvendo os conceitos aprendidos nesta unidade, proporcionando a você uma experiência inovadora e utilizável no mercado de trabalho. Estamos esperando o quê? Vamos lá!

1 CONCEITOS BÁSICOS DE BANCO DE DADOS relacionais

Olá, aluno(a)! Nesta aula, resgataremos alguns comandos que são utilizados no SQL e que serão utilizados no decorrer do nosso material. Vale lembrar que os comandos apresentados podem ser testados em qualquer software que execute a linguagem SQL. O objetivo é apenas resgatar os conceitos e apresentar a sua sintaxe.

O nome SQL significa *Structured Query Language* — Linguagem Estruturada de Pesquisa. A SQL se tornou um padrão no mundo dos ambientes de banco de dados relacionados. A linguagem SQL é um recurso muito utilizado por DBAs e desenvolvedores para a execução de comandos em bancos de dados relacionais, e é por meio dela que criamos tabelas, colunas, índices, atribuímos permissões a usuários, bem como realizamos consultas a dados, realizamos manutenção de transações e backups.

A linguagem SQL é dividida, então, em quatro grandes grupos de acordo com o tipo de operação a ser executada no banco de dados. A saber, temos:

- DML (*Data Manipulation Language*, ou Linguagem de Manipulação de Dados em português).
- DDL (*Data Definition Language*, ou Linguagem de Definição de Dados em português).
- DCL (*Data Control Language*, ou Linguagem de Controle de Dados em português).
- DTL (*Data Transaction Language*, ou Linguagem de Transação de Dados em português).

- DQL (*Data Query Language*, ou Linguagem de Consulta de Dados em português), que tem apenas um comando (SELECT).

Vejam, a seguir, alguns tipos de dados que serão utilizados para a criação de nossas tabelas.

Tipo	Descrição	Tamanho bytes
Char (n)	Tamanho fixo, completado com espaços em branco. 8,000 caracteres.	Tamanho definido
varchar(n)	Tamanho variável com limite. 1,073,741,824 caracteres.	2 bytes + número de caracteres
text	Tamanho variável. 2GB de dados (texto).	4 bytes + número de caracteres
Tinyint	Permite números inteiros de 0 a 255.	1 byte
Smallint	Permite números inteiros entre -32,768 e 32,767.	2 bytes
int	Permite números inteiros entre -2,147,483,648 e 2,147,483,647.	4 bytes
bigint	Permite números inteiros entre -9,223,372,036,854,775,808 e 9,223,372,036,854,775,807.	8 bytes
decimal(p,s)	O parâmetro p indica o número total máximo de dígitos que podem ser armazenados (ambos à esquerda e à direita do ponto decimal). O p deve ser um valor de 1 a 38. O padrão é 18. O parâmetro s indica o número máximo de dígitos armazenados à direita do ponto decimal. s deve ser um valor de 0 a p. O valor padrão é 0.	5-17 bytes
datetime	De 1 de janeiro de 1753 a 31 de dezembro de 9999, com uma precisão de 3,33 milissegundos.	8 bit
Date	Armazena apenas uma data. De 1 de janeiro de 0001 a 31 de dezembro de 9999.	3 bytes
Timestamp	Armazena um número único que é atualizado sempre que uma linha é criada ou modificada.	8-10 bytes

Quadro 1 - Tipos de dados a serem armazenados nos atributos / Fonte: os autores.



conecte-se

Prezado(a) aluno(a), saiba que existem outros tipos de dados de armazenamento além dos apresentados. Vejamos no link a seguir:



Agora, veremos as categorias separadamente.

O DDL ou *Data Definition Language* (Linguagem de Definição de Dados) permite ao desenvolvedor ou DBA (*Database Administrator*, em tradução: Administrador de Banco de Dados) definir as novas tabelas e os elementos que serão associados a elas. É responsável pelos comandos de criação e alteração no banco de dados, sendo composto por três comandos: CREATE, ALTER e DROP.

O comando CREATE DATABASE é responsável pela criação de um novo banco de dados vazio, conforme podemos ver a seguir:

```
CREATE DATABASE unicesumarEad;
```

Ao ser executado, criaremos o banco de dados chamado “unicesumarEad”.

Já o comando **CREATE TABLE** criará uma nova tabela. Os bancos de dados relacionais guardam seus dados dentro de tabelas que são divididas em colunas. Cuidados ao escolher os nomes das tabelas, nomes de colunas e nomes de outros objetos do banco de dados:

- O primeiro caractere deve ser alfabético (A–Z ou a–z).
- Após o primeiro caractere, pode-se utilizar números.
- Não são permitidos caracteres especiais.
- Apenas os seguintes caracteres não alfanuméricos são permitidos: #, \$ e _.

Desta forma, veremos, a seguir, a criação de uma tabela de usuário. Ao criar, especificaremos as suas colunas e quais tipos de dados elas receberão (neste caso, um id, nome, email, dataNascimento, telefone e dataCadastro).

```
CREATE TABLE usuario (
  id INT NOT NULL AUTO_INCREMENT primary key,
  nome VARCHAR (255),
  email VARCHAR (255),
  dataNascimento DATE,
  telefone VARCHAR(20),
  dataCadastro DATETIME
);
```

Note que cada campo contém seus respectivos tipo e tamanho, que é colocado entre parênteses, não sendo permitido juntar campos do mesmo tipo em uma única linha. Utilizamos vírgulas para separar as colunas da tabela. Não utilizamos vírgula após a última coluna, salvo se for necessário definir *constraints* (restrições), conforme veremos a seguir.

Há situações em que precisamos garantir que valores NULOS não sejam inseridos em determinadas colunas do banco. Para esse objetivo, devemos utilizar a restrição NOT NULL. Queremos garantir, por exemplo, que não sejam inseridos valores NULOS nas colunas id de nossa tabela usuário. Observe, a seguir, como podemos garantir isso ao utilizar a restrição NOT NULL:

```
CREATE TABLE usuario (
  id INT NOT NULL AUTO_INCREMENT primary key,
  nome VARCHAR (255),
  email VARCHAR (255),
  dataNascimento DATE,
  telefone VARCHAR(20),
  dataCadastro DATETIME
);
```

Notem, ainda, que temos a propriedade AUTO_INCREMENT no campo “Id”, que gerará automaticamente um número único quando um novo registro é inserido em uma tabela.

Por fim, temos a *constraints* do tipo PK - PRIMARY KEY (CHAVE PRIMÁRIA) para identificar com exclusividade cada linha da tabela.

Alter Table

O comando ALTER, por sua vez, é o comando utilizado para alterar uma tabela ou um banco de dados já existente. No exemplo a seguir, adicionaremos (ADD) uma nova coluna à nossa tabela de usuário criada anteriormente. Esta nova coluna, “login”, será criada após a nossa coluna “nome”, desta forma, usaremos:

```
ALTER TABLE usuario ADD login char(15) AFTER nome;
```

Outra situação com a qual um profissional da área de banco de dados pode se deparar é a necessidade de alterar (MODIFY) o tamanho de uma coluna. Por exemplo, alteraremos o campo de login, que está com o tamanho 15, e o tipo char para o tipo varchar com tamanho (20).

```
ALTER TABLE usuario MODIFY login varchar(20);
```

Há situações não muito frequentes em que é necessário alterar (RENAME) o nome de uma coluna. Alteraremos o campo login para user.

```
ALTER TABLE usuario MODIFY user varchar(20);
```

É possível que, em algumas outras situações, cheguemos à conclusão de que não faz mais sentido determinados dados serem mantidos em um banco. Desta forma, queremos excluir (DROP COLUMN) alguns campos. Excluiremos o campo de dataNascimento.

```
ALTER TABLE CLIENTE DROP COLUMN dataNascimento;
```

O comando DROP é utilizado para a remoção de uma tabela ou do banco de dados por completo. Desta forma, para remover um banco de dados por completo, basta inserirmos o seguinte comando:

```
DROP DATABASE unicesumarEad;
```

Ou excluir uma tabela ao utilizar o seguinte comando:

```
DROP TABLE usuario;;
```



quadro-resumo

O comando DROP é extremamente perigoso, pois o mesmo exclui totalmente os dados. No caso de alguns SGBDs, não é possível recuperar os dados, mas, em alguns, é possível aplicar o comando ROLLBACK, desfazendo as transações. Mesmo sabendo desta possibilidade, indicamos usar esse comando em último caso e em situações muito particulares.

DML – *Data Manipulation Language*

Os comandos do subgrupo denominado DML (*Data Manipulation Language* ou Linguagem de Manipulação de Dados) têm como objetivo recuperar dados (realizar consultas), inserir novas linhas, alterar linhas existentes e remover linhas do banco de dados. Os comandos que fazem parte desse subgrupo são: SELECT, INSERT, UPDATE e DELETE.

INSERT

O comando DML utilizado para inserir ou incluir novos dados em uma tabela ou visão é o INSERT. Para incluir uma nova linha em uma tabela, além da palavra reservada INSERT, utilizamos a preposição (em inglês) INTO, o nome da tabela, os nomes das colunas (que deverão ser declaradas entre parênteses e, dependendo da situação, podem ser opcionais), a palavra VALUES e, finalizando, os valores que deverão ser incluídos (também entre parênteses).

Voltando à tabela `usuario`:

Usuario						
Id	nome	login	email	dataNascimento	Telefone	dataCadastro

Digamos que os seguintes dados deverão ser inclusos na tabela usuário:

Usuario						
id	nome	login	email	dataNascimento	Telefone	dataCadastro
12	Rafael	Rafa	rafa@unicesumar.edu	1982-12-02	3024-7845	2019-08-09

O comando a ser utilizado para incluir esses dados na tabela `CLIENTE` deverá ser o seguinte:

```
INSERT INTO usuario (id, nome, login, email, dataNascimento,
telefone, dataCadastro) VALUES (12, 'Rafael', 'rafa', 'rafael@
uniceumar.edu', 1982-12-02, '3024-7845', 2019-08-09);
```

Conforme você pode observar, o valor 12 aparece sem aspas simples, isso ocorre porque se refere a um valor numérico. Por outro lado, os campos `nome`, `login`, `email` e `telefone` aparecem entre aspas simples, porque não são valores numéricos. Note, ainda, que os campos de data estão no formato americano (ano-mês-dia).

Quando informamos os dados referentes a todas as colunas da tabela e na exata sequência como foram declarados no momento em que a tabela foi criada, podemos omitir os nomes das colunas ao inserir uma nova linha na tabela. Observe o próximo exemplo:

```
INSERT INTO usuario VALUES (18, 'Arthur', 'arturo', 'arthur@
unicesumar.edu', 2007-12-02, '3487-4745', 2019-08-09);
```

Usuario						
id	nome	login	email	dataNascimento	Telefone	dataCadastro
12	Rafael	Rafa	rafa@unicesumar.edu	1982-12-02	3024-7845	2019-08-09
18	Arthur	arturo	arthur@unicesumar.edu	2007-12-02	3487-4745	2019-08-09

Caso queira inserir mais de um registro de uma vez, após os parênteses do `VALUES`, coloque um vírgula e insira novos registros dentro de parênteses. Vejamos um novo exemplo:

```
INSERT INTO usuario (id, nome, login, email, dataNascimento,
telefone, dataCadastro) VALUES (14, 'Carlos', 'carlos',
'carlos@unicesumar.edu', 1982-12-02, '3024-7845', 2019-08-09),
(21, 'Ricardo', 'rick', 'ricardo@unicesumar.edu', 1982-12-02,
'3024-7845', 2019-08-09);
```

Usuario						
id	nome	login	email	dataNascimento	Telefone	dataCadastro
12	Rafael	Rafa	rafa@unicesumar.edu	1982-12-02	3024-7845	2019-08-09
14	Carlos	carlos	carlo@unicesumar.edu	1982-12-02	3024-7845	2019-08-09
18	Arthur	arturo	arthur@unicesumar.edu	2007-12-02	3487-4745	2019-08-09
21	Ricardo	rick	ricardo@unicesumar.edu	1982-12-02	3487-4745	2019-08-09

DELETE

A exclusão de linhas de uma tabela é realizada por meio do comando **DELETE**. Podemos excluir uma ou mais linhas de uma tabela ao utilizar esse comando.

Após a palavra reservada **DELETE**, utilizamos a preposição (em inglês) **FROM**, que deverá ser seguida pelo nome da tabela. Quando desejamos excluir uma ou mais linhas, após a palavra **WHERE** (onde), devemos informar a condição para que a exclusão ocorra. Se a condição for verdadeira, a linha será excluída da tabela. O exemplo a seguir apresenta a exclusão de uma linha (do usuário, cujo id é igual a 12) da tabela **usuario**.

```
DELETE FROM usuario WHERE id = 12;
```

Usuario						
id	nome	login	email	dataNascimento	Telefone	dataCadastro
14	Carlos	carlos	carlo@unicesumar.edu	1982-12-02	3024-7845	2019-08-09
18	Arthur	arturo	arthur@unicesumar.edu	2007-12-02	3487-4745	2019-08-09
21	Ricardo	rick	ricardo@unicesumar.edu	1982-12-02	3487-4745	2019-08-09

Para excluir todas as linhas de uma tabela, é preciso apenas utilizar o comando **DELETE**, seguido da preposição **FROM** e do nome da tabela. O exemplo apresentado a seguir exclui, por meio de um único comando, todas as linhas da tabela **usuario**. No entanto, tome cuidado: ao executar esse comando de forma errada, apagará todo o conteúdo da tabela.

```
DELETE FROM usuario;
```

Usuario						
id	nome	login	email	dataNascimento	Telefone	dataCadastro

UPDATE

Um banco de dados apresenta uma coleção de dados relacionados que, constantemente, passa por atualizações. Portanto, quando precisamos atualizar os dados de uma tabela, utilizamos o comando `UPDATE`.

Quando utilizamos o comando `UPDATE`, devemos informar, na sequência, o nome da tabela que será seguido pela palavra `SET` e do nome da coluna em que ocorrerá a alteração ou atualização dos dados. A seguir, será preciso informar qual será o novo valor e, muito importante, em qual linha a atualização deverá ser realizada. Utilizamos como referência, para identificar a linha a ser atualizada, a chave primária, pois, assim, sempre teremos certeza de atualizar a linha correta. Vamos alterar o telefone do registro do `usuario` 18.

```
UPDATE usuario SET telefone = '3028-9711' WHERE id = 18;
```

Usuario						
id	nome	login	email	dataNascimento	Telefone	dataCadastro
12	Rafael	Rafa	rafa@unicesumar.edu	1982-12-02	3024-7845	2019-08-09
14	Carlos	carlos	carlo@unicesumar.edu	1982-12-02	3024-7845	2019-08-09
18	Arthur	arturo	arthur@unicesumar.edu	2007-12-02	3028-9711	2019-08-09
21	Ricardo	rick	ricardo@unicesumar.edu	1982-12-02	3487-4745	2019-08-09

De forma análoga ao comando `DELETE`, o `UPDATE` permite alterar todos os registros de um campo específico. Para isso, deve-se omitir a cláusula `WHERE`. Veja, a seguir, como ficou:

```
UPDATE usuario SET telefone = '3028-9711';
```


Usuario						
id	nome	login	email	dataNascimento	Telefone	dataCadastro
12	Rafael	Rafa	rafa@unicesumar.edu	1982-12-02	3028-9711	2019-08-09
14	Carlos	carlos	carlo@unicesumar.edu	1982-12-02	3028-9711	2019-08-09
18	Arthur	arturo	arthur@unicesumar.edu	2007-12-02	3028-9711	2019-08-09
21	Ricardo	rick	ricardo@unicesumar.edu	1982-12-02	3028-9711	2019-08-09

SELECT

O comando `SELECT` é utilizado para a realização de consultas em tabelas ou visões. Por meio das consultas, podemos obter todos os dados de uma tabela ou apenas aqueles que desejamos, utilizando, para isso, filtros. Veja, a seguir, como realizar uma consulta para obter todos os registros da tabela `usuario`.

```
SELECT * FROM usuario;
```

Usuario						
id	nome	login	email	dataNascimento	Telefone	dataCadastro
12	Rafael	Rafa	rafa@unicesumar.edu	1982-12-02	3028-9711	2019-08-09
14	Carlos	carlos	carlo@unicesumar.edu	1982-12-02	3028-9711	2019-08-09
18	Arthur	arturo	arthur@unicesumar.edu	2007-12-02	3028-9711	2019-08-09
21	Ricardo	rick	ricardo@unicesumar.edu	1982-12-02	3028-9711	2019-08-09

Agora, para obter os nomes dos usuários, utilizando, para isso, a tabela `usuario`.

```
SELECT nome FROM usuario;
```

Usuario
nome
Rafael
Carlos
Arthur
Ricardo

Agora, se desejarmos buscar algum valor em específico, utilizaremos a cláusula `WHERE`. A cláusula `WHERE` é a responsável por filtrar resultados, utilizando parâmetros comparativos como igual, diferente, maior, menor e entre outros. Além disso, podemos usar diversos filtros em nossas consultas, inclusive, adicionar parênteses para priorizar consultas dentro de escopos. A cláusula `WHERE` funciona da seguinte forma:

```
SELECT coluna, coluna FROM nome_da_tabela WHERE condicao_
da_consulta;

SELECT * FROM usuario WHERE id = 12;
```

Usuario						
id	nome	login	email	dataNascimento	Telefone	dataCadastro
12	Rafael	Rafa	rafa@unicesumar.edu	1982-12-02	3028-9711	2019-08-09

Nesta aula, estudamos alguns comandos em SQL que serão utilizados nos próximos tópicos. Para igualar os comandos e ilustrações, utilizaremos o PHPMyadmin, que é um software web, *free* e multiplataforma. Para tê-lo, basta instalar o software xampp, que, por sua vez, já possui o servidor apache, o PHP, o banco de dados mysql e o phpmyadmin. Creio que você já instalou este ou outro servidor, assim, já o tem instalado.



conecte-se

Utilizando o prompt de comando e o PHPMyAdmin na manipulação de dados no Banco de Dados (Parte 1): <https://www.youtube.com/watch?v=OaPMvrA0cA4>

Utilizando o prompt de comando e o PHPMyAdmin na manipulação de dados no Banco de Dados (Parte 2): https://www.youtube.com/watch?v=EC_1ZtXsUtA

2 DIFERENCIANDO AS APIS

Atualmente, o PHP se conecta com diversos gerenciadores de bancos de dados disponíveis no mercado, sejam pagos ou *free*, tais como: Oracle, SQLServer, PostgreSQL, Mysql, Interbase, Db2, o MariaDB, entre outros. Para cada tipo de gerenciador de banco de dados, o PHP dispõe de funções específicas para conexão, bem como para realizar as consultas ao banco.

Neste livro, abordaremos o gerenciador Mysql na sua versão gratuita, que, atualmente, é ofertada na versão do MariaDB. Sua aplicação não será afetada caso sua hospedagem seja o Mysql ou MariaDb, pois ambos compartilham da mesma sintaxe, ou seja, a única diferença entre as duas versões está no seu custo e na capacidade de armazenamento.



conecte-se

O DB-Engines Ranking classifica os sistemas de gerenciamento de banco de dados de acordo com a sua popularidade. O ranking é atualizado mensalmente:



A versão do servidor web que utilizamos no livro é o (Apache na versão 2.4.41, MariaDB na versão 5.5.5-10.4.11 e, o PHP na versão 7.4.4), provido pelo pacote XAMMP, que está na versão 3.2.4.

**conecte-se**

O XAMPP é completamente gratuito e de fácil instalação do servidor Web Apache, SGBD, MySQL, PHP e Perl. O pacote de código aberto do XAMPP foi criado para ser extremamente fácil de instalar e de usar.



Como dito anteriormente, o PHP dispõe de diversas funções para trabalhar com banco de dados. Contudo, para que isso seja possível, é necessário que o seu servidor web tenha habilitado as configurações das extensões necessárias para que o seu gerenciador possa se conectar com o servidor PHP. Caso o seu não esteja habilitado, você deverá abrir o arquivo de configuração (`php.ini`). No meu servidor, este arquivo se encontra no diretório (`C:\xampp\php`). Ao abri-lo, procure pelas linhas a seguir, e, caso elas tenham um (`;`) no início, retire-os, salve o arquivo e reinicie o seu servidor web, caso o tenha inicializado.

```
extension=mysqli  
extension=pdo_mysql
```

A primeira extensão `extension=mysqli` habilita a conexão do PHP com o gerenciador do banco de dados Mysql, e a segunda extensão `extension=pdo_mysql` habilita o driver de conexão com o PDO (*PHP Data Objects*). Ambas serão explicadas no decorrer desta aula e das demais. Sugiro olhar o arquivo todo para que tenha uma maior familiaridade com o poder do seu servidor, mas cuidado: uma alteração errada pode inativar o seu servidor.

Agora, se estiver com um servidor web que seja diferente do apresentado, você deverá procurar por este arquivo dentro da pasta de instalação do seu servidor. A localização pode ser diferente, mas o conteúdo é similar, assim, procure as extensão e, caso necessário, habilite-as.

Se você for trabalhar com outra extensão de banco de dados, basta habilitá-la. Se a mesma não estiver listada, dê uma conferida no manual do php e veja se o seu gerenciador está listado.

Extensão MySQLi

O Mysqli é uma extensão do PHP que substitui a extensão antiga API do

Mysql, que poderia ser utilizado até a versão 5.6 do PHP. Agora, a partir da versão 6 do PHP, essa é a extensão disponível para a nova versão da API do MySQL, e esta, por sua vez, aproveita os recursos avançados do Mysql. MySQLi inclui recursos aprimorados que o MySQL não realiza. Por exemplo, MySQLi pode suportar declarações preparadas para o cliente e do lado do servidor, incluindo várias instruções.

O MySQLi pode ser utilizado de duas formas: orientada ou estrutural. Neste livro, abordaremos a forma orientada por ser a mais recomendada e por fazer parte do paradigma abordado no livro.



conecte-se

A extensão MySQLi nada mais é que o PHP MySQL melhorado! Basicamente, é possível fazer as mesmas funções de acesso a servidores de banco de dados MySQL. Vejamos, a seguir, alguns exemplos de como utilizar a extensão MySQLi com PHP:



Para utilizar o banco de dados com o PHP, devemos realizar a conexão, ou seja, este processo realiza a ligação dos servidores para aquela aplicação. Para que essa conexão seja realizada, devemos, primeiramente, instanciar a classe nativa Mysqli do PHP e passar à classe alguns parâmetros de conexão. A classe Mysqli espera receber, pelo método construtor, os parâmetros para a conexão, que são (servidor, usuário, senha e nome do banco). Confira a sintaxe da conexão:

```
<?php
$objetoInstancia = new Mysqli("<servidor>", "<usuário>",
"<senha>", "<nome do banco de dados>");
?>
```



conecte-se

A classe MySQLi — Representa uma conexão entre o PHP e um banco de dados MySQL:



Após a conexão ser realizada, ou seja, após o objeto de conexão ser instanciado, ele carregará consigo todos os atributos e comportamentos da classe. Desta forma, poderemos utilizar os métodos que a classe disponibiliza, além de trabalhar com as declarações preparadas.

PreparedStatement

A API do Mysql fornece uma maior performance e segurança em suas consultas ao banco de dados. Para isso, devemos utilizar o recurso de preparação de consultas, recurso conhecido como *PreparedStatement* (declaração preparada). Esse recurso prepara as consultas de forma que recebam por parâmetros as variáveis necessárias, evitando, assim, a injeção de códigos maliciosos.

Uma instrução preparada ou parametrizada é usada para executar a mesma instrução repetidamente com alta eficiência, melhorando a performance, pois é preparada apenas uma vez e executada quantas vezes forem necessárias. A sua execução consiste em duas etapas: preparar e executar.

No estágio de preparação, utilizaremos a função `prepare()`. O método `prepare` enviará uma consulta SQL com valores vazios como espaços reservados (com um ponto de interrogação para cada valor). Após a preparação, é hora de realizar as vinculações, neste caso, utilizaremos o método `bind_param()`, que vinculará as variáveis aos espaços reservados delimitados pelo sinal de (?) na consulta preparada, informando cada variável, junto com seu tipo. Existem, atualmente, quatro tipos de variáveis, sendo:

i - Integer

s - String

d - duplo

b - Blob

Após a preparação e a vinculação dos parâmetros, é hora de executar a consulta, para isso, utilizaremos o método `execute()`, que fará uma verificação da sintaxe e inicializará os recursos internos do servidor para uso posterior. Confira o exemplo:

```
$sql = $conectar->prepare("SELECT FROM noticias WHERE autor  
= ?")
```

```
// Atribui valores às variáveis da consulta
$autor = "Rafael";
$sql->bind_param('s', $autor);
// Executa a consulta
$sql->execute();
```

Não se preocupe, retomaremos esses códigos nas próximas aulas. O objetivo é que você entenda a diferença entre eles.

Extensão PDO

O PDO (PHP *Data Objects*) é um módulo/extensão de PHP que trabalha com o paradigma Orientado a Objetos e que tem por objetivo prover uma padronização da forma com que o PHP se comunica com um banco de dados relacional. PDO pode ser definido como uma interface que define um conjunto de classes e a assinatura dos métodos de comunicação com uma base de dados.

O PDO nos oferece recurso de abstração de implementação em toda a nossa aplicação, sem se preocupar com qual banco utilizamos. Essa abstração se refere ao fato de que uma mudança na escolha do banco, após colocado em produção, não trará grandes problemas à sua aplicação.

Uma vantagem do PDO em relação a outras extensões de banco de dados para PHP estudadas é a possibilidade de executar consultas em diferentes SGBDs (Sistema Gerenciador de Banco de Dados), isso é muito importante. Desta forma, o desenvolvedor não precisa conhecer os recursos de várias extensões diferentes para trabalhar com diferentes SGBD na linguagem.

Para utilizar o PDO, como já informado anteriormente, você deve ativar a extensão de acordo com o SGDB. Após ativar, você pode agir da seguinte forma: toda a conexão com banco de dados é realizada ao criar uma instância da classe PDO. O construtor da classe PDO recebe as informações do banco como parâmetro obrigatório, além dos parâmetros opcionais username, password. Confira a sintaxe a seguir:

```
$objeto_de_conexao = new PDO("<drive_do_banco>: host =
<localização_do_banco>; dbname = <nome_do_banco>", "<usuario_
do_banco>", "");
```

3 CONECTANDO PHP ÀS APIS E EXECUTANDO COMANDOS SQL em um programa PHP (insert e select)

Apresentaremos as conexões do PHP com as APIs e aplicaremos os comandos de inserção e seleção simples de registros utilizando a API *Mysqli* e a *PDO Object*. Todos os nossos códigos seguirão o paradigma de Orientação a Objetos. Se tiver alguma dúvida, retorne aos capítulos anteriores.

Agora, criaremos uma conexão com o nosso servidor. Essa conexão, por sua vez, pode ser local ou remota, tudo dependerá da localização do seu servidor. No nosso caso, realizaremos uma conexão local.

Primeiramente, criaremos a base de dados, para isso, abra o seu gerenciador de banco de dados e execute o comando a seguir:

```
CREATE DATABASE dbteste;
```

Agora, crie um arquivo com o nome **conexao.php** e digite o seguinte código:

```
<?php
abstract class Conexao{
    protected function conectarDB(){
        try{
```



```

        return $conectar = new Mysqli("localhost",
"root", "", "dbteste");
    }catch(mysqli_sql_exception $e){
        return $e->getMessage();
    }
}
}
?>

```

Notem que criamos uma classe abstrata com o nome de `Conexao`. Dessa forma, a classe que necessitar utilizar o banco de dados deverá estender (generalização/herança) a classe `Conexao`. Caso a classe não seja instanciada, o servidor do PHP retornará um **Fatal error**: `Uncaught Error: Cannot instantiate abstract class Conexao`. Além de não poder instanciar a classe, o método `conectarDB` possui visibilidade `protected`, ou seja, esse método só pode ser acessado pela própria classe ou classes que estendem dela.



quadro-resumo

API: conceito

Em uma definição formal, o conceito de API está relacionado a um conjunto de rotinas e padrões estabelecidos por um software para a utilização das suas funcionalidades por outros aplicativos.

O conceito de API nada mais é do que uma forma de comunicação entre sistemas. Elas permitem a integração entre dois sistemas, em que um deles fornece informações e serviços que podem ser utilizados pelo outro, sem a necessidade de o sistema que consome a API conhecer detalhes de implementação do software.

Fonte: Take (2019, on-line).

Dentro do método `protected function conectarDB()`, colocamos um tratamento de exceção `try{ }catch() { }`, de forma que, se tudo der certo com a conexão, o `Try` será executado. Caso a conexão falhe, o `catch` é acionado, realiza a captura do erro da exceção e devolve para quem a chamou.

A linha da conexão `return $conectar = new Mysqli("localhost", "root", "", "dbteste");` realiza a conexão com o banco de dados MySQL/MariaDB por intermédio da extensão `mysqli`. Para realizar a conexão com o servidor de banco de dados, são necessários os parâmetros de acesso ao banco: localização (localhost ou 127.0.0.1), usuário (root), senha (no meu caso, é vazio) e banco de dados (neste caso, é dbteste). Vale lembrar que não é ideal utilizar a senha root do seu banco; se possível, crie um usuário específico para a aplicação com uma senha. Por questões pedagógica, utilizaremos este usuário.

Executando comandos SQL com PHP ao banco

Agora que já possuímos um objeto `$conectar` do tipo `Mysqli`, utilizaremos o método `query()` para executar todos dos comandos SQL. Primeiramente, construiremos uma string de consulta para que, futuramente, possa ser executada pelo método. Veja a seguir:

```
$sql = "INSTRUÇÃO EM SQL";
```

Agora, aplicando o método `query()`, temos:

```
$retorno = $this->conectar->query($sql);
```

Vamos praticar?

Agora que já temos o banco de dados criado, execute o comando em SQL a seguir para a criação da tabela `usuario`.

```
CREATE TABLE usuario (
    id int(11) NOT NULL,
    nome varchar(255) DEFAULT NULL,
    email varchar(255) DEFAULT NULL,
    dataNascimento date DEFAULT NULL,
    telefone varchar(20) DEFAULT NULL,
    dataCadastro datetime DEFAULT NULL
)
```

Agora, criaremos um arquivo com o nome **Usuario.php**, para tanto, digite o código a seguir:

```
<?php
require_once('Conexao.php');

class Usuario extends Conexao{
    protected $conectar;
    private $nome, $email, $dataNascimento, $telefone;

    public function __construct(){
        $this->conectar = $this->conectarDB();
    }

    public function setDados($dados) {
        $objCadastro = json_decode($dados);
        $this->nome = $objCadastro->nome;
        $this->email = $objCadastro->email;
        $this->dataNascimento = $objCadastro->dataNascimento;
        $this->telefone = $objCadastro->telefone;
    }

    public function inserirUsuario($dados){
        $this->setDados($dados);

        $sql = "INSERT INTO usuario (nome, email, dataNascimento,
        telefone)      value      ('$this->nome','$this->email',      '$this->dataNascimento', '$this->telefone')";

        $retorno = $this->conectar->query($sql);
        if ($retorno)
            return 1;
        else
            return 0;
    }
}
```

```

public function listarUsuario(){
    $sql = "SELECT * FROM usuario order by nome ASC";

    $retorno = $this->conectar->query($sql);

    $usuarios = array();
    while($linha=$retorno->fetch_assoc()){
        $usuarios[]=$linha;
    }
    return json_encode($usuarios);
}

?>

```

Notem que a classe `Usuario` estende da classe `Conexao`, dessa forma, temos o método `conectarDb()` que será utilizado para realizar a conexão. Na classe `Usuario`, criamos um atributo privado `conectar` que receberá a conexão realizada no momento da instância da classe `Usuario` pelo método construtor. O método construtor armazenará, no atributo `conectar`, o retorno do método `conectaBd`. Após esse ponto, poderemos utilizar o atributo `conectar` em toda a nossa classe.

Notem que declaramos os atributos de usuário, todos com a visibilidade *private*. Criamos um método `setDados($dados)`, que tem a responsabilidade de realizar o *decode* do JSON recebido em um objeto `$objCadastro = json_decode($dados)` e, depois, setar cada atributo desse objeto em um atributo da classe.

No método `inserirUsuario($dados)`, ele recebe um JSON, chama o método `setDados`, passando por parâmetro o JSON recebido. Depois de interpretado e setado nos atributos, é hora de montar a *string* e executar a *query* no banco.

Após inserir o valor, retornamos um resultado *true* ou *false*, representado por 1 e 0, respectivamente.

Na classe `Usuario`, temos o método `listarUsuario` que realizará uma consulta à tabela `usuario`, e retornará todos os usuários **ordenados por nome em ordem ascendente**. Esse retorno é armazenado no atributo `$sql`, que será um parâmetro a ser executado no método `query()`. Após executar o

método *query(\$sql)*, necessitamos utilizar um laço de repetição para percorrer esse resultado e de um *array \$usuarios = array()* para receber os dados que serão tratados. Este laço *while(\$linha=\$retorno->fetch_assoc())* é responsável por pegar as informações no objeto e associar os campos (nome e valor) pelo método *fetch_assoc()* em uma variável local *\$linha*. Essa variável será armazenada na posição atual do *array \$usuarios[]=\$linha*. Após preencher o *array*, montaremos um JSON e enviaremos para quem o chamou **return json_encode(\$results)**.

Para apresentar o retorno, criaremos um arquivo **index.php**, incluiremos a classe **Usuario**, instanciaremos o objeto **\$usuario** e invocaremos o método **listarUsuario**. Após o processamento, será armazenado o retorno no atributo **\$listaUsuarios**.

Uma outra forma de utilizar a classe *mysqli* é fazer a preparação da *query* de consulta, ou seja, aplicar as declarações preparadas (*PreparedStatement*).

Edite a classe *Usuario.php* e inclua este método:

```
public function inserirUserPreparado($dados){
    $this->setDados($dados);

    $sql = "INSERT INTO usuario (nome, email, dataNascimento,
    telefone) value (?, ?, ?, ?)";

    $stmt = $this->conectar->prepare($sql);
    $stmt->bind_param("ssss", $this->nome, $this->email,
    $this->dataNascimento, $this->telefone);

    $results = $stmt->execute();

    if ($results > 0)
        return 1;
    else
        return 0;
}

public function buscarUsuario($id){
    $sql = "SELECT * FROM usuario where id = ?";
```

```

$stmt = $this->conectar->prepare($sql);
$stmt->bind_param("d", $id);

$stmt->execute();

$results = $stmt->get_result()->fetch_all(MYSQLI_ASSOC);
return json_encode($results);
}

```

No método `inserirUserPreparado($dados)`, a parte inicial é idêntica ao `inserir` que utilizamos anteriormente. Agora, montaremos a *string* que irá para o banco. **`$sql = "INSERT INTO usuario (nome, email, dataNascimento, telefone) value (?, ?, ?, ?)"`**, **preparar a string** `$stmt = $this->conectar->prepare($sql)`; e realizar a associação dos valores `$stmt->bind_param("ssss", $this->nome, $this->email, $this->dataNascimento, $this->telefone)`. Por fim, executaremos `$results = $stmt->execute()`.

Notem que a preparação da *string* `$sql = "SELECT * FROM usuario where id = ?"` contém um espaço reservado para a associação de um parâmetro `$id` do tipo inteiro `$stmt->bind_param("d", $id)` que será recebida por parâmetro na função. Na sequência, executamos a consulta no banco de dados pelo método `$stmt->execute()`. Após a execução, resgatamos o objeto resultante pelo método `$stmt->get_result()->fetch_all(MYSQLI_ASSOC)`, o qual resgatará e associará cada registro em uma posição no *array* `$result`. Após a associação, encodamos o resultado em um vetor de JSON `return json_encode($results)`; e o retornamos.

Para testarmos, edite o arquivo `index.php` e inclua a chamada da função.

```

<?php
    require_once('Usuario.php');
    require_once('Conexao.php');

    $usuario = new Usuario();

```

```

$dados=[
    'nome' => 'Carlos Fernades',
    'email' => 'josefernades@hotmail.com',
    'dataNascimento' => '2010-10-12',
    'telefone' => '(48) 98754-5825'
];
$dados = json_encode($dados);

$addUsuarios = $usuario->inserirUsuario($dados);

echo "<h2>Listar Usuário:</h2>";
$listaUsuarios = $usuario->listarUsuario();
echo $listaUsuarios;

$dados=[
    'nome' => 'Felipe Fernades',
    'email' => 'josefernades@hotmail.com',
    'dataNascimento' => '2010-10-12',
    'telefone' => '(48) 98754-5825'
];
$dados = json_encode($dados);
$addUsuario2 = $usuario->inserirUserPreparado($dados);
echo "<h2>Busca e Usuário:</h2>";
$buscaUsuario = $usuario->buscarUsuario(2);
echo $buscaUsuario;

?>

```

A última API a ser testada é a PDO. De forma similar à classe `Conexao`, criaremos a classe `ConexaoPDO`. Digite o código a seguir.

```

<?php
abstract class ConexaoPDO{
    protected function conectarDB(){
        try{

```

```

                                return $conectar = new
PDO("mysql:host=localhost;dbname=dbteste", "root", "");
                                } catch(PDOException $e) {
                                    return $e->getMessage();
                                }
                            }
                        }
                    }
                }
            }
        }
    }
?>

```

Como já explicamos na conexão anterior, a única diferença é a linha da conexão **\$conectar = new PDO("mysql:host = localhost; dbname = dbteste", "root", "")**.

Agora, criaremos a classe `UsuarioPDO`. Deixe conforme o código a seguir:

```

<?php
require_once('ConexaoPDO.php');

class Usuario extends ConexaoPDO{
    protected $conectar;
    private $nome, $email, $dataNascimento, $telefone;

    function __construct(){
        $this->conectar = $this->conectarDB();
    }

    function setDados($dados){
        $objCadastro = json_decode($dados);
        $this->nome = $objCadastro->nome;
        $this->email = $objCadastro->email;
        $this->dataNascimento = $objCadastro->dataNascimento;
        $this->telefone = $objCadastro->telefone;
    }

    public function inserirUserPreparado($dados){
        $this->setDados($dados);

        $sql = "INSERT INTO usuario
(nome, email, dataNascimento, telefone)

```


value

```

        (:NOME, :EMAIL, :DATANASCIMENTO, :TELEFONE)";

$stmt = $this->conectar->prepare($sql);

$results = $stmt->execute(
    array(
        ":NOME"=>$this->nome,
        ":EMAIL"=>$this->email,
        ":TELEFONE"=>$this->telefone,
        ":DATANASCIMENTO"=>$this->dataNascimento
    )
);

$count = $stmt->rowCount();
if($count > 0) return true;
else return false;

}

public function listarUsuario(){
    $sql = "SELECT * FROM usuario order by nome ASC";

    $stmt = $this->conectar->prepare($sql);
    $stmt->execute();

    $results = $stmt->fetchAll(PDO::FETCH_ASSOC);
    return json_encode($results);
}

public function buscarUsuario($id){

    $sql = "SELECT * FROM usuario where id = :ID";

    $stmt = $this->conectar->prepare($sql);
    $stmt->bindParam(":ID", $id);

```

```

        $stmt->execute();

        $results = $stmt->fetchAll(PDO::FETCH_ASSOC);
        return json_encode($results);
    }

}

?>

```

Quando trabalhamos com PDO, utilizamos as declarações preparadas, contudo seguindo o padrão do PDO.

O início dessa classe é idêntica à classe `Usuario`, o que altera são os métodos. O método `inserirUserPreparado`, prepara a inserção do registro com os parâmetros **:NOME**, **:EMAIL**, **:DATANASCIMENTO**, **:TELEFONE**. Contudo, não utilizamos o `bindParam()`, executamos um array dentro do comando `execute()`. Essa é uma outra forma de se utilizar a associação e a preparação. Caso deseje optar por fazer os `bindParam()`, terá que fazer para todos os atributos.

Notem que, no método `listarUsuario`, estamos apenas preparando a seleção `$stmt = $this->conectar->prepare("SELECT * FROM usuario order by nome ASC")`. Não estamos associando parâmetros à *string* preparada. Logo, executamos e resgatamos todos os registros que estão no objeto `$stmt->fetchAll(PDO::FETCH_ASSOC)` e, após, associamos cada registro ao atributo `$results`.

Agora, no método `buscarUsuario($id)`, estamos realizando a preparação da *string* que será executada `$stmt = $this->conectar->prepare("SELECT * FROM usuario where id = :ID")`. Após preparada, associaremos o parâmetro utilizando o método `bindParam` conforme: `$stmt->bindParam(":ID", $id)`. Na sequência, executa a consulta preparada e, por fim, realiza a associação do objeto, retornado no atributo `results`.

Agora, crie o arquivo `indexPDO.php` e deixe-o desta forma:

```
<?php
    require_once('UsuarioPDO.php');

    $usuario = new Usuario();
    echo "<h2>Listar Usuário:</h2>";
    $listaUsuarios = $usuario->listarUsuario();

    echo $listaUsuarios;

    echo "<h2>Busca e Usuário:</h2>";
    $buscaUsuario = $usuario->buscarUsuario(1);
    echo $buscaUsuario;
?>
```

A execução desse arquivo dependerá dos *inserts* realizados na tabela.

EXECUTANDO COMANDOS SQL EM UM PROGRAMA PHP (update, delete)



Nesta aula, edite o arquivo da classe Usuario e acrescente os métodos a seguir. Os dois primeiros métodos são para editar e excluir o usuário utilizando a API Mysqli.

```
public function editarUsuario($id, $dados){
    $this->id = $id;
    $this->setDados($dados);

    echo $sql = "UPDATE usuario SET nome = '$this->nome',
email='$this->email',    dataNascimento='$this->dataNascimento',
telefone='$this->telefone' WHERE id = '$this->id'";

    $retorno = $this->conectar->query($sql);
    if ($retorno)
        return 1;
    else
        return 0;
}

public function deletarUsuario($id){
    $this->id = $id;
```

```

        $sql = "DELETE * FROM usuario WHERE id = $this->id";

        $retorno = $this->conectar->query($sql);
        if ($retorno)
            return 1;
        else
            return 0;
    }

```

Os dois últimos primeiros métodos são para editar e excluir o usuário utilizando a API Mysql prepared.

```

    public function editarUsuarioPreparado($id, $dados){
        $this->id = $id;
        $this->setDados($dados);

        $sql = "UPDATE usuario SET nome = ?, email=?,
dataNascimento=?, telefone=? WHERE id = ?";

        $stmt = $this->conectar->prepare($sql);
        $stmt->bind_param("sssd", $this->nome, $this->
>email, $this->dataNascimento, $this->telefone, $this->id);
        $results = $stmt->execute();

        if ($results > 0)
            return 1;
        else
            return 0;
    }

    public function deletarUsuarioPreparado($id){
        $this->id = $id;
        $sql = "DELETE * FROM usuario WHERE id = ?";

        $stmt = $this->conectar->prepare($sql);
        $stmt->bind_param("d", $id);

        $results = $stmt->execute();
    }

```

```

        if ($results > 0)
            return 1;
        else
            return 0;
    }

```

Edite o arquivo `index.php` de forma que possamos testar os trechos de código a seguir:

```

$id = 3;
$dados=[
    'nome' => 'Carlos Alberto',
    'email' => 'carlos@hotmail.com',
    'dataNascimento' => '2010-10-12',
    'telefone' => '(48) 98754-5825'
];
$dados = json_encode($dados);
$editarUsuarios = $usuario->editarUsuarioPreparado($id,
$dados);

$id = 3;
$deletarUsuarios = $usuario->deletarUsuario($id);
$id = 4;
$deletarUsuarios = $usuario->deletarUsuarioPreparado($id);

```

Agora, realizaremos o editar e excluir com o PDO. Para isso edite a classe `UsuarioPDO` e implemente os métodos a seguir:

```

public function editarUser($id, $dados) {
    $this->setDados($dados);
    $this->id = $id;
}

```

```
$sql = "UPDATE usuario SET nome=:NOME, email=:EMAIL,
dataNascimento=:DATANASCIMENTO, telefone=:TELEFONE WHERE id= :ID";
```

```
$stmt = $this->conectar->prepare($sql);
```

```
$results = $stmt->execute(
    array(
        ":NOME"=>$this->nome,
        ":EMAIL"=>$this->email,
        ":TELEFONE"=>$this->telefone,
        ":DATANASCIMENTO"=>$this->dataNascimento,
        ":ID"=>$this->id
    )
);
```

```
$count = $stmt->rowCount();
if($count > 0) return true;
else return false;
```

```
}
```

```
public function deletarUser($id){
    $this->id= $id;
```

```
$sql = "DELETE FROM usuario WHERE id= :ID";
```

```
$stmt = $this->conectar->prepare($sql);
$stmt->bindParam(":ID", $this->id);
$results = $stmt->execute();
```

```
$count = $stmt->rowCount();
if($count > 0) return true;
else return false;
```

```
}
```

Edite o arquivo `indexPDO.php` de forma que possamos testar os trechos de código a seguir:

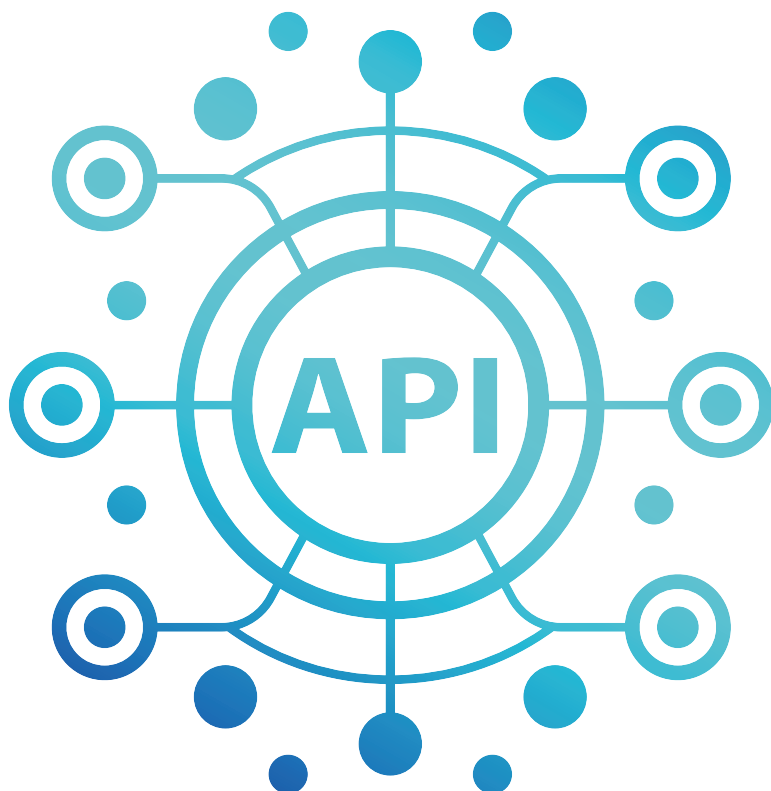
```
$id = 8;

$dados=[
    'nome' => 'Tania Vanso',
    'email' => 'vanso@hotmail.com',
    'dataNascimento' => '2010-10-12',
    'telefone' => '(48) 98754-5825'
];

$dados = json_encode($dados);
echo $editarUsuario2 = $usuario->editarUser($id, $dados);
echo "<h2>Busca e Usuário:</h2>";

$id = 8;
echo $excluirUsuario2 = $usuario->deletarUser($id);
echo "<h2>Busca e Usuário:</h2>";
```

Após o estudo dessas API, sugiro a você praticar bastante para que possa se familiarizar com todos esses métodos, pois eles são base no mundo de Programação Orientada a Objetos.



5 APLICAÇÃO PRÁTICA DOS COMANDOS SQL

Agora, colocaremos em prática um exemplo utilizando o *drive* de conexão PDO. Recordaremos alguns pontos que antecedem a construção do sistema. Analisemos a situação a seguir:

“O senhor José é um comerciante do interior de São Paulo e possui uma papelaria no centro da cidade de Flórida Paulista. Ele deseja informatizar o seu comércio, e seu primeiro pedido é possuir algumas rotinas para cadastrar, alterar, excluir, listar todos os produtos e listar apenas um produto. Para o cadastro dos produtos, ele deseja ter apenas um código único: nome do produto, descrição, quantidade e valor. Sr. José deseja, ainda, que uma mesma tela tenha o formulário para preenchimento e uma tabela que liste os produtos em ordem crescente por produto. Após cada cadastro, edição e exclusão, o sistema deve apresentar uma mensagem personalizada. Nessa tabela, deve haver dois botões que permitem alterar e excluir os registros sem sair dessa tela”.

Notem que, neste estudo, podemos retirar requisitos funcionais e não funcionais, bem como as regras de negócio. Vejamos os requisitos:

REQUISITOS FUNCIONAIS	
Requisito	Nome
RF001	O sistema permitirá que os produtos sejam cadastrados e deverá ter os seguintes campos: id, produto, descricao, quantidade e valor.
RF002	O sistema permitirá que os produtos sejam alterados.
RF003	O sistema permitirá que os produtos sejam excluídos.
RF004	O sistema permitirá que os produtos sejam listados.
RF005	O sistema permitirá que os produtos sejam pesquisados.
REQUISITOS NÃO FUNCIONAIS	
RNF001	O formulário de cadastro e alteração, bem como a tabela que lista os produtos devem estar na mesma tela.
RNF002	O sistema deverá apresentar mensagens de sucesso ou de erro.

Quadro 2 - Exemplos de requisitos dos sistemas / Fonte: os autores.

Vejamos como ficou nosso diagrama de Casos de Uso.

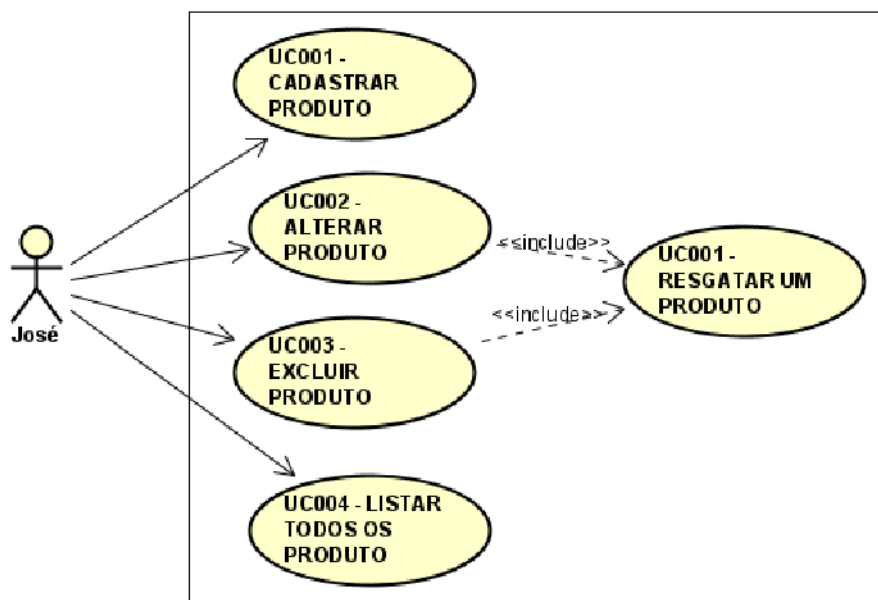


Figura 1 - Diagrama de Casos de Uso do sistema de produto / Fonte: os autores.

Agora, vejamos o diagrama de classe. No diagrama, temos duas classes:

- A primeira é a classe de `Conexao` com o banco de dados. Essa classe é abstrata, assim só pode ser estendida, e temos um método protegido `conectaDB()`.
- Na segunda classe, temos alguns atributos e, dentre eles, temos um atributo na classe `Produto`, o atributo `id`, normalmente, ele fica implícito na classe.

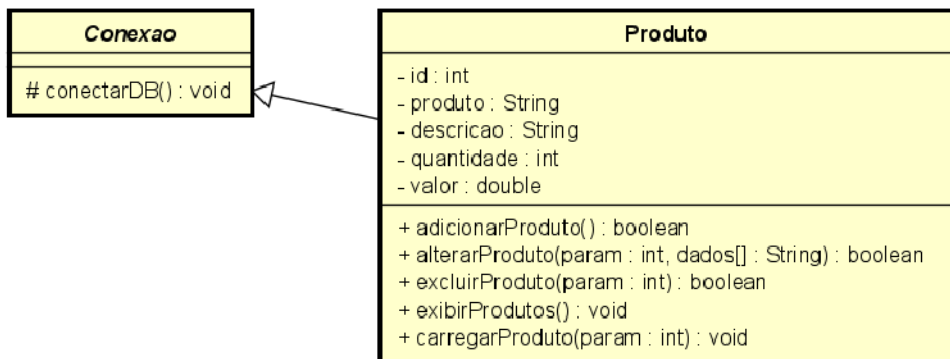


Figura 2 - Diagrama de Classe do sistema de produto / Fonte: os autores.

Agora, crie uma tabela em nossa base de dados `dbTeste` conforme o código a seguir.

```

CREATE TABLE produto (
  id int(11) NOT NULL PRIMARY KEY AUTO_INCREMENT,
  produto varchar(255) NOT NULL,
  descricao varchar(255) NOT NULL,
  quantidade int(11) NOT NULL,
  valor decimal(10,2) NOT NULL
);
  
```

```

INSERT INTO produto (produto, descricao, quantidade, valor)
VALUES
  ('Borracha', 'Faber Castel Verde Dura', 15, '1.50'),
  ('Caneta', 'BIC - Esferográfica cor vermelho', 78, '1.78');
  
```

Após a criação dessa tabela, iniciaremos o nosso desenvolvimento.

Criaremos um arquivo **index.php**. Deixe-o conforme o código:

```
<!doctype html>
<html lang="pt-br">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1, shrink-to-fit=no">

    <!-- Bootstrap CSS -->
    <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.
com/bootstrap/4.0.0/css/bootstrap.min.css" integrity="sha384-Gn
5384xqQ1aoWXA+058RXPxPg6fy4IWvTNh0E263XmFcJlSAwiGgFAW/dAiS6JXm"
crossorigin="anonymous">

    <title></title>
  </head>
  <body>
    <div class="container">
      <?php
        $acao="";
      ?>

      <div class="row">
        <div class="col col-lg-10">
          <?php
            include('formulario.php');
          ?>
        </div>
      </div>
      <div class="row">
        <div class="col col-lg-10">
          <?php
            include('ListarProdutos.php');
          ?>
```

```

        </div>
    </div>
</body>
</html>

```

Nesse arquivo, notem que incluímos o link do *framework* bootstrap 4.0 e incluímos dois scripts php, um para incluir o `formulario.php`, e outro, `listarProdutos.php`. Colocamos uma variável para controlar a visibilidade do formulário e da tabela.

Criaremos um arquivo **conexao.php**. Deixe-o conforme o código:

```

<?php
    abstract class Conexao{
        protected function conectarDB(){
            try{
                return $conectar = new
PDO("mysql:host=localhost;dbname=dbteste", "root", "");
            }catch(PDOException $e){
                return $e->getMessage();
            }
        }
    }
?>

```

Esse arquivo de conexão já foi explicado em um tópico anterior.

Agora, criaremos um arquivo de **autoload.php**. Esse arquivo serve para carregar classes de forma automática. A função **spl_autoload_register()**, funciona toda vez que você chamar uma classe pelo operador *new*. Dessa forma, não precisamos incluir diversas classes, deixamos, para ela, a inclusão, e deixe-o conforme o código:

```

<?php
spl_autoload_register(function ($class_name) {
    include $class_name . '.php';
});

```

Após criados e codificados os arquivos anteriores, criaremos a classe `Produto` no arquivo **`Produto.php`** e o deixamos conforme o código. A classe `Produto` estende de `Conexao`, dessa forma, implementamos o construtor, de forma que receba a conexão e armazene o objeto de conexão no atributo `$conn`. A segunda função é a `setDados($dados)`, que recebe um JSON contendo os atributos: `produto`, `descricao`, `quantidade` e `valor`. Inicialmente, decodamos esse JSON no atributo `$dados` por meio da função `json_decode()` e, posteriormente, setamos cada posição deste JSON nos seus respectivos atributos de classe. Notem que, nos demais métodos, implementamos o tratamento de exceção.

```
<?php
include('Conexao.php');
class Produto extends Conexao{
    private $id, $produto, $descricao, $quantidade, $valor;
    private $conn;

    public function __construct(){
        $this->conn = $this->conectarDB();
    }
    private function setDados($dados){
        $dados = json_decode($dados);
        $this->produto = $dados->produto;
        $this->descricao = $dados->descricao;
        $this->quantidade = $dados->quantidade;
        $this->valor = $dados->valor;
    }

    public function adicionarProduto($dados):bool{
        $this->setDados($dados);
        try{
            $stmt = $this->conn->prepare("INSERT INTO
produto (produto, descricao, quantidade, valor) VALUES
(:PRODUTO,:DESCRICAO,:QUANTIDADE,:VALOR)");
            $stmt->execute(
                array(
```

```

        ':PRODUTO'=>$this->produto,
        ':DESCRICAO'=>$this->descricao,
        ':QUANTIDADE'=>$this->quantidade,
        ':VALOR'=>$this->valor)
    );
    $count = $stmt->rowCount();
    return $count ? true: false;
} catch(PDOException $e) {
    echo 'Error: ' . $e->getMessage();
}
}

public function alterarProduto($id, $dados):bool{
    $this->id = $id;
    $this->setDados($dados);

    try{
        $stmt = $this->conn->prepare("
        UPDATE produto SET
        produto = :PRODUTO,
        descricao = :DESCRICAO,
        quantidade = :QUANTIDADE,
        valor = :VALOR
        WHERE id = :ID");
        $stmt->execute(
            array(
                ':PRODUTO'=>$this->produto,
                ':DESCRICAO'=>$this->descricao,
                ':QUANTIDADE'=>$this->quantidade,
                ':VALOR'=>$this->valor,
                ':ID'=>$this->id)
            );
        $count = $stmt->rowCount();
        return $count ? true: false;
    } catch(PDOException $e) {

```

```

        echo 'Error: ' . $e->getMessage();
    }
}

public function excluirProduto($id):bool{
    $this->id = $id;
    try{
        $stmt = $this->conn->prepare("
        DELETE FROM produto
        WHERE id = :ID");
        $stmt->bindParam(':ID', $this->id);
        $stmt->execute();
        $count = $stmt->rowCount();
        return $count ? true: false;
    }catch(PDOException $e) {
        echo 'Error: ' . $e->getMessage();
    }
}

public function exibirProdutos(){
    try{
        $stmt = $this->conn->prepare("
        SELECT * FROM produto ORDER BY produto ASC");
        $stmt->execute();

        $results = $stmt->fetchAll(PDO::FETCH_ASSOC);

        return $results;
    }catch(PDOException $e) {
        echo 'Error: ' . $e->getMessage();
    }
}

public function carregarProduto($id){
    $this->id = $id;
    try{
        $stmt = $this->conn->prepare("SELECT * FROM produto
WHERE id = :ID");

```



```

$stmt->bindParam(':ID', $this->id);
$stmt->execute();

$results = $stmt->fetchAll(PDO::FETCH_ASSOC);
return $results[0];
} catch(PDOException $e) {
    echo 'Error: ' . $e->getMessage();
}
}
}

```

Criaremos um arquivo **formulario.php** e o deixaremos conforme o código. Nesse arquivo, tivemos que realizar diversas validações de forma que evitasse erros de parâmetros na tela.

O primeiro foi verificar a resposta que o formulário recebe sobre as ações de inserir, alterar e excluir `if(isset($_GET["resposta"]) && !empty($_GET["resposta"]) && isset($_GET["mensagem"]) && !empty($_GET["mensagem"]))`, armazenando estes parâmetros de resposta e mensagem: `$reposta = $_GET["resposta"];` `$mensagem = $_GET["mensagem"];`, para que possam ser impressos na tela mediante outras validações.

O segundo ponto de validação foi verificar se recebemos uma ação e, depois, `if(isset($_GET["acao"]) && !empty($_GET["acao"]))`, receber a ação e verificar qual ação executar. Se for editar, primeiro, precisamos receber e armazenar o id instanciar a classe e invocar e armazenar o retorno do método `$resgatarProduto = $produto->carregarProduto($id)`. Caso a ação seja inserir, apenas receber a ação.

Agora, se não houver valores no atributo de ação, devemos inicializar o nosso array `resgatarProduto` com todas as posições vazias, a fim de evitar erros de notice no formulário por falta de valores. Após, temos o formulário, o qual serve para inserir e editar um produto. Em cada `input`, temos um atributo `name` específico para cada item do *array*. Além deste, temos o atributo `value`, o qual recebe e imprime o valor da posição do *array*.

Temos mais uma validação que é essencial para o funcionamento do formulário, pois ele está controlando por meio do atributo `acao` quais dados serão encaminhados para o `controleProduto.php`.

```

<?php
include('autoLoad.php');

if ($_GET){
    if(isset($_GET["resposta"]) && !empty($_GET["resposta"]))
    && isset($_GET["mensagem"]) && !empty($_GET["mensagem"])){
        $reposta = $_GET["resposta"];
        $mensagem = $_GET["mensagem"];
        if($reposta=="Sucesso"){
            ?>
            <div class="alert alert-success" role="alert">
                <?=$mensagem;?>
            </div>
            <?php
        }elseif($reposta=="Erro"){
            ?>
            <div class="alert alert-danger" role="alert">
                <?=$mensagem;?>
            </div>
            <?php
        }else{
            ?>
            <div class="alert alert-warning" role="alert">
                Algo de errado com a Aplicação
            </div>
            <?php
        }
    }

    if(isset($_GET["acao"]) && !empty($_GET["acao"])){
        $acao = $_GET["acao"];
        if($acao == "editar"){
            if(isset($_GET["id"]) && !empty($_GET["id"])){

```

```

        $id = $_GET["id"];
        $produto = new Produto();
        $resgatarProduto = $produto->carregarProduto($id);
    }
    }elseif($acao == "inserir"){
        $acao = $_GET["acao"];
    }else{
    }
    }else{
    }
}
if($acao==""){
    $resgatarProduto=array("produto"=>"", "descricao"=>"",
"quantidade"=>"", "valor"=> "");
}
?>
<form action="controleProduto.php" method="POST">
    <div class="row">
        <div class="col col-lg-6">
            <h1>Formulário</h1>

            <div class="form-group">
                <label for="produto">Nome do Produto:</label>
                <input type="text" class="form-control" id="produto"
name="produto" value="<?=$resgatarProduto["produto"];?>">
            </div>
            <div class="form-group">
                <label for="descricao">Descrição:</label>
                <input type="text" class="form-control" id="descricao"
name="descricao" value="<?=$resgatarProduto["descricao"];?>">
            </div>

```

```

        </div>
    </div>
    <div class="row">
        <div class="col col-lg-3">
            <div class="form-group">
                <label for="quantidade">Quantidade:</label>
                <input type="text" class="form-control" id="quantidade"
name="quantidade" value="<?=$resgatarProduto["quantidade"];?>">
            </div>
        </div>
        <div class="col col-lg-3">
            <div class="form-group">
                <label for="valor">Valor:</label>
                <input type="text" class="form-control"
id="valor" name="valor" value="<?=$resgatarProduto["valor"];?>">
            </div>
        </div>
    </div>
    <div class="row">
        <div class="col col-lg-6">
            <?php
            if ($acao=="editar"){
                ?>
                <input type="hidden" name='id' value="<?=$id;?>">
                <input type="hidden" name='operacao'
value="alterar">
                <button type="submit" class="btn btn-
primary">ALTERAR</button>
                <?php
            }else{
                ?>
                <input type="hidden" name='operacao'
value="inserir">

```

```

        <button type="submit" class="btn btn-
primary">ADICIONAR</button>
        <?php
        }?>

    </div>
</div>
</form>

```

Criaremos um arquivo **controleProduto.php** e o deixaremos conforme o código. Iniciamos o arquivo com a inclusão do arquivo de autoload.php, e realizamos o tratamento dos parâmetros recebidos. Para os parâmetro de dados, os encodamos em um JSON `$dados = json_encode($_POST)`. Posteriormente, verificamos se a operação era inserir, alterar ou excluir. Para cada ação, realizaram-se as suas devidas tratativas. Notem que, em todas essas tratativas, elas direcionam para a página `index.php`, por meio do comando `Header('location: <página>')`, dois parâmetros `resposta` e `mensagem`, sendo que o primeiro leva ao resultado `Sucesso` ou `Erro`, e o segundo, às mensagens personalizadas.

```

<?php
    require_once('autoload.php');
    if (!empty($_POST)) {
        if(isset($_POST['operacao']) && !empty($_POST['operacao']))
        {
            $operacao = $_POST['operacao'];
        }
        if (isset($_POST['produto']) && isset($_POST['descricao'])
        && isset($_POST['quantidade']) && isset($_POST['valor'])) {
            if (!empty($_POST['produto']) && !empty($_
POST['descricao']) && !empty($_POST['quantidade']) && !empty($_
POST['valor'])) {
                $dados = json_encode($_POST);
            }
        }
    }

```

```

    }
    if($operacao=="inserir"){
        $produto = new Produto();

        $resultado = $produto->adicionarProduto($dados);
        if($resultado) header('location: index.
php?resposta=Sucesso&mensagem=Produto inserido com sucesso');
        else header('location: index.
php?resposta=Erro&mensagem=Ocorreu um erro ao inserir o produto');
    }elseif($operacao=="alterar" && isset($_POST['id']) &&
!empty($_POST['id'])){
        $id = $_POST["id"];
        $produto = new Produto();
        $resultado = $produto->alterarProduto($id, $dados);
        if($resultado) header('location: index.
php?resposta=Sucesso&mensagem=Produto alterado com sucesso');
        else header('location: index.
php?resposta=Erro&mensagem=Ocorreu um erro ao alterar o produto');
    }else{
    }
    }elseif (!empty($_GET)) {
        if(isset($_GET['operacao']) && !empty($_GET['operacao']))
        {
            $operacao = $_GET['operacao'];
        }

        if($operacao=="excluir" && isset($_GET['id']) &&
!empty($_GET['id'])){
            $id = $_GET["id"];
            $produto = new Produto();
            $resultado = $produto->excluirProduto($id);
            if($resultado) header('location: index.
php?resposta=Sucesso&mensagem=Produto excluído com sucesso');

```

```

        else header('location: index.
php?resposta=Erro&mensagem=Ocorreu um erro ao excluir o produto');
    }
}

```

Agora, criaremos um arquivo `listarProdutos.php` e o deixaremos conforme o código. Notem que, neste arquivo, temos uma tabela dinâmica, mediante retorno da execução do método `exibirProdutos`. Em todas as linhas, temos dois links: o primeiro é responsável por passar os parâmetros para editar; e o segundo, para excluir o produto.

```

<div class="container">
    <div class="row">
        <div class="col col-lg-12">
            <div class="table-responsive-lg">
                <table class="table">
                    <thead>
                        <tr>
                            <th scope="col">ITEM</th>
                            <th scope="col">PRODUTO</th>
                            <th scope="col">DESCRIÇÃO</th>
                            <th scope="col">QUANTIDADE</th>
                            <th scope="col">VALOR</th>
                            <th scope="col">AÇÃO</th>
                        </tr>
                    </thead>
                    <?php
                        include('autoLoad.php');
                        $produtos = new Produto();
                        $retorno = $produtos->exibirProdutos();
                        $i=1;
                        foreach($retorno as $linha){
                            ?>

```

```

        <tr>
            <th scope="row"><?=$i++?></th>
            <td><?=$linha['produto'];?></td>
            <td><?=$linha['descricao'];?></td>
            <td><?=$linha['quantidade'];?></td>
            <td><?=number_format($linha['valor'],
2, ',', '.');?></td>

            <td>
                <a class="btn btn-primary"
href="?acao=editar&id=<?=$linha['id']?>" role="button">EDITAR</a>
                <a class="btn btn-danger"
href="controleProduto.php?operacao=excluir&id=<?=$linha['id']?>"
role="button">EXCLUIR</a>

            </td>
        </tr>
    <?php
    }
    ?>
</table>
</div>
</div>
</div>
</div>

```


Ao executar os arquivos, você poderá constatar a tela da Figura 3. Lógico que, se você incluir mais produtos, ela ficará diferente.

Formulário

Nome do Produto:

Descrição:

Quantidade:

Valor:

ADICIONAR

ITEM	PRODUTO	DESCRIÇÃO	QUANTIDADE	VALOR	AÇÃO
1	Borracha	Faber Castel Verde Dura	15	1,50	EDITAR EXCLUIR
2	Caneta	BIC - Esferográfica cor vermelho	78	1,78	EDITAR EXCLUIR

Figura 3 - Tela do sistema / Fonte: os autores.

A programação nos permite manipular as informações e, também, atua na diagramação visual do projeto. Já um banco de dados é fundamental para o armazenamento e manutenção dos dados. Por isso, ao associar as boas práticas de ambos conceitos, conseguimos criar projetos dinâmicos e altamente funcionais.

CONSIDERAÇÕES FINAIS

Chegamos ao fim de mais uma unidade com conteúdos enriquecedores. Um sistema WEB é composto, basicamente, de *FRONT END* e *BACK END*. Entretanto não vale de nada um sistema altamente atrativo ao usuário, sem *bugs* ou problemas de codificação, caso não seja possível fazer o armazenamento de dados, pois ele se torna inútil para empresa.

Os dados armazenados são extremamente úteis e necessários para a gestão estratégica da organização. Por meio deles, é possível compreender o andamento dos setores e suas metas como, também, a tomada de decisões. Já se foi a época em que uma empresa era gerida pela intuição, agora, tudo é pensado e analisado com base em diversas fontes de dados interno e externo.

Nesta unidade, pudemos relembra os conceitos sobre o uso de SGBD relacional e os seus comandos de SQL básicos necessários para a criação de *databases* e tabelas, inclusão e exclusão de dados e, por fim, alteração e seleção de dados. Tais comandos dão vida ao sistema, fazendo com que o mesmo se torne útil e eficiente à organização em que será implementado. Para exemplificar esses comandos foi adotado o SGBD Relacional MySQL, considerado um dos melhores para se trabalhar com aplicações web e a linguagem de programação PHP.

Como a base de nosso livro é o paradigma Orientado a Objetos, apresentaram-se as APIs de conexão com o banco de dados em PHP, instrumentando, junto a todos, desde uma simples conexão até, também, a manipulação dos dados. Devo lembrar que o foco de nossa disciplina é o *BACK END*, assim apresentaram-se elementos elaborados de *FRONT END*. Mesmo assim, os conteúdos apresentados não deixaram a desejar com base no esperado.

Esta unidade teve como objetivo levar até você, aluno(a), o conhecimento de como manipular dados de *input* e *output* gerados e destinados às requisições aos usuários. A implementação da API PDO *Object* esteve presente seguindo os conceitos da Orientação a Objetos. Ao final de nossa unidade, foi possível vermos uma aplicação prática dos elementos estudados em um estudo de caso. Espero que tenham gostado do material e nos vemos em nossa última unidade. Até lá, um abraço a todos!



1. José trabalha como Administrador de Banco de Dados e utiliza comandos SQLs para definir, manipular, controlar e consultar a Base de Dados da empresa. Ao ser requisitado para um novo controle de custo no sistema, precisou criar uma nova tabela no banco de dados. Utilizou o comando CREATE TABLE e definiu as permissões de acesso para que o setor de desenvolvimento pudesse utilizar essa tabela para criar o novo controle de custo no sistema.

Em relação ao comando CREATE TABLE, leia as afirmações a seguir.

- I - Como parte da Linguagem de Definição de Dados (DDL), o comando CREATE TABLE é utilizado para criar uma tabela no banco de dados.
- II - O comando CREATE TABLE é parte da DTL — *Data Transaction Language* (Linguagem de Transação de Dados) e é utilizado para criar uma tabela no sistema.
- III - O comando CREATE TABLE cria uma tabela no sistema que não pode ser mais modificada por outro comando.
- IV - Os comandos CREATE TABLE e CREATE SCHEMAS são comandos de controle da linguagem, nunca utilizados por um Administrador de Banco de Dados.

É correto o que se afirma em:

- a) I, apenas.
 - b) III, apenas.
 - c) I, II, III e IV.
 - d) I e II, apenas.
 - e) I, II e III, apenas.
2. O departamento de RH solicitou à equipe de sistemas que fizesse um reajuste do salário em 10% para todos os funcionários da empresa. Baseado nesse contexto, analise os comandos SQL a seguir e assinale a alternativa que contempla a necessidade solicitada.

- I - UPDATE FUNCIONARIOS SET VL_SALARIO = VL_SALARIO * 1.1;
- II - UPDATE FUNCIONARIOS SET VL_SALARIO = 1.1;
- III - UPDATE FUNCIONARIOS ADD VL_SALARIO = VL_SALARIO * 1.1 + 10;
- IV - UPDATE FUNCIONARIOS SET VL_SALARIO + 10;



- a) I, apenas.
- b) II, apenas.
- c) III, apenas.
- d) IV, apenas.
- e) I e III, apenas.

3. SQL é um padrão de banco de dados. Isso decorre da sua simplicidade e facilidade de uso. Ele se diferencia de outras linguagens de consulta a banco de dados, no sentido em que uma consulta SQL especifica a forma do resultado, e não o caminho para chegar a ele. Ela é uma linguagem declarativa em oposição a outras linguagens procedurais. Alguns dos principais comandos SQL para manipulação de dados são: INSERT (inserção), SELECT (consulta), UPDATE (atualização), DELETE (exclusão). SQL possibilita, ainda, a criação de relações entre tabelas e o controle do acesso aos dados. Baseado nessas informações, analise o código a seguir.

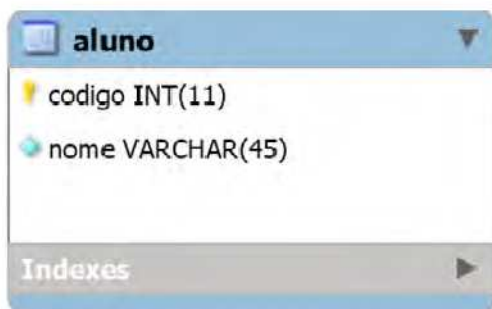
```
<?php
    // conexão está presente na variável $mysqli
    $insert = "INSERT INTO usuario (nome, idade, cidade)
VALUES ('João Paulo', 23, 'São Paulo')";
    $res = ???($insert);
?>
```

De acordo com o exposto anteriormente, assinale a alternativa correta que aponta qual é o método PHP, para \$mysqli, que permite que seja efetuada de maneira eficiente essa consulta INSERT:

- a) \$mysqli->insert(\$insert)
- b) \$mysqli->query(\$insert)
- c) \$mysqli->query_execute(\$insert)
- d) \$mysqli->connect_insert(\$insert)
- e) \$mysqli->connect_execute(\$insert)



4. Em PHP, podemos interagir com o servidor de banco de dados Mysql, e essa interação é realizada por meio de consultas chamadas de *queries*. Essas consultas são escritas na linguagem (*Structured Query Language*).



Fonte: os autores.

Diante do contexto de query e da tabela aluno, analise as afirmativas a seguir, tendo como premissa que a classe mysqli já está instanciada no objeto \$my.

- I - `$my->query("select codigo, nome from aluno");`
- II - `$my->query("insert into aluno (codigo, nome) values ('1','Rafael')");`
- III - `$my->query("update aluno set nome = 'Rafael' where codigo='1'");`
- IV - `$my->query("delete from aluno where codigo='1'");`

É correto afirmar que as consultas válidas são:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, II e III, apenas.
- d) II e III, apenas.
- e) I, II, III e IV.



5. SQL (*Standard Query Language*) — traduzindo, temos Linguagem Padrão de Consulta — é uma linguagem declarativa utilizada em SGDB em bancos relacionais. Essa linguagem é muito utilizada para criarmos *databases*, tabelas, colunas, índices, atribuímos permissões a usuários, bem como realizarmos consultas aos dados armazenados. A linguagem SQL foi organizada em subconjuntos para um melhor entendimento e aplicabilidade.

Dentro do contexto anterior, cite e explique quais são os subconjuntos em que a SQL foi dividida.

6. *Structured Query Language*, ou Linguagem de Consulta Estruturada, ou, ainda, SQL é a linguagem de pesquisa declarativa padrão para banco de dados relacional (base de dados relacional). Muitas das características originais do SQL foram inspiradas na álgebra relacional. A linguagem é um grande padrão de banco de dados. Isso decorre da sua simplicidade e facilidade de uso. Ela se diferencia de outras linguagens de consulta a banco de dados no sentido em que uma consulta SQL especifica a forma do resultado, e não o caminho para chegar a ele.

Tomando como base essas informações, e supondo que exista uma tabela carro com os campos (modelo, marca, cor, ano_fabricacao), todos obrigatórios, crie um script SQL que insira um novo registro nessa tabela com os seguintes dados:

modelo = Golf

marca = Volkswagen

cor = Prata

ano_fabricacao = 2017



UMA FERRAMENTA WEB PARA AUXILIAR O DESENVOLVIMENTO DE RELATÓRIOS EM PDF PARA SISTEMAS EM PHP

Nos últimos anos o desenvolvimento de sistemas para internet tem crescido muito, isso devido ao barateamento dos custos para acesso à internet banda larga, a facilidade de integração destes sistemas com aparelhos móveis (celulares e *smartphones*). De acordo com a pesquisa realizada em 2009 pela agência NIC.br (Núcleo de Informação e Coordenação do Ponto BR) e pela Cetic.br (Centro de Estudos as Tecnologias da Informação e da Comunicação) entre os anos de 2008 e 2009 houve uma taxa de crescimento de 35% no acesso à internet entre os usuários do Brasil [Cetic 2009].

Assim como soluções desenvolvidas para ambientes *desktops*, soluções para a internet necessitam de diversos relatórios, que vão desde simples listagens de clientes até os mais complexos como financeiros, contábeis, gestão integrada, dentre outros.

Desde o surgimento da internet em meados de 1990, evoluiu-se muito em relação às linguagens de programação para esse ambiente, melhorando principalmente em aspectos como a segurança e a GUI (*Graphics User Interface*).

A partir de 1994 o desenvolvedor canadense-dinamarquês Rasmus Lerdorf, começou o desenvolvimento da linguagem de programação PHP (*Hypertext Preprocessor*), para auxiliar no desenvolvimento de sua página pessoal. A liberdade de código fonte aberto e software gratuito do PHP garantida por um grupo de esquemas de licença, denominadas GPL (*General Public License*), colocou o PHP como uma das linguagens mais usadas para desenvolvimento web, tendo grande reconhecimento mundial [Converse and Park 2003].



Objetiva-se com o uso dessa ferramenta aproximar a facilidade no desenvolvimento de relatórios em PDF (*Portable Document Format*) com o PHP de outras linguagens de programação que já possuem em muitos casos um ambiente com recursos baseados em interfaces para a criação de relatórios em PDF (*JReport* para linguagem *Java*, *Crystal Reports* para *C#* no *Visual Studio*, etc). Entretanto, na linguagem de programação PHP uma interface gráfica na geração de relatórios PDF seria muito desejável e de grande valia.

Além do fator despendido para confecção dos relatórios, um dos principais problemas é de que os navegadores utilizam de dois principais posicionamentos de objetos na tela, a posição relativa ou absoluta. Para impressão utiliza-se da posição absoluta, mas esta faz referência conforme a resolução do monitor onde está sendo desenvolvido.

Fonte: Soares, Rosa e Silva (2013, p. 47-48).



eu recomendo!



livro

Desenvolvimento Web com PHP e MySQL

Autor: Evaldo Junior Bento

Editora: Casa do Código

Sinopse: imagine a internet na qual você pode apenas consumir conteúdos, como se fosse um jornal, uma revista ou, ainda, um programa na televisão. Chato, né? O segredo da famosa web 2.0 é a capacidade de interação entre as pessoas e os serviços on-line.

Aprenda, com esse livro, os primeiros passos para usar o PHP, uma ferramenta que possibilita a criação de páginas web dinâmicas. Veja, também, como integrar o PHP ao MySQL, um dos bancos de dados mais usados no mundo. A dupla PHP e MySQL é usada por diversos tipos de aplicações e sites, de blogs a portais de notícias e grandes redes sociais.



livro

Construindo Aplicações Web com PHP e MySQL

Autor: André Milani

Editora: Novatec

Sinopse: a construção de sites e aplicações cresce de forma acelerada na internet, e, por trás de grande parte desses projetos, o PHP e o MySQL são utilizados por serem tecnologias muito eficientes e terem sido criados visando este nicho de mercado: aplicações web (intranet e internet). Com a capacidade de criar códigos de forma simples e rápida, o PHP é uma linguagem de programação extremamente funcional, com recursos voltados para web e em constante evolução. Por ser uma das tecnologias mais utilizadas em aplicações de internet, diversas bibliotecas e módulos são criados e disponibilizados para uso de programadores todos os dias. Além disso, há outro fator importante: é gratuito e, ao ser integrado com o MySQL, gera um ambiente perfeito e completo para o desenvolvimento de aplicações.





eu recomendo!



livro

PHP e MySQL — Desenvolvimento Web (3. ed.)

Autor: Laura Thomson e Luke Welling

Editora: Campus

Sinopse: PHP é uma linguagem script de programação disponível de graça sob licença open source e utilizada, principalmente, em servidores web Linux como alternativa ao ASP da Microsoft. MySQL é um sistema open source de gerenciamento de banco de dados relacional que utiliza SQL, a linguagem mais usada para adicionar, acessar e processar dados em um banco de dados. Por ser de código aberto, MySQL é grátis e pode ser adaptado de acordo com as necessidades do desenvolvedor, além de ser mais rápido, estável e flexível. PHP e MySQL — Desenvolvimento Web é um guia muito respeitado no mercado, e sua segunda edição, publicada pela Campus, em abril de 2003, foi um dos livros mais vendidos de 2003. A terceira edição vem com atualizações para a versão 5.0 do PHP e, certamente, será tão bem-sucedida quanto a sua edição anterior.





MODELOS DE APLICAÇÃO DO BACK END

PROFESSORES

Me. Rafael Alves Florindo

Esp. Carlos Danilo Luz

PLANO DE ESTUDO ▼

A seguir, apresentam-se as aulas que você estudará nesta unidade: • Conceitos de MVC • Frameworks • Sistema de Versionamento • Webservice e API RESTful • Princípios de Docker.

OBJETIVOS DE APRENDIZAGEM ▼

Conceituar e apresentar ao aluno os conceitos de três camadas no desenvolvimento de sistemas Web • Conhecer alguns frameworks de programação WEB que auxiliam e agilizam no desenvolvimento da aplicação • Conceituar e apresentar ao aluno um sistema de versionamento utilizado no mercado de desenvolvimento de software • Conhecer os conceitos e aplicações de um Webservice e o seu consumo por meio de uma API RESTful • Conceituar os princípios de Container.



Bem-vindo(a) à última unidade do nosso material. Conforme já conversamos em momentos anteriores, o desenvolvimento de um sistema não está limitado apenas à codificação, sendo possível utilizar diversos recursos e processos para se obter um bom sistema.

Nesta unidade, estudaremos o processo de desenvolvimento baseado em Model View Controller – MVC, em que é possível fazer uma codificação do sistema organizada, separando o sistema em camadas de interação. Ao aplicar o conceito de MVC, pode-se reutilizar a codificação desenvolvida em diversas partes do sistema, pois, uma vez criadas as funções, estas podem ser facilmente replicadas em trechos da implementação do sistema.

Atualmente, para o(a) desenvolvedor(a) moderno(a), é possível contar com diversas ferramentas que auxiliam e aceleram o processo de desenvolvimento de uma sistema, sendo ele WEB, MOBILE ou Desktop. Estamos falando dos frameworks, que podem ser IDEs de desenvolvimento ou códigos comuns que podem ser utilizados entre vários projetos de software, fornecendo funcionalidades genéricas que podem ser facilmente utilizadas. No entanto, não se engane em pensar que, ao usar um framework, você não precisa conhecer a codificação, tampouco saber programar; o uso dele é aplicado para ajudar, não para implementar por você. Assim, conheceremos alguns dos frameworks que são destaques no mercado de trabalho.

Outro aspecto relevante é saber como construir e consumir recursos de fácil leitura ao utilizar serviços como WebService e API RESTFul, visto que este é um recurso muito utilizado por grandes empresas de ERP, sendo possível interagir com sistemas em poucas trocas de requisições aos servidores.

Por fim, analisaremos o conceito de versionamento. Raramente, teremos um única versão de desenvolvimento, é natural que um sistema possua várias versões de seus módulos e sistema, por isso, conhecer um sistema de gerenciamento e armazenado de versões é um fator muito relevante. Compreenderemos, também, como são a arquitetura e os princípios de Docker na implementação e gestão do desenvolvimento e conteúdo do sistema.

O que estamos esperando? Vamos lá!

1 CONCEITOS DE MVC

Nesta aula, trabalharemos com um conceito de desenvolvimento em camadas, o MVC. Durante o desenvolvimento, é necessário que diversos passos sejam executados para que ele possa ter uma qualidade eficaz, ou seja, é necessário que se tenha um roteiro que o software precisa seguir de acordo com suas necessidades, e este passo é chamado de Processo de Software (PRESSMAN; MAXIM, 2016).

Para Sommerville (2011, p. 18), existem diversos processos de software diferentes, mas todos devem incluir quatro atividades que são consideradas fundamentais para a Engenharia de Software:



1. Especificação de software. A funcionalidade do software e as restrições a seu funcionamento devem ser definidas.
2. Projeto e implementação de software. O software deve ser produzido para atender às especificações.
3. Validação de software. O software deve ser validado para garantir que atenda às demandas do cliente.
4. Evolução de software. O software deve evoluir para atender às necessidades de mudança dos clientes

Contudo, Pressman e Maxim (2016) afirmam que, para que o desenvolvimento de software possa alcançar o resultado final, é necessário que se tenha uma arquitetura de software condizente com o processo. Algumas delas são: arquitetura centralizada em dados; arquitetura de fluxo de dados; arquitetura de chamada e retorno; arquitetura orientada a objetos; arquitetura em camadas; e arquitetura MVC.



quadro-resumo

De acordo com Pressman e Maxim (2016, p. 230):

A arquitetura de software de um programa ou sistema computacional é a estrutura ou estruturas do sistema, que abrange os componentes de software, as propriedades externamente visíveis desses componentes e as relações entre eles.

Dentre as arquiteturas citadas anteriormente, aprofundaremos nossos conhecimentos na arquitetura MVC, sendo esta a mais utilizada para o desenvolvimento de sistemas Web. Esta, por sua vez, tem por objetivo auxiliar os desenvolvedores a construir aplicações, separando seus principais componentes. Essa separação ocorre com a manipulação e armazenamento dos dados (*Model*), as funções que trabalharão com as entradas dos dados (*Controller*) e a visualização do usuário (*View*). Estudaremos cada uma dessas camadas ou componentes separadamente.

A **camada Model** contém toda a comunicação com os dados armazenados que serão visualizados na camada *View*, que podem estar armazenados em um banco de dados, em um JSON, em um arquivo XML, em um arquivo, entre outros locais. Toda a manipulação de dados acontece em um SGBD (comandos de manipulação: *create*, *reader*, *update* e *delete*), o famoso (CRUD), com um banco de dados relacional, ou não deve ocorrer nesta camada, ou seja, é ela que conterá um maior nível de abstração do sistema.

A **camada View** trabalha apenas com a apresentação dos dados da aplicação, passando ao usuário final uma confiança de que tudo está se realizando no seu terminal. É nesta camada que se tem os *frameworks* de *front-end*, *templates*, entre outros.

Enquanto isso, a **camada Controller** é responsável por administrar todo o fluxo da aplicação. Nesta camada, é onde ocorre toda a movimentação da aplicação, ou seja, a partir da entrada de uma *View*, ela resolve qual operação utilizará da camada *Model* e, assim, devolverá a sua *View* correspondente junto aos dados.

Confira, na figura a seguir, o fluxo de dados da arquitetura MVC.

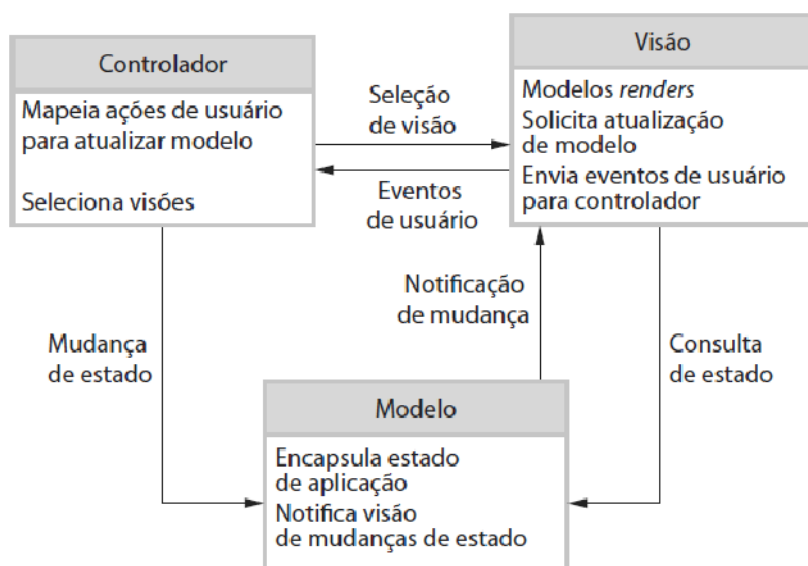


Figura 1 - A organização do MVC / Fonte: Sommerville (2011, p. 109).

De forma análoga, a aplicação do MVC para WEB não é tão diferente, altera-se apenas o tipo de fluxo, como podemos perceber na Figura 2.

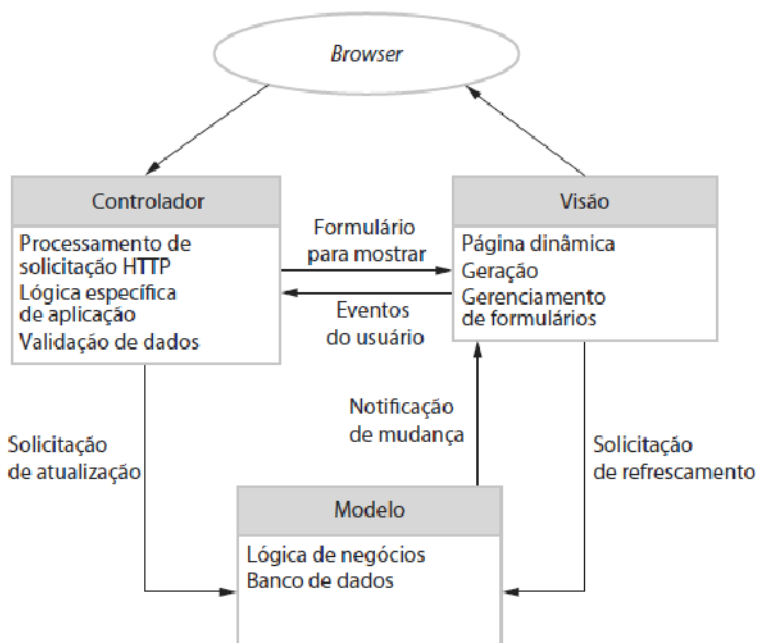


Figura 2 - Arquitetura de aplicações Web usando o padrão MVC
Fonte: Sommerville (2011, p. 110).

Como em toda arquitetura, o MVC possui algumas vantagens e desvantagens. Veja, no quadro a seguir, alguns apontamentos:

VANTAGENS	DESVANTAGENS
As camadas podem ser reutilizadas em outros sistemas ou substituídas, promovendo, assim, a reusabilidade.	Número maior de classes existentes no sistema.
Princípio baixo acoplamento: caso a interface entre as camadas seja mantida, a implementação de uma camada pode ser mudada sem afetar outra camada.	Aplicações simples podem se tornar extremamente complexas caso haja um exagero na criação de camadas.
Isolar as regras de negócios da lógica de apresentação, a interface com o usuário.	Alterar um nome de campo ou de tabela implicará diversas partes da sua aplicação.
Manutenção do sistema se torna mais fácil.	Requer um tempo maior para modelar o sistema.
Permite o desenvolvimento, os testes e a manutenção de forma isolada entre as camadas.	Exige muita disciplina dos desenvolvedores em relação à separação das camadas.
O projeto passa a ter uma melhor organização em sua arquitetura.	

Quadro 1 - Vantagens e desvantagens da arquitetura MVC / Fonte: os autores.

Não há uma receita de bolo que indique a melhor arquitetura a se utilizar, pois sempre que um sistema for desenvolvido, terá, com certeza, a necessidade de manutenção, como melhorias ou correção de erros, e, também, pode necessitar de várias atualizações. Assim, o MVC vem a calhar com a manutenção exclusiva das camadas da aplicação.



2 FRAMEWORKS

A partir do que foi estudado na aula anterior sobre o MVC, é necessário que se mantenha uma qualidade de desenvolvimento, possibilitando um benefício para o resultado final. Nesse viés, o *framework* consiste em uma boa prática de desenvolvimento, pois, para desenvolver uma aplicação — seja ela web, desktop ou mobile — completamente do zero, tem-se uma tarefa árdua que requererá mais tempo de trabalho.

Para Sommerville (2011, p. 299), um *framework* de aplicações é considerado “coleções de classes abstratas e concretas [que] são adaptadas e estendidas para criar sistemas de aplicação”. Em outras palavras, podemos entender que os *frameworks* funcionam como uma biblioteca de arquivos que armazena diversas funções básicas para o desenvolvimento de uma aplicação.

Os *frameworks*, para o desenvolvimento, diminuem o tempo gasto e o nível de complexidade por não ser necessário reinventar códigos, pois eles já implementam diversas funções de forma automática. Cada linguagem de programação tem os seus respectivos *frameworks*, que podem ser utilizados, dependendo do objetivo do seu sistema, da equipe, do tempo de desenvolvimento, da manutenibilidade, entre outros.

Contudo, como toda boa tecnologia, temos algumas vantagens e desvantagens em um *framework* para desenvolvimento WEB. Confira:

VANTAGENS	DESVANTAGENS
Aproveitamento e reutilização de código.	Baixa compatibilidade com outro framework.
Padrões de codificação e processos.	Devido a diversos módulos prontos, alguns desenvolvedores acabam por subestimar o seu conhecimento na linguagem de desenvolvimento do framework.
Segurança.	Curva de aprendizagem e domínio do framework é baixa devido à complexidade do desenvolvimento.
Produtividade.	
Rede de apoio de outros desenvolvedores (suporte).	

Quadro 2 - Vantagens e desvantagens em trabalhar com *frameworks* / Fonte: os autores.

Nesta aula, focaremos nos *frameworks* de desenvolvimento WEB no *front-end* e no *back-end*.

Frameworks de desenvolvimento WEB no front-end

Sabemos que o *front-end* é quem atua nas interfaces, mostrando o que o usuário solicitou e gerenciando as telas que são apresentadas. Nesse contexto, os *frameworks front-end* servem como ferramentas que auxiliam na criação de interfaces e na forma como os dados são exibidos.

Atualmente, temos à disposição diversos *frameworks*, assim falaremos, a seguir, sobre alguns dos mais utilizados no mercado de trabalho.

- **Angular:** esse é um *framework* JavaScript que implementa a arquitetura MVC e é de código aberto. Mantido pela Google, ele é ideal para criar aplicativos web avançados de uma única página, mais conhecidos como SPA (sigla para *Single Page Applications*, ou Aplicações de Página Única). Com o *Angular*, podemos desenvolver aplicações web voltadas tanto para resoluções desktop quanto para resoluções *mobile*, tornando-as dinâmicas, modernas e escaláveis (ANGULAR, [2021], on-line)¹.

- **React:** trata-se de uma biblioteca JavaScript para criar interfaces de usuário para se utilizar no *front-end*, embora possa ser considerada uma estrutura por implementar conceitos de componentes. *React* foi o primeiro *framework* a usar arquitetura baseada em componentes, sendo o seu maior diferencial a rapidez do uso e a facilidade para aprender. Esse *framework* foi desenvolvido e é mantido pelo Facebook. Como a lógica do componente é escrita em JavaScript e não em *templates*, você pode facilmente passar diversos tipos de dados ao longo da sua aplicação e ainda manter o estado fora do DOM (REACT, [2021], on-line)².
- **React Native:** é um *framework* baseado no *React JS*, desenvolvido pela equipe do Facebook, que possibilita o desenvolvimento de aplicações *mobile* tanto para Android como para iOS, utilizando apenas JavaScript. No *React Native*, todo código desenvolvido deve ser convertido para a linguagem nativa do sistema operacional, o que torna o *app* muito mais fluido.
- **Vue:** é um *framework* progressivo para a construção de interfaces de usuário. Ao contrário de outros *frameworks* monolíticos, *Vue* foi projetado desde a sua concepção para ser adotável incrementalmente. A biblioteca principal é focada exclusivamente na camada visual (*view layer*), sendo fácil adotar e integrar com outras bibliotecas ou projetos existentes. Por outro lado, *Vue* também é perfeitamente capaz de dar poder a sofisticadas *Single Page Applications* quando usado em conjunto com ferramentas modernas e bibliotecas de apoio (VUE.JS, [2021], on-line)³.
- **Bootstrap:** é uma *framework* gratuito para desenvolvimento HTML, CSS e JS com a tecnologia *mobile* (responsivo), desenvolvido pelo Twitter em 2010. O *Bootstrap* possui uma diversidade de componentes (*plug-ins*) em JavaScript (jQuery) que auxiliam o designer a implementar *tooltip*, *menu-dropdown*, *modal*, *carousel*, *slideshow*, entre outros, sem a menor dificuldade, apenas acrescentando algumas configurações ao código, sem a necessidade de criar scripts e mais scripts. *Bootstrap* é um *framework* CSS utilizado em aplicações *front-end*, ou seja, na camada de interface com o usuário para o desenvolvimento de aplicações adaptáveis à tela de qualquer dispositivo.

Frameworks de desenvolvimento WEB no *back-end*

O *back-end* de uma aplicação trabalha com a toda a lógica do sistema, seja no controle de rotas, cálculos ou comunicação das informações trocadas entre banco de dados e navegador. Ele é considerado o centro de processamento, uma vez que se relaciona diretamente com o *front-end*. O *back-end* não é visto pelo usuário externo. O uso de um *framework* voltado para o *back-end* facilita e agiliza o desenvolvimento da aplicação.

A seguir, conversaremos um pouco sobre alguns dos que são usados no mercado de trabalho.

- **Laravel:** é um *framework* PHP utilizado para o desenvolvimento web e faz uso da arquitetura MVC. Ele tem como principal característica desenvolver aplicações seguras e performáticas de forma rápida, com código limpo e simples. O *Laravel* faz uso da arquitetura de desenvolvimento MVC (*Model, View, Controller*) e do padrão de desenvolvimento PSR-2, que tem como foco a escrita de código limpo e com boa legibilidade. Além disso, o *framework* tem como objetivo aumentar a velocidade de codificação, sem esquecer características importantes, como a segurança e performance da aplicação. O *Laravel* utiliza o *Composer* para gerenciar suas dependências. O *Composer* é um gerenciador de dependências, ou seja, uma ferramenta que permite gerenciar de forma fácil os pacotes de terceiros da sua aplicação. O *Laravel* possui um sistema de *template* que facilita a criação da camada de visualização de dados (páginas HTML).
- **Zend PHP:** é um *framework* orientado a objetos, baseado em MVC, que permite que você carregue apenas componentes que deseja como bibliotecas individuais. Em outras palavras, o Zend permite que você foque apenas nos componentes e funções que você deseja e ignore todo o resto. Ele segue os padrões PHP-FIG e foca nos critérios de segurança, como o armazenamento de senhas, usando o recurso de *bcrypt* ou gerando criptografia usando AES. O Zend também é a mantenedora da linguagem PHP.

Existem outros frameworks php, tais como: cakePhp, Symfony, Phalcon, CodeIgniter, Sline, entre outros.

3 SISTEMA DE VERSIONAMENTO

Um sistema de controle de versão tem a finalidade de gerenciar diferentes versões de um documento, seja para desenvolvimento, documentação jurídica, entre outros. O sistema de versionamento registra as alterações em um arquivo ou conjunto de arquivos ao longo do tempo para que você possa lembrar versões específicas em um momento posterior. Desta forma, esse sistema tem como premissa tornar mais inteligente e eficaz a organização de seu projeto, pois ele possibilita um acompanhamento do histórico de escrita, alteração e exclusão quanto ao desenvolvimento. Ele, ainda, permite que o desenvolvimento seja realizado paralelamente, sem mexer no projeto principal ou resgatar o sistema em um ponto que estava estável.

Normalmente, os sistemas de versionamentos são utilizados por desenvolvedores de software, mas eles podem ser utilizados para diversas áreas afins. Os arquivos do projeto ficam armazenados em um repositório, e o histórico de suas versões é salvo nele. Atualmente, existem diversos sistemas de versionamento no mercado, tais como: CVS, Darcs, GIT, Mercurial, Microsoft Visual SourceSafe, RedMine, TFS, entre outros. Cada sistema possui vantagens e desvantagens sobre o outro, assim cabe à equipe decidir qual sistema utilizará, mediante a estrutura e necessidade.

Nesta aula, abordaremos o GIT em conjunto com o GitHub da atual Microsoft. Começaremos pelo GitHub.

O GitHub é um serviço web para hospedar seus projetos pessoais, projetos de software, códigos-fonte, entre outros e oferece diversas funcionalidades que

são aplicadas ao GIT. Para desfrutar dos recursos do GIT, é necessário ter uma conta no GitHub e criar um repositório para cada projeto. Para acessar o GitHub, entre no link <https://github.com/> e crie uma conta gratuita. Talvez, seja necessária a ativação de conta por e-mail, assim sugere-se que fique atento à criação de sua conta e confira seu e-mail (caixa de entrada e spam). Com a conta criada e ativada, você poderá criar seus repositórios ou pesquisá-los. Confira a Figura 3 a seguir.

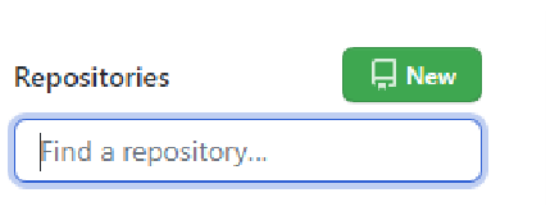


Figura 3 - Criação de novos repositórios ou pesquisas de repositórios / Fonte: os autores.

Após clicar em New, será aberta uma nova página, conforme a Figura 4, em que será necessário introduzir algumas informações, conforme apresentado a seguir:

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere?
[Import a repository.](#)

Repository template

Start your repository with a template repository's contents.

No template ▾

Owner *

 rafaelflorindo ▾

Repository name *

/

Great repository names are short and memorable. Need inspiration? How about *literate-couscous*?

Description (optional)

☒  **Public**

Anyone on the internet can see this repository. You choose who can commit.

☐  **Private**

You choose who can see and commit to this repository.

Figura 4 - Criando repositórios / Fonte: os autores.

- A primeira é o nome do repositório. Ao informá-lo, será realizada uma validação para verificar se já não existe um repositório com o mesmo nome informado.

- Após validar o nome do repositório, você pode colocar uma descrição condizente para ele, caso não coloque a descrição na hora, pode fazê-la posteriormente.
- Agora, é necessário escolher a visibilidade do seu repositório. Normalmente, a maioria dos repositórios são público (*public*), pois, antes da compra da Microsoft, os repositórios privados (*private*) tinham um custo de, aproximadamente, nove dólares. Contudo, com a compra pela Microsoft, podemos criar repositórios tanto privados quanto públicos e ter um limite de espaço e horas por mês. Caso haja necessidade de mais, você deve comprar junto à Microsoft. Vale lembrar que, como esse serviço é de uma empresa com fins lucrativos, esses benefícios podem sofrer alterações, assim fique atento ao criar e manter seus repositórios.

Para finalizar a criação do seu repositório, você pode adicionar algumas configurações iniciais importantes, tais como: Arquivo de leitura (Readme file). arquivos que serão ignorados no pull e no push e licenças sobre o seu repositório. Todas essas configurações podem ser incluídas posteriormente, contudo sugiro que marque o campo Add à README file e preencha a tela que será aberta com alguma informação sobre o seu projeto para que já ative o seu repositório, depois, clique no botão Create repository.

Initialize this repository with:

Skip this step if you're importing an existing repository.

☐ **Add a README file**

This is where you can write a long description for your project. [Learn more.](#)

☐ **Add .gitignore**

Choose which files not to track from a list of templates. [Learn more.](#)

☐ **Choose a license**

A license tells others what they can and can't do with your code. [Learn more.](#)

Create repository

Figura 5 - Configurações extras do repositório / Fonte: os autores.

Após a criação de uma conta no GitHub e a criação de um repositório, podemos trabalhar com o sistema de versionamento do GitHub com o GIT. Para o livro da disciplina de *Back End II*, criou-se o GIT com o nome (LivroBackEndII). Para acessá-lo, prossiga com o link: <https://github.com/rafaelflorindo/LivroBackEndII>.

Agora, estudaremos o GIT e como utilizá-lo. O **GIT** é um sistema de controle de versão de arquivos *open-source*, desenvolvido por Linus Torvald — isto mesmo: o criador do Sistema Operacional Linux. O principal foco do GIT é possibilitar que o time de desenvolvimento possa desenvolver seus códigos com segurança, sem se preocupar em versionar os seus códigos de forma manual (CTRL + C e CTRL + V). Ele criará um histórico de alterações dos arquivos de acordo com a sua chegada. Assim, poderemos saber quais foram as alterações realizadas, quem fez cada uma dessas alterações e, ainda, baixá-las e, se necessário, revertê-las para uma versão anterior. Vejamos alguns conceitos importantes sobre o GIT.

- **Repositório:** é o local físico onde os arquivos e suas cópias ficarão armazenados.
- **Branch:** é conhecido como ramos, ou seja, cópias originais dos códigos e que podem ser manipuladas de forma livre, sem afetar as funcionalidades em produção no código-fonte principal.
- **Merge:** sua função é fundir/unir a cópia *branch* de seus arquivos modificados com o ramo principal do projeto.
- **Push Request:** realiza o envio das modificações após o *Merge* para o repositório central. Caso não o faça, a equipe continuará trabalhando com cópias desatualizadas, e isso pode ocasionar problemas críticos em seus códigos.
- **Pull Request:** é utilizado para puxar (baixar) as modificações realizadas para a sua máquina, fundindo a nova versão com o seu código local.
- **Fork:** cria uma cópia fiel do repositório remoto para a máquina local, e te deixa conectado com o repositório remoto. Após as modificações, as devolve para o repositório original, contudo a efetivação dessas mudanças dependerá de autorização do responsável pelo repositório.

Para utilizar o GIT, é necessário tê-lo instalado em seu sistema operacional, para isso, faça o download em <https://git-scm.com/downloads>, escolha qual é o seu sistema operacional e proceda com a instalação. Os exemplos que serão apresentados terão como base o sistema operacional Windows.

Após instalar o GIT e ter um repositório criado no GitHub, crie um diretório no servidor apache de seu computador (C:\xampp\htdocs) com o nome do seu repositório. Para abrir um terminal GIT no Windows, clique com o botão direito do mouse no diretório criado e escolha a opção `Git Bash Here`, conforme Figura 6.

O terminal do GIT será aberto conforme Figura 7. Lembro que, para cada sistema operacional, a exposição da tela pode ser diferente. Notem que o nosso diretório criado é o `Livro-BackEndII`.

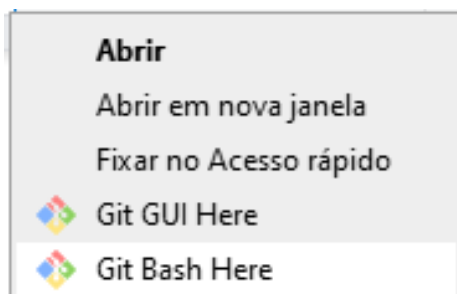


Figura 6 - Git Bash Here / Fonte: os autores.

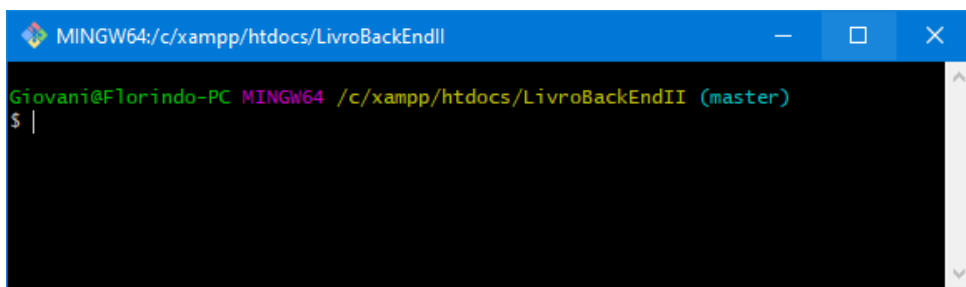


Figura 7 - Git Bash Here / Fonte: os autores.

Agora, realizaremos a configuração inicial. Ela é de fundamental importância, pois é por meio dela que se diferenciam os `commit` no serviço de versionamento. Essa configuração só será executada uma vez em cada terminal que tenha instalado o GIT. Para configurar, digite os comandos a seguir após \$.

```
$ git config --global user.name "Fulano de Tal"
$ git config --global user.email fulanodetal@exemplo.br
```

Agora, com a identidade criada, inicializaremos o repositório em um diretório existente. Os códigos a seguir devem estar dentro do diretório criado, e este deve ter o mesmo nome do repositório.

```
$ git init
```

Essa ação cria um novo subdiretório chamado `.git`, que contém todos os arquivos necessários de seu repositório. Agora, realizaremos um clone do seu repositório para esse diretório que você criou. Para isso, volte no GitHub e acesse o seu repositório, após, clique no botão Code e copie o primeiro link. Confira a Figura 8.

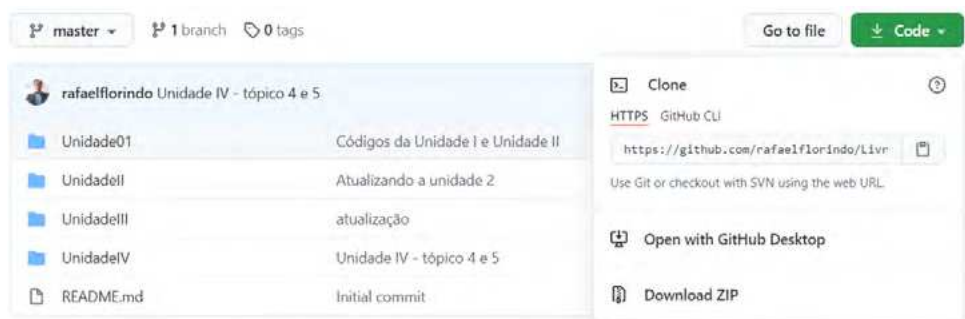


Figura 8 - Clonando o repositório do GitHub / Fonte: os autores.

Agora, volte no terminal do GIT e digite o comando seguido do link copiado:

```
$ git clone https://github.com/rafaelflorindo/Livro-BackEndII.git
```

Para começar a controlar as alterações, devemos executar os comandos a seguir toda vez que deseja atualizar o repositório. O primeiro comando adiciona todos os arquivos no pacote chamado de *staging* de arquivos desde a raiz do repositório, passando por todos os subdiretórios, e, aqui, está a diferença, não importa se você está na raiz ou no subdiretório.

```
$ git add --all
```

O segundo comando é o *commit*, que capturará um instantâneo das mudanças preparadas do projeto no momento. Podemos incluir uma mensagem de commit transmitida pelo valor `-m 'texto da mensagem'`.

```
$ git commit -m 'initial project version'
```

Após ter preparado o commit, está na hora de enviar o *staging* para o repositório remoto no GitHub. Para isso, digite o comando a seguir:

```
$ git push origin master
```

Agora, volte no GitHub e confira se os arquivos que foram versionados estão no repositório.

WEBSERVICE E API RESTFUL

Para entender o funcionamento e a necessidade de um *WebService*, imagine dois sistemas totalmente diferentes e que necessitem, de alguma forma, trocar informações constantemente. Essas informações, normalmente, são coletadas por um usuário e introduzidas em um outro sistema, ou, ainda, podem ser manipuladas e exportadas em arquivos como: JSON, XML, entre outros.. Nesse processo, podem ocorrer atrasos, perdas de informações, deixando a aplicação suscetível a erros.



quadro-resumo

O XML (Extensible Markup Language) é uma linguagem de marcação criada pelo W3C, e ela oferece uma certa padronização, flexibilidade e a possibilidade de descrever classes com diversos tipos de dados, reduzindo significativamente os custos de desenvolvimento (XML, W3C).

Analizando a situação anterior, temos um problema que pode ser resolvido com a aplicação de um *Web Service*. Ele é um conjunto de métodos que são acessados por outros programas ao utilizar tecnologias Web. Dessa forma, quando for necessário trocar informações entre os sistemas, basta executar o *Web Service* ou, ainda, deixar programada a troca de informações. Essa troca de informação é muito útil, pois vem ao encontro da necessidade de se ter uma interoperabilidade entre sistemas (W3C, [2021]).

Como o *Web Service* é uma tecnologia Web, logo deve ser desenvolvido utilizando-se de tecnologias de desenvolvimento Web, tais como: PHP, Java, Nodejs, entre outros. Para transferir dados entre as aplicações, são utilizados protocolos de comunicação WEB (http) (JOSUTTIS, 2007).

Além de trocas informação, um *Web Service* pode incluir novas funcionalidades na aplicação sem ser necessário realizar manutenções na aplicação. Esta ação é muito importante quando se tem um software legado e que necessite incluir uma nova funcionalidade, uma vez que já não é tão trivial desenvolver para essas aplicações de forma simples e rápida.

Agora, vejamos como funciona um *Web Service*. A aplicação A solicita uma operação no *WebService*, ele efetua o processamento e envia os dados para a aplicação que requereu a operação por meio de protocolos web. A aplicação recebe os dados e faz a sua interpretação, convertendo-os para a sua linguagem própria.

Normalmente, esses sistemas *WebService* foram desenvolvidos em linguagens de programações distintas, assim será necessária uma linguagem intermediária que garanta a comunicação entre a linguagem do *Web Service* e o sistema que faz o pedido ao *Web Service*.

No mercado, já existem protocolos de comunicação como o SOAP (*Simple Object Access Protocol*) e o REST (*Representational State Transfer*) (JOSUTTIS, 2007). Vejamos esses protocolos a seguir.

Protocolo REST

O REST (*Representational State Transfer*) é um protocolo de comunicação que se baseia no protocolo HTTP, ou seja, transmite os dados diretamente via protocolo HTTP por meio da Web Semântica. Permite, também, utilizar vários formatos para representação de dados, como JSON, XML, RSS, entre outros, permitindo uma flexibilidade e não limitando os formatos de representação de dados (W3C).

Protocolo SOAP

O protocolo SOAP (*Simple Object Access Protocol*) utiliza a linguagem XML para enviar mensagens e, geralmente, serve-se do protocolo HTTP para transportar os dados. Esse protocolo possibilita a passagem de comandos e parâmetros entre as entidades *Requester* e *Provider*, respectivamente, requerente e provedora (JOSUTTIS, 2007).

Associado ao protocolo SOAP, está o documento WSDL (*Web Service Definition Language*), que é uma linguagem de metadados utilizada nesses documentos que descreve a localização do *Web Service* e as operações de que dispõe. Um documento WSDL define um XML para descrever o serviço Web entre a interface de um serviço e as operações suportadas. Esse XML especificará os formatos de dados e protocolos específicos que serão utilizados (JOSUTTIS, 2007).

Além do WSDL, o SOAP necessita do UDDI (*Universal Description, Discovery, and Integration*), que é um diretório de serviço em que os sistemas podem registrar e procurar por serviços, ou seja, é um *framework* para disponibilizar, utilizar e pesquisar por serviços na internet (JOSUTTIS, 2007). Confira o fluxo do SOAP na Figura 9 a seguir.

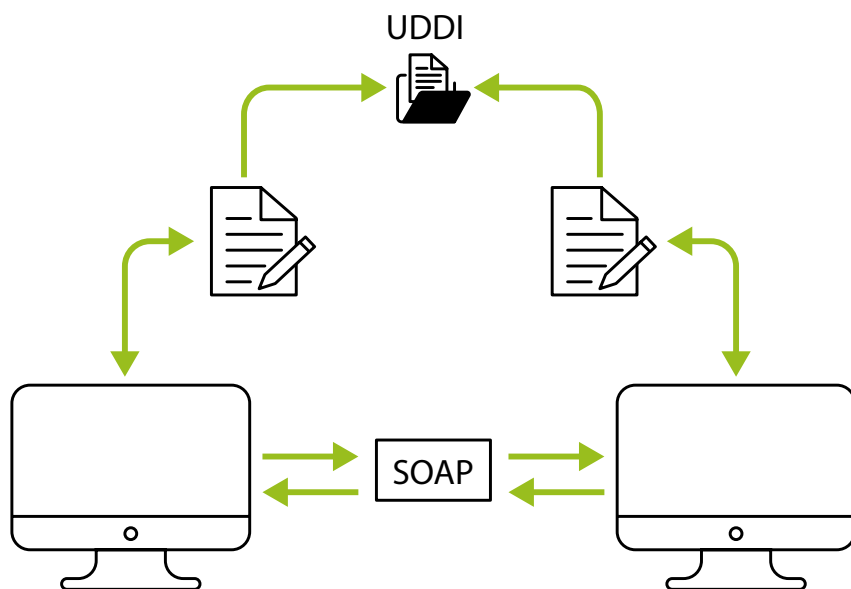


Figura 9 - Fluxo do SOAP / Fonte: os autores.

A partir desses conceitos de *Web Service*, podemos entender melhor os conceitos de *API RestFull*. Este termo une dois conceitos que já trabalhamos no decorrer do livro, *API* e *Rest*. Apenas relembrando, uma *API* são os conjuntos de instruções produzidas em uma linguagem de programação — que, no nosso caso, até aqui, foi o PHP — e que servem para fornecer dados e informações relevantes de uma determinada aplicação. O *Rest* trabalha com web semântica, por meio de metadados, e tem linguagem própria.

Vamos a um exemplo de aplicação de uma *API*. Quando você vai logar em um serviço na Web ou realizar algum cadastro, e ele te dá a possibilidade de se logar com os serviços do Gmail, do Facebook ou outro serviço, provavelmente, estamos utilizando uma *API RestFull*. Nesse caso, a *API* fica esperando a solicitação *request*, realiza o processamento dos metadados recebidos e realiza a devolução das informações por meio de um *Json* ou *XML* e atualiza a tela do solicitante.

PRINCÍPIOS DE DOCKER

Olá, aluno(a)! Abordaremos um conceito que vem, ao longo dos anos, concretizando-se no setor de desenvolvimento, o *Docker*. Ele é uma plataforma de código-fonte aberto, criado pela própria Docker.Inc em 2013, e foi desenvolvido na linguagem *Go* da Google. O *Docker* possui um alto desempenho em relação aos servidores de apli-

cações Web atuais. Essa plataforma garante ao desenvolvedor/cliente maior facilidade na criação e administração de ambientes isolados. Isso mesmo: ambiente isolado, dessa forma, cada aplicação tem todos os recursos para si, chamado de *container* (DOCKER, [2021]).

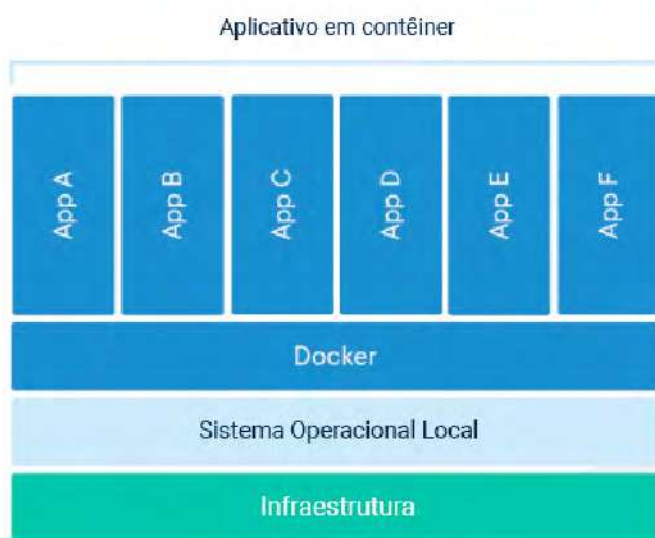


Figura 10 - Aplicações em *containers*
Fonte: adaptada de Docker ([2021], on-line).

Docker é uma virtualização de contêiner tecnologia. Então, é como uma máquina virtual muito leve [VM]. Além de construir contêineres, oferecemos o que chamamos de fluxo de trabalho de desenvolvedor, que é realmente sobre ajudar as pessoas a construir contêineres e aplicativos dentro de recipientes e, em seguida, compartilhá-los entre seus companheiros de equipe (ANDERSON, 2015).

Você deve estar se perguntando: mas esse conceito se parece com uma máquina virtual? A resposta é: são diferentes. A máquina virtual (VM) funciona como uma abstração do hardware físico que transforma um servidor em vários servidores. O hipervisor permite que várias VMs sejam executadas em uma única máquina. Cada máquina virtual inclui uma cópia completa de um sistema operacional, o aplicativo, binários e bibliotecas necessários — ocupando dezenas de GB.

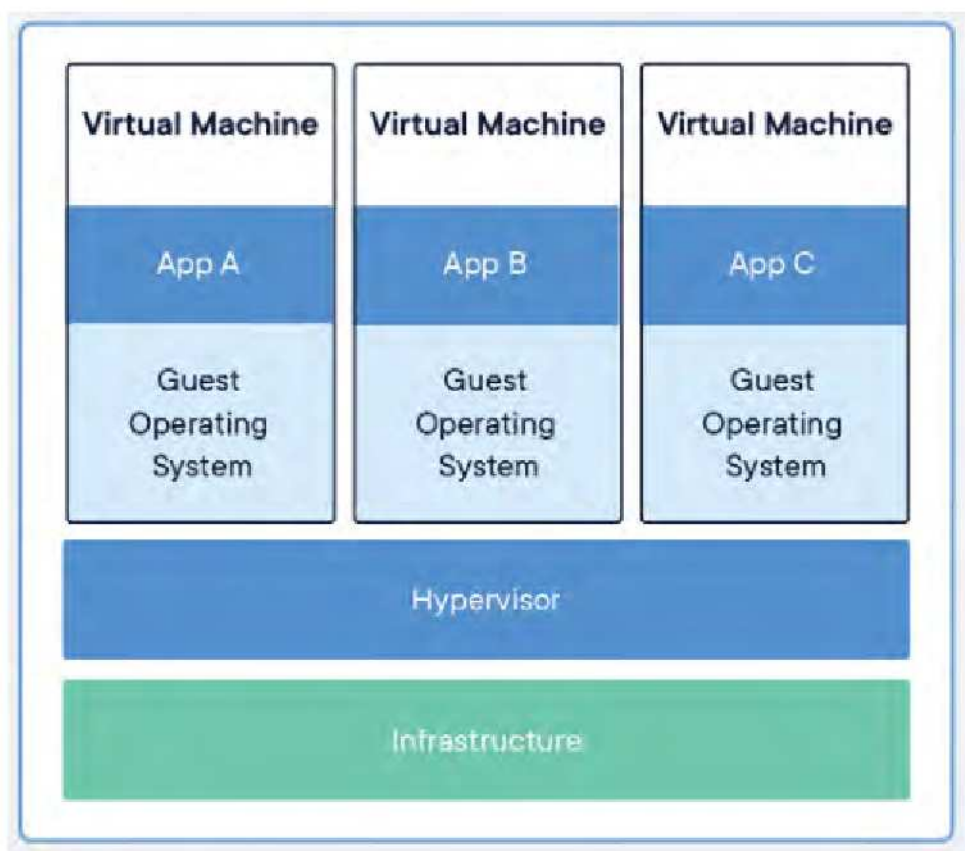


Figura 11 - Máquina virtual / Fonte: adaptada de Docker ([2021], on-line).

O contêiner empacota todos os códigos-fonte, bem como suas dependências de forma padronizada em imagens. Isso ocorre porque a plataforma disponibiliza funções básicas para a sua execução, como código, bibliotecas, *runtime* e ferramentas do sistema por meio de imagens de *docker*. As imagens de contêiner se tornam contêineres no tempo de execução no *Docker Engine*.

Docker é uma plataforma aberta para desenvolvedores e administradores de sistema construírem, enviarem e executarem aplicativos distribuídos usando o *Docker Engine*, um tempo de execução leve e portátil e uma ferramenta de empacotamento, e o *Docker Hub*, um serviço de nuvem para compartilhar aplicativos e automatizar fluxos de trabalho (PREETH, 2015).

Docker criou o padrão da indústria para contêineres para que eles pudessem ser transportados por qualquer lugar. Os contêineres compartilham o *kernel* do sistema operacional e, portanto, não exigem um sistema operacional por aplicativo, gerando maior eficiência do servidor e reduzindo os custos do servidor e de licenciamento. Os aplicativos são mais seguros em contêineres, e o *Docker* fornece os recursos de isolamento padrão mais fortes do setor.

Atualmente, o *Docker* está disponível para aplicativos baseados em Linux e Windows. O software em contêineres rodará da mesma forma, independentemente da infraestrutura. Os contêineres isolam o software de seu ambiente e garantem que ele funcione uniformemente, apesar das diferenças, por exemplo, entre o desenvolvimento e a preparação. *Docker* está, atualmente, liderando o mercado. Mais do que uma solução de *container*, é uma ferramenta completa de empacotamento e entrega de software.

CONSIDERAÇÕES FINAIS

É com um sentimento de alegria e tristeza que chegamos ao fim de nossa última unidade: tristeza por saber que estamos no fim de nossos conteúdos, mas alegria por saber que pudemos compartilhar momentos maravilhosos e compartilhar com todos os nossos conhecimentos.

Em nossa última unidade, tentamos demonstrar a todos os alunos o desenvolvimento de um sistema web seguindo o paradigma Orientado a Objetos ou não, pois temos diversos recursos, arquiteturas computacionais de desenvolvimento e processos que podem auxiliar em diversas fases do sistema a ser construído.

Além de compreender a relação entre o paradigma de programação OO com o PHP, é possível observar que os elementos que compõem esta unidade estão diretamente ligados ao *back-end* de um sistema web. Em todos os pontos apresentados, tivemos como objetivo fazer com que você, prezado(a) aluno(a), conseguisse ver algumas arquiteturas e componentes de desenvolvimento que podem facilitar o nosso dia a dia.

Um dos assuntos que podemos destacar como importantes para o seu conhecimento é o uso de alguns *frameworks* de programação WEB, os quais auxiliam e agilizam no desenvolvimento da aplicação. Detalhe interessante que pode ser destacado é o exercício da criatividade, que contribui muito com a qualidade do sistema final. Outro assunto, também, são os conceitos e aplicações de um *WebService* e o seu consumo por meio de uma API *RESTful*, que permite uma diferenciação em relação a outros projetos web que seguem padrões prontos.

O mais importante de se destacar, nesta unidade, é saber que existem práticas que são adotadas pelo mercado de trabalho, pois, além de torná-lo(a) um graduado(a) em Sistema para Internet, temos como objetivo prepará-lo(a) para o mercado de trabalho. Por isso, nesta unidade, trabalhamos com diversos conceitos que poderão ser requisitados a você, futuro(a) desenvolvedor(a) de sistemas.

Esperamos que tenham aproveitado ao máximo todo o conteúdo disponibilizado para você ao longo destas cinco unidades. Todo o material foi desenvolvido por nós pensando no seu melhor desenvolvimento pessoal, intelectual e pessoal. Abraço e até uma próxima oportunidade!



1. A programação para Web tem ganhado cada vez mais terreno. A maioria das aplicações desenvolvidas hoje é voltada para Web. Primeiro, porque as pessoas precisam conectar tecnologias, e as ferramentas precisam estar on-line para suprir essa necessidade. Depois, porque uma aplicação Web é independente da plataforma do usuário final, as aplicações Web rodam em servidores.

De tantas tecnologias e linguagens que podemos utilizar para desenvolver uma aplicação para Web, o PHP é uma delas. Podemos utilizar alguns *frameworks* para o desenvolvimento de aplicações Web, assim facilitando o processo. Com base nos nossos estudos, assinale a afirmativa que contenha apenas *frameworks* WEB.

- a) JavaScript, Zend, React, Pump, Visio.
 - b) Angular, CSS, JS, Bootstrap, Laravel.
 - c) Angular, React, Vue, Bootstrap, Laravel.
 - d) HTML5, PDO, Vue, React, Zend.
 - e) Zend, VsCode, Docker, CSS, JavaScript.
2. Ao desenvolver sistemas baseados em *Model View Controller* (MVC), os benefícios são a modularidade dos componentes; isolamento das funcionais umas das outras; tanto quanto possível, torna mais fácil para que o designer do aplicativo entenda e modifique cada unidade específica, sem ter que saber de tudo sobre as outras funcionalidades.

Com base em nossos estudos, dentro da arquitetura MVC, o controlador é responsável por:

- a) Solicitar as atualizações de modelo.
- b) Encapsular o estado da aplicação.
- c) Notificar a visão sobre as mudanças de estado.
- d) Selecionar as visões.
- e) Enviar eventos de usuário para o Modelo.



3. Com a rápida adoção do *Representational State Transfer* (REST), mais organizações de software expõem seus aplicativos como APIs da web *RESTful*, e os desenvolvedores de código de cliente integram APIs *RESTful* em seus aplicativos. Quando as APIs da web evoluem, os desenvolvedores de código do cliente precisam atualizar seus aplicativos para incorporar as alterações da API de acordo.

Ao analisarmos o protocolo SOAP, observamos que, além da linguagem XML, esse protocolo está associado à documentação:

- a) WSDL.
 - b) RSS.
 - c) WUDI.
 - d) ESRF.
 - e) SOUR.
4. A qualidade de um sistema pode ser afetada pela degradação das versões do software. Existem várias versões de grande número de pacotes de um sistema, podendo haver diferentes definições sobre sua qualidade ou número de recursos (em vez de sua data de desenvolvimento ou lançamento). Um método para organizar e armazenar seria aplicar o versionamento do sistema em fase de produção ou final.

Uma ferramenta web muito utilizado é o GitHub, mas, para fazer o seu uso adequado, são necessárias algumas ações. Para finalizar a criação do seu repositório, você pode adicionar algumas configurações iniciais importantes, tais como:

- a) Habilitar a permissão para que outros usuários possam acessar.
- b) Inclusão do arquivo de leitura (*Readme file*).
- c) Compartilhar arquivos iniciar via FTP.
- d) Configurar o Repositório para ignorar qualquer tipo de PULL e PUSH.
- e) Fazer um *Branch* de tudo que esteve na pasta de versionamento antes de finalizar o processo de criação.



5. A maioria dos desenvolvedores que atua na área já está acostumada ao termo *frameworks* quando nos referimos a alguns projetos de *front-end* como *Bootstrap* ou de *back-end* como *Zend* (que, agora, é Laminas), Laravel etc.

Todo(a) desenvolvedor(a) deverá, em algum momento de sua carreira, escolher alguns frameworks, assim como terá que decidir qual linguagem de programação, banco de dados e outras tecnologias utilizar para um projeto e para a sua vida profissional. A vantagem é que existem vários frameworks e para todos os tipos e portes de projeto, assim como linguagem de programação. Com base em seus conhecimentos adquiridos, bem como pesquisas realizadas por você, responda as principais perguntas sobre frameworks PHP:

- a) O que são *frameworks*?
- b) Quais são as características comuns entre *frameworks*?
- c) Quais critérios você levaria em consideração para escolher um para seu projeto?
- d) Quais as vantagens e desvantagens?



ANÁLISE COMPARATIVA DE FRAMEWORKS PARA DESENVOLVIMENTO FRONT-END EM APLICAÇÕES WEB

Atualmente, no mercado de desenvolvimento de sistemas, há uma necessidade extrema por codificação rápida e eficiente, tendo como foco a redução de custos e a excelência no produto final. O tempo de desenvolvimento tem ligação direta com o raciocínio lógico e criatividade do desenvolvedor, tornando assim o processo de quantificação de custos e de prazos extremamente difícil.

Magalhães e Silva (2013) afirmam que, “quando bem utilizados, os frameworks vêm a ser um facilitador no desenvolvimento, com ganho no tempo de desenvolvimento e no desempenho final”. Assim, a utilização de frameworks torna-se essencial no mercado de desenvolvimento de software, tendo impacto direto na produtividade e sucesso de um projeto de sistema, independentemente da plataforma e dispositivo.

O uso de frameworks para desenvolvimento de aplicações web pode ocorrer em projetos de pequeno, médio e grande porte, e através da análise de alguns fatores é possível verificar a sua aplicabilidade e viabilidade. Porém, devido à variedade e ao grande número de frameworks para desenvolvimento web, mais especificamente para desenvolvimento front-end, torna-se inviável a verificação e análise de todos os frameworks disponíveis, ou até mesmo dos mais populares e mais utilizados no mercado de desenvolvimento.

Muitas vezes, por falta de conhecimento sobre a existência, comodidade pela utilização de algum outro framework ou curtos prazos para entrega de projetos, os envolvidos no processo de análise de projeto e implementação do sistema podem acabar optando pelo uso de frameworks que não oferecem a melhor resolução e recursos para o determinado problema.

Fonte: Mattos, Kernitskei e Paula Junior ([2021], s.p.)⁴.



eu recomendo!



livro

Containers com Docker: do desenvolvimento à produção

Autor: Daniel Romero

Editora: Casa do Código

Sinopse: Dores de cabeça com configuração de ambiente, noites perdidas com versões inválidas de algum pacote e tempo perdido na instalação de pré-requisitos, sendo que tudo isso poderia ser facilmente automatizado, e nenhum dos inconvenientes precisaria acontecer. Neste livro, Daniel Romero mostrará como esses e outros problemas podem ser resolvidos com Docker, poderosa ferramenta que nos permite isolar e manter diferentes ambientes de forma simples e facilmente configurável. Aproveite o primeiro livro sobre Docker do Brasil.



livro

Frameworks para Desenvolvimento em PHP

Autor: DElton Luis Minetto

Editora: Novatec

Sinopse: os frameworks facilitam o desenvolvimento de software, permitindo que os programadores se ocupem mais com os requerimentos do software do que com os detalhes tediosos, de baixo nível do sistema. Com o uso de frameworks, os programadores voltam a ter o controle de seu tempo e de seus códigos-fonte, as tarefas repetitivas são minimizadas, os projetos são concluídos em menos tempo, os padrões são seguidos, e a programação volta a ser uma arte. Frameworks para Desenvolvimento em PHP mostra aos leitores diversos frameworks de desenvolvimento que agilizam o processo de criação e manutenção de aplicativos web. Cada framework é apresentado na prática por meio do desenvolvimento de um aplicativo, de modo que o leitor pode ver sua serventia em um caso real.





É com muita alegria e satisfação que chegamos ao fim de nossa jornada de estudos em Back End II. No decorrer do nosso livro, procuramos levar até você conteúdos relevantes que serão aplicados no futuro como, também, procuramos exemplificar ao máximo todos os conceitos aplicados. Esperamos ter esclarecido suas dúvidas e despertado em você a curiosidade e vontade de colocar o conteúdo em prática.

No decorrer de nossas unidades, pudemos compreender os aspectos sobre o paradigma Orientação a Objetos, assim como elaborar e explorar as oportunidades que esse paradigma pode nos proporcionar ao implementá-lo com a linguagem web PHP. Ao final de cada unidade, foram apresentadas a você implementações práticas para que fosse possível uma melhor absorção e assimilação de conteúdos.

Nossas duas primeiras unidades tiveram como proposta levar você a compreender como funciona o paradigma de Orientação a Objetos, suas características e possibilidades de uso, analisando e interpretando modelos UML para uma abstração do contexto e levando para a prática com programação php.

Na sequência, trabalhamos com a integração do desenvolvimento de códigos com o banco de dados MySQL, sendo possível analisar como os dados são manipulados no back end de uma aplicação que tem como base um SGBD relacional, relembramos os comandos SQL baseados em MySQL, que são 95% compatíveis com os demais SGBDs relacionais. A escolha em optar pelo relacional e não por um NoSQL está no fato de que os SGBDs relacionais ainda são os líderes de mercado e implementação de sistemas.



Foram bservados alguns modelos e ferramentas para auxiliar no desenvolvimento de sistemas web. A utilização de um framework faz poupar tempo, além de proporcionar uma dinâmica muito boa no código-fonte. O modelo MVC também se demonstrou uma ótima opção para termos uma organização e separação de funções em nosso código-fonte.

De modo geral, ficamos muito felizes em poder compartilhar com vocês um pouco de nossos conhecimentos, acreditamos que estamos formando profissionais de alta qualidade e desempenho, que utilizarão os conceitos apresentados. Se pudermos ajudar de alguma forma, estamos à sua disposição. Abraço e até a próxima oportunidade!

UNIDADE 1

BEZERRA, E. **Princípios de Análise e Projeto de Sistemas com UML**. 2. ed. Rio de Janeiro: Elsevier, 2007.

DALL'OGGIO, P. **PHP: Programando com Orientação a Objetos**. 2. ed. São Paulo Novatec Editora Ltd., 2009.

LÓSCIO, B. F.; OLIVEIRA, H. R. de; PONTES, J. C. de S. NoSQL no desenvolvimento de aplicações Web colaborativas. In: SIMPÓSIO BRASILEIRO DE SISTEMAS COLABORATIVOS, 8., 2011, Paraty. **Anais [...]**. Rio de Janeiro: SBC, 2011. Disponível em: https://www.researchgate.net/profile/Bernadette_Loscio/publication/268201466_NoSQL_no_desenvolvimento_de_aplicacoes_Web_colaborativas/links/576aa72008aef2a864d1ef8c/NoSQL-no-desenvolvimento-de-aplicacoes-Web-colaborativas.pdf. Acesso em: 03 set. 2020.

MORAIS JÚNIOR. PSR - Seguindo Padrões em PHP. **Tableless**, [s. l.], 21 abr. 2018. Disponível em: <https://tableless.com.br/psr-padrao-para-codigo-php/>. Acesso em: 9 nov. 2020.

O QUE FAZ um bom programador? **Eu Faço Programas**, 31 jul. 2019. Disponível em: <https://eufacoprogramas.com/o-que-faz-um-bom-programador/>. Acesso em: 7 nov. 2020.

PHP. **O básico**. [2020]. Disponível em: https://www.php.net/manual/pt_BR/language.oop5.basics.php. Acesso em: 7 nov. 2020.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson, 2011.

YOURDON, E. **Análise Estruturada Moderna**. Rio de Janeiro: Campus, 1990.

UNIDADE 2

BEZERRA, E. **Princípios de Análise e Projeto de Sistemas com UML**. 2. ed. Rio de Janeiro: Elsevier, 2007.

CHAVES, A. M.; SILVA, G. da. Proposta de uma arquitetura de software e funcionalidades para implementação de um ambiente integrado de desenvolvimento para a linguagem PHP. In: JORNADA CIENTÍFICA, 1., FIPA DO CEFET, 6., 2008, Bambuí. **Anais [...]**. [S.l.: s.n.], 2008. Disponível em: https://www.bambui.ifmg.edu.br/jornada_cientifica/str/artigos_aprovados/informatica/68-CO-5.pdf. Acesso em: 25 jan. 2021.

DALL'OGGIO, P. **PHP: Programando com Orientação a Objetos**. 2. ed. São Paulo Novatec Editora Ltd., 2009.

PHP. **Arrays**. [2020]. Disponível em: https://www.php.net/manual/pt_BR/language.types.array.php. Acesso em: 22 jan. 2020.

UNIDADE 3

DALL'OGGIO, P. **PHP: Programando com Orientação a Objetos**. 2. ed. São Paulo: Novatec Editora Ltd., 2009.

DEVMEDIA. Web Components na prática. **Front-end Magazine**, 5. ed., [2021]. Disponível em: <https://www.devmedia.com.br/web-components-na-pratica/32476#Components>. Acesso em: 26 jan. 2021.

GEROSA, M. A.; STEINMACHER, I. Componentes de Software para **Sistemas Colaborativos**. In: PIMENTEL, M.; FUKS, H. (orgs.). **Sistemas Colaborativos**. Rio de Janeiro: Elsevier, 2011.

GUEDES, G. T. A. **UML 2: uma abordagem prática**. 2. ed. São Paulo: Novatec, 2011.

MELO, A. C. **Desenvolvendo Aplicações com UML 2.0: do conceitual à implementação**. Rio de Janeiro: Brasport, 2004.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

UNIDADE 4

SOARES, J. L. P.; ROSA, M. V. C.; SILVA, F. A. Uma Ferramenta WEB para Auxiliar o Desenvolvimento de Relatórios em PDF para Sistemas em PHP. **Colloquium Exactarum**, v. 3, n. 2, p. 46-54, 2011.

TAKE Blip. **API: conceito, exemplos de uso e importância da integração para desenvolvedores**. 8 maio 2019. Disponível em: <https://take.net/blog/devs/api-conceito-e-exemplos>. Acesso em: 27 jan. 2021.

UNIDADE 5

ANDERSON, C. Docker [software engineering]. **IEEE Software**, v. 32, n. 3, p. 102-c3, 2015.

DOCKER. **What is a Container?** A standardized unit of software. [2021]. Disponível em: <https://www.docker.com/resources/what-container>. Acesso em: 28 jan. 2021.

JOSUTTIS, N. M. **SOA in Practice**. The Art of Distributed System Design. Sebastopol: O'Reilly Media Inc., 2007. Disponível em: <http://index-of.co.uk/Misc/O'Reilly%20SOA%20in%20Practice.pdf>. Acesso em: 28 jan. 2021.

PREETH, E. N. et al. Evaluation of Docker Containers Based on Hardware Utilization. In: INTERNATIONAL CONFERENCE ON CONTROL COMMUNICATION & COMPUTING, 3., 2015, Trivandrum. **Proceedings** [...]. [S.l.: s.n.], 2015. p. 697-700.

PRESSMAN, R. S.; MAXIM, B. **Engenharia de Software**. Uma abordagem profissional. 8. ed. Porto Alegre: McGraw Hill Brasil, 2016.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

W3C. **World Wide Web Consortium**. [2021]. Disponível em: <http://www.w3.org/>. Acesso em: 28 jan. 2021.

Referências on-line:

¹Em: <https://angular.io/>. Acesso em: 28 jan. 2021.

²Em: <https://pt-br.reactjs.org/>. Acesso em: 28 jan. 2021.

³Em: <https://br.vuejs.org/v2/guide/>. Acesso em: 28 jan. 2021.

⁴Em: https://www.academia.edu/42913917/An%C3%A1lise_Comparativa_de_Frameworks_para_Desenvolvimento_Front_End_em_Aplica%C3%A7%C3%B5es_Web?auto=download. Acesso em: 3 fev. 2021.

UNIDADE 1

1. C.
2. B.
3. A.
4. A visibilidade de atributos e métodos serve para proteger os atributos e métodos de interferência externa, indicando como cada um fica visível para outros objetos. Existem três tipos de visibilidade: 1) público: todos os objetos podem ver e manipular; 2) protegido: apenas o próprio objeto e as subclasses podem ver e manipular; 3) privado: apenas o próprio objeto pode ver e manipular.
5. Gato é o nome da classe; andar, correr, pular e comer são métodos, sendo que, para comer, é preciso informar a ração (provavelmente, a quantidade em gramas, por isso, float); idade é o atributo. Como sabemos que o gato tem dois anos, é um gato real, por isso, instanciado. Nesse caso, foi atribuído 2 ao objeto gato.

Gato
- idade : int = 2
+ andar() : void + correr() : void + pular() : void + comer (ração : float) : void

UNIDADE 2

1. B.
2. C.
3. D.
4. A.
5. Privado (private): os atributos e métodos da classe não são visíveis a nenhuma classe externa. São somente visíveis dentro da classe.
6. Para não repetir o que está implementado na classe mãe, em PHP, usamos a palavra parent para invocar o método, usando o operador de referência a métodos de classe (::).

```
public function imprimir(){
    parent::imprimir();
}
```

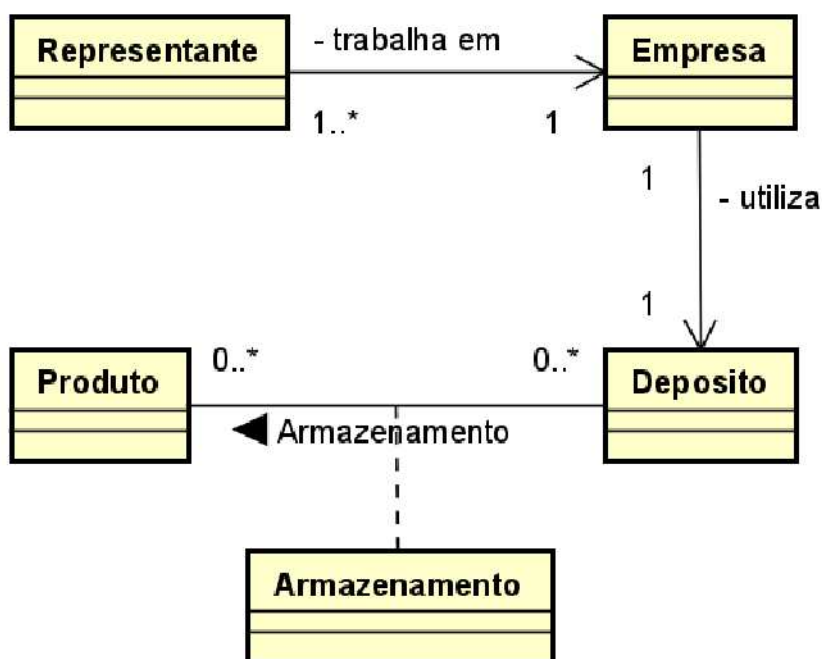
UNIDADE 3

1. B.
2. C.
3. B.
4. E.

Fatal error: Declaration of Teste::teste(\$n1) must be compatible with PrimeiraInterface::teste()

O método a ser implementado deve ter a mesma assinatura.

5.



É fundamental, para considerar totalmente correto, que o aluno faça a classe associativa Armazenamento, pois é ela que garante saber o representante associado a um produto.

6. Um produto pode participar de 0, 1 ou várias vendas; uma venda é composta por, no mínimo, um produto ou vários; item armazena a quantidade de produto e o valor unitário cobrado na venda.

UNIDADE 4

1. A.
2. A.
3. B.
4. E.
5. DDL – Linguagem de Definição de Dados.

DML – Data Manipulation Language, ou Linguagem de Manipulação de Dados.

DQL – Data Query Language, ou Linguagem de Consulta de Dados.

DCL – Data Control Language, ou Linguagem de Controle de Dados.

DTL – Data Transaction Language, ou Linguagem de Transação de Dados.

6. INSERT INTO carro (modelo, marca, cor, ano_fabricacao) VALUES ("Golf", "Volkswagen", "Prata", 2017);

DML juntamente com insert faz uso do comando INSERT INTO nome_da_tabela (coluna1, coluna2) VALUES ('dado1', 'dado2')

UNIDADE 5

1. C.
2. D.
3. A.
4. B.
5. Em uma busca rápida na web, é possível encontrar diversas definições para framework, algumas simples, outras mais elaboradas, mas o ponto comum entre todas é a reusabilidade.

Assim, um framework tem como principal objetivo resolver problemas recorrentes com uma abordagem genérica, permitindo ao desenvolvedor focar seus esforços na resolução do problema em si, e não ficar reescrevendo software. Você pode se perguntar: então, o framework é uma biblioteca?

Bem, quase isso. Pode-se dizer que é um conjunto de bibliotecas ou componentes que são usados para criar uma base em que sua aplicação será construída. Os frameworks ajudam no desenvolvimento rápido e seguro de aplicações, mas é recomendável estudar antes a tecnologia em que ela é desenvolvida.

Possuindo o conhecimento das tecnologias do framework, é possível fazer suas próprias modificações para que o framework possa atender todas as necessidades do desenvolvedor.

Dentre as vantagens: redução de custos; redução de time-to-market; maximização de reuso (análise, design, código, testes); melhor consistência e compatibilidade entre aplicações; alavancagem do conhecimento de especialistas.

Desvantagens, no entanto, são poucas, algumas esperadas no início, e outras passíveis de resolução, como: curva de aprendizagem e constante atualização; benefícios são colhidos em longo prazo; produtividade é vista no longo prazo; vício e dependência; menor esforço e interesse em conhecer a linguagem.

Alguns critérios para escolha de um: popularidade e tamanho da comunidade; filosofia; sustentabilidade; suporte; técnica; segurança; documentação; licença; disponibilidade de recursos no mercado; facilidade na experimentação/curva de aprendizado.





anotações

Handwriting practice lines consisting of horizontal dotted lines on a white background.